BACKTRACKING

46. Permutations
47. Permutations II
78. Subsets
90. Subsets II
39. Combination Sum
40. Combination Sum II
77. Combinations
79. Word Search
131. Palindrome Partitioning
51. N-Queens
52. N-Queens II
17. Letter Combinations of a Phone Number

1D DYNAMIC PROGRAMMING

70. Climbing Stairs
746. Min Cost Climbing Stairs
198. House Robber
213. House Robber II
53. Maximum Subarray (Kadane's)
152. Maximum Product Subarray
322. Coin Change
518. Coin Change II
300. Longest Increasing Subsequence
139. Word Break
91. Decode Ways
416. Partition Equal Subset Sum
343. Integer Break

2D DYNAMIC PROGRAMMING

62. Unique Paths
63. Unique Paths II
64. Minimum Path Sum
1143. Longest Common Subsequence
72. Edit Distance
5. Longest Palindromic Substring
516. Longest Palindromic Subsequence
221. Maximal Square
97. Interleaving String
312. Burst Balloons
115. Distinct Subsequences
10. Regular Expression Matching

TRIE (Prefix Tree)

1. 208. Implement Trie (Prefix Tree)
2. 211. Design Add and Search Words Data Structure
3. 212. Word Search II
4. 336. Palindrome Pairs
5. 648. Replace Words
6. 677. Map Sum Pairs
7. 720. Longest Word in Dictionary
8. 1268. Search Suggestions System
9. 745. Prefix and Suffix Search
10. 1032. Stream of Characters

UNION-FIND (Disjoint Set)

547. Number of Provinces
323. Number of Connected Components
261. Graph Valid Tree
684. Redundant Connection
685. Redundant Connection II
721. Accounts Merge
1101. The Earliest Moment When Everyone Become
959. Regions Cut By Slashes
1202. Smallest String With Swaps
305. Number of Islands II

BIT MANIPULATION

136. Single Number
191. Number of 1 Bits
338. Counting Bits
190. Reverse Bits
268. Missing Number
371. Sum of Two Integers
78. Subsets
201. Bitwise AND of Numbers Range
137. Single Number II
260. Single Number III

INTERVALS

1. 56. Merge Intervals
2. 57. Insert Interval
3. 252. Meeting Rooms
4. 253. Meeting Rooms II
5. 435. Non-overlapping Intervals
6. 452. Minimum Number of Arrows to Burst Balloons
7. 986. Interval List Intersections
8. 759. Employee Free Time
9. 228. Summary Ranges
10. 715. Range Module

GRAPHS

200. Number of Islands
133. Clone Graph
695. Max Area of Island
417. Pacific Atlantic Water Flow
994. Rotting Oranges
207. Course Schedule
210. Course Schedule II
323. Number of Connected Components
61. Graph Valid Tree
127. Word Ladder
841. Keys and Rooms
547. Number of Provinces
1091. Shortest Path in Binary Matrix
130. Surrounded Regions

DESIGN & SYSTEM DESIGN

1. 146. LRU Cache
2. 460. LFU Cache
3. 155. Min Stack
4. 232. Implement Queue using Stacks
5. 295. Find Median from Data Stream
6. 380. Insert Delete GetRandom O(1)
7. 297. Serialize and Deserialize Binary Tree
8. 348. Design Tic-Tac-Toe
9. 355. Design Twitter
10. 588. Design In-Memory File System

BACKTRACKING

46. Permutations

Given an array nums of distinct integers, return all the possible permutations. You can return the answer in **any order**.

Example 1:

Input: nums = [1,2,3]
Output: [[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]

Example 2:

Input: nums = [0,1]
Output: [[0,1],[1,0]]

Example 3:

Input: nums = [1]
Output: [[1]]

Solution 1: Backtracking (Swap)

```
def permute(nums):
    def backtrack(start):
        if start == len(nums):
            result.append(nums[:])
            return

        for i in range(start, len(nums)):
            nums[start], nums[i] = nums[i], nums[start]
            backtrack(start + 1)
            nums[start], nums[i] = nums[i], nums[start]

    result = []
    backtrack(0)
    return result
```

Time Complexity: O(n * n!) | Space Complexity: O(n!)
Idea: Fix elements one by one by swapping, backtrack.

Solution 2: Backtracking (Used Array)

```
def permute(nums):
    def backtrack(path):
        if len(path) == len(nums):
            result.append(path[:])
            return

        for num in nums:
            if num not in path:  # O(n) check
                path.append(num)
                backtrack(path)
                path.pop()

    result = []
    backtrack([])
    return result
```

Time Complexity: O(n * n!) | Space Complexity: O(n!)
Idea: Build permutations incrementally using used array.

Solution 3: Backtracking with Boolean Array

```
def permute(nums):
    def backtrack(path, used):
        if len(path) == len(nums):
            result.append(path[:])
            return

        for i in range(len(nums)):
            if not used[i]:
                used[i] = True
                path.append(nums[i])
                backtrack(path, used)
                path.pop()
                used[i] = False

    result = []
    backtrack([], [False] * len(nums))
    return result
```

Time Complexity: O(n * n!) | Space Complexity: O(n!)
Idea: Track used indices with boolean array for O(1) lookup.

Solution 4: Heap's Algorithm

```
def permute(nums):
    def generate(k, arr):
        if k == 1:
            result.append(arr[:])
            return

        generate(k - 1, arr)

        for i in range(k - 1):
            if k % 2 == 0:
                arr[i], arr[k-1] = arr[k-1], arr[i]
            else:
                arr[0], arr[k-1] = arr[k-1], arr[0]
            generate(k - 1, arr)

    result = []
    generate(len(nums), nums[:])
    return result
```

Time Complexity: O(n!) | Space Complexity: O(n!)
Idea: Efficient algorithm that generates permutations by swapping.

Solution 5: Iterative (Next Permutation)

```
def permute(nums):
    def next_permutation(arr):
        # Find first decreasing element from right
        i = len(arr) - 2
        while i >= 0 and arr[i] >= arr[i + 1]:
            i -= 1

        if i >= 0:
            # Find element just larger than arr[i]
            j = len(arr) - 1
            while arr[j] <= arr[i]:
                j -= 1
            arr[i], arr[j] = arr[j], arr[i]

        # Reverse suffix
        left, right = i + 1, len(arr) - 1
        while left < right:
            arr[left], arr[right] = arr[right], arr[left]
            left += 1
            right -= 1
        return arr

    result = []
    nums.sort()
    result.append(nums[:])
    while True:
        nums = next_permutation(nums[:])
        if nums == result[0]:
            break
        result.append(nums[:])

    return result
```

Time Complexity: O(n * n!) | Space Complexity: O(n!)
Idea: Generate permutations in lexicographic order.

---

47. Permutations II

Given a collection of numbers, nums, that might contain duplicates, return *all possible unique permutations* ***in any order***.

Example 1:

Input: nums = [1,1,2]
Output:
[[1,1,2],
 [1,2,1],
 [2,1,1]]

Example 2:

Input: nums = [1,2,3]
Output: [[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]

Solution 1: Backtracking with Sorting & Skipping

```
def permuteUnique(nums):
    def backtrack(path, used):
        if len(path) == len(nums):
            result.append(path[:])
            return

        for i in range(len(nums)):
            if used[i]:
                continue
            # Skip duplicates: if same as previous and previous not used
            if i > 0 and nums[i] == nums[i-1] and not used[i-1]:
                continue

            used[i] = True
            path.append(nums[i])
            backtrack(path, used)
            path.pop()
            used[i] = False

    nums.sort()
    result = []
    backtrack([], [False] * len(nums))
    return result
```

Time Complexity: O(n * n!) | Space Complexity: O(n!)
Idea: Sort first, skip duplicates when previous identical element not used.

Solution 2: Backtracking with Counter

```
def permuteUnique(nums):
    def backtrack(path, counter):
        if len(path) == len(nums):
            result.append(path[:])
            return

        for num in counter:
            if counter[num] > 0:
                counter[num] -= 1
                path.append(num)
                backtrack(path, counter)
                path.pop()
                counter[num] += 1

    from collections import Counter
    result = []
    backtrack([], Counter(nums))
    return result
```

Time Complexity: O(n * n!) | Space Complexity: O(n!)
Idea: Use counter to track remaining counts of each number.

Solution 3: Iterative with Set

```
def permuteUnique(nums):
    result = [[]]

    for num in nums:
        new_result = []
        for perm in result:
            # Insert num at every possible position
            for i in range(len(perm) + 1):
                new_result.append(perm[:i] + [num] + perm[i:])
                # Break if we find duplicate to avoid inserting again
                if i < len(perm) and perm[i] == num:
                    break
        result = new_result

    return result
```

Time Complexity: O(n * n!) | Space Complexity: O(n!)
Idea: Build permutations iteratively, break early to avoid duplicates.

Solution 4: Using itertools (Python built-in)

```
def permuteUnique(nums):
    from itertools import permutations
    return list(set(permutations(nums, len(nums))))
```

Time Complexity: O(n * n!) | Space Complexity: O(n!)
Idea: Use Python's built-in permutations and filter duplicates.

Solution 5: BFS Level Order

```
def permuteUnique(nums):
    from collections import deque

    nums.sort()
    queue = deque([([], nums)])
    result = []

    while queue:
        perm, remaining = queue.popleft()

        if not remaining:
            result.append(perm)
            continue

        for i in range(len(remaining)):
            if i > 0 and remaining[i] == remaining[i-1]:
                continue
            queue.append((perm + [remaining[i]], remaining[:i]
+ remaining[i+1:]))

    return result
```

Time Complexity: O(n * n!) | Space Complexity: O(n!)
Idea: BFS approach, process each level of permutation building.

78. Subsets

Given an integer array nums of **unique** elements, return *all possible subsets (the power set)*.

The solution set **must not** contain duplicate subsets. Return the solution in **any order**.

Example 1:

Input: nums = [1,2,3]
Output: [[],[1],[2],[1,2],[3],[1,3],[2,3],[1,2,3]]

Example 2:

Input: nums = [0]
Output: [[],[0]]

Solution 1: Backtracking

```
def subsets(nums):
    def backtrack(start, path):
        result.append(path[:])

        for i in range(start, len(nums)):
            path.append(nums[i])
            backtrack(i + 1, path)
            path.pop()

    result = []
    backtrack(0, [])
    return result
```

Time Complexity: O(n * 2^n) | Space Complexity: O(n * 2^n)
Idea: At each step, either include or exclude each element.

Solution 2: Bit Manipulation

```
def subsets(nums):
    n = len(nums)
    result = []

    for mask in range(1 << n):
        subset = []
        for i in range(n):
            if mask & (1 << i):
                subset.append(nums[i])
        result.append(subset)

    return result
```

Time Complexity: O(n * 2^n) | Space Complexity: O(n * 2^n)
Idea: Each subset corresponds to a bitmask of length n.

Solution 3: Iterative

```
def subsets(nums):
    result = [[]]

    for num in nums:
        result += [subset + [num] for subset in result]

    return result
```

Time Complexity: O(n * 2^n) | Space Complexity: O(n * 2^n)
Idea: Start with empty set, for each num, add to all existing subsets.

Solution 4: BFS/Queue

```
def subsets(nums):
    from collections import deque

    result = []
    queue = deque([([], 0)])  # (current subset, start index)

    while queue:
        subset, start = queue.popleft()
        result.append(subset)

        for i in range(start, len(nums)):
            queue.append((subset + [nums[i]], i + 1))

    return result
```

Time Complexity: O(n * 2^n) | Space Complexity: O(n * 2^n)
Idea: BFS exploring all subset possibilities.

Solution 5: Mathematical (Power Set)

```
def subsets(nums):
    def generate_subsets(index):
        if index == len(nums):
            return [[]]

        # Get all subsets without current element
        subsets_without = generate_subsets(index + 1)

        # Add current element to each subset
        subsets_with = []
        for subset in subsets_without:
            subsets_with.append(subset + [nums[index]])

        return subsets_without + subsets_with

    return generate_subsets(0)
```

Time Complexity: O(n * 2^n) | Space Complexity: O(n * 2^n)
Idea: Recursive definition: subsets(nums) = subsets(nums[1:]) ∪ {x ∪ nums[0] for x in subsets(nums[1:])}.

90. Subsets II

Given an integer array nums that may contain duplicates, return *all possible subsets (the power set)*.

The solution set **must not** contain duplicate subsets. Return the solution in **any order**.

Example 1:

Input: nums = [1,2,2]
Output: [[],[1],[1,2],[1,2,2],[2],[2,2]]

Example 2:

Input: nums = [0]
Output: [[],[0]]

Solution 1: Backtracking with Sorting

```
def subsetsWithDup(nums):
    def backtrack(start, path):
        result.append(path[:])

        for i in range(start, len(nums)):
            # Skip duplicates
            if i > start and nums[i] == nums[i-1]:
                continue

            path.append(nums[i])
            backtrack(i + 1, path)
            path.pop()

    nums.sort()
    result = []
    backtrack(0, [])
    return result
```

Time Complexity: O(n * 2^n) | Space Complexity: O(n * 2^n)
Idea: Sort first, skip duplicates when not the first occurrence at current level.

Solution 2: Iterative with Duplicate Handling

```
def subsetsWithDup(nums):
    nums.sort()
    result = [[]]
    start = 0

    for i in range(len(nums)):
        # If duplicate, only add to subsets created in previous step
        if i > 0 and nums[i] == nums[i-1]:
            start = size
        else:
            start = 0
        size = len(result)
        for j in range(start, size):
            result.append(result[j] + [nums[i]])

    return result
```

Time Complexity: O(n * 2^n) | Space Complexity: O(n * 2^n)
Idea: Track where to start adding duplicates to avoid repetition.

Solution 3: Bit Manipulation with Set

```
def subsetsWithDup(nums):
    nums.sort()
    n = len(nums)
    result_set = set()

    for mask in range(1 << n):
        subset = []
        for i in range(n):
            if mask & (1 << i):
                subset.append(nums[i])
        result_set.add(tuple(subset))

    return [list(subset) for subset in result_set]
```

Time Complexity: O(n * 2^n) | Space Complexity: O(n * 2^n)
Idea: Generate all subsets via bitmask, use set to filter duplicates.

Solution 4: BFS with Level Tracking

```
def subsetsWithDup(nums):
    from collections import deque

    nums.sort()
    result = []
    queue = deque([([], 0)])

    while queue:
        subset, start = queue.popleft()
        result.append(subset)

        for i in range(start, len(nums)):
            # Skip duplicates except for the first occurrence at
this level
            if i > start and nums[i] == nums[i-1]:
                continue
            queue.append((subset + [nums[i]], i + 1))

    return result
```

Time Complexity: O(n * 2^n) | Space Complexity: O(n * 2^n)
Idea: BFS with duplicate skipping at each level.

Solution 5: Counter-Based Backtracking

```
def subsetsWithDup(nums):
    from collections import Counter

    def backtrack(index, path):
        if index == len(unique_nums):
            result.append(path[:])
            return

        num, count = unique_nums[index]

        # Include 0 to count copies of current number
        for i in range(count + 1):
            # Add i copies of current number
            for _ in range(i):
                path.append(num)

            backtrack(index + 1, path)

            # Remove i copies of current number
            for _ in range(i):
                path.pop()

    counter = Counter(nums)
    unique_nums = list(counter.items())
    result = []
    backtrack(0, [])
    return result
```

Time Complexity: O(n * 2^n) | Space Complexity: O(n * 2^n)
Idea: Use counter, for each number decide how many copies to include (0 to count).

39. Combination Sum

Given an array of **distinct** integers candidates and a target integer target, return *a list of all **unique combinations** of* candidates *where the chosen numbers sum to* target. You may return the combinations in **any order**.

The **same** number may be chosen from candidates an **unlimited number of times**. Two combinations are unique if the frequency of at least one of the chosen numbers is different.

The test cases are generated such that the number of unique combinations that sum up to target is less than 150 combinations for the given input.

Example 1:

Input: candidates = [2,3,6,7], target = 7
Output: [[2,2,3],[7]]
Explanation:
2 and 3 are candidates, and 2 + 2 + 3 = 7. Note that 2 can be used multiple times.
7 is a candidate, and 7 = 7.
These are the only two combinations.

Example 2:

Input: candidates = [2,3,5], target = 8
Output: [[2,2,2,2],[2,3,3],[3,5]]

Example 3:

Input: candidates = [2], target = 1
Output: []

Solution 1: Standard Backtracking

```
def combinationSum(candidates, target):
    def backtrack(start, path, remaining):
        if remaining == 0:
            result.append(path[:])
            return
        if remaining < 0:
            return

        for i in range(start, len(candidates)):
            path.append(candidates[i])
            backtrack(i, path, remaining - candidates[i])
            path.pop()

    result = []
    backtrack(0, [], target)
    return result
```

Time Complexity: O(n^(target/min)) | Space Complexity: O(target/min) for recursion depth

Solution 2: Backtracking with Sorting Pruning

```
def combinationSum(candidates, target):
    def backtrack(start, path, remaining):
        if remaining == 0:
            result.append(path[:])
            return

        for i in range(start, len(candidates)):
            if candidates[i] > remaining:  # prune early
                break
            path.append(candidates[i])
            backtrack(i, path, remaining - candidates[i])
            path.pop()

    candidates.sort()
    result = []
    backtrack(0, [], target)
    return result
```

Time Complexity: O(n^(target/min)) | Space Complexity: O(target/min)
Idea: Sort candidates to enable early pruning when candidate > remaining.

Solution 3: Dynamic Programming (Bottom-up)

```
def combinationSum(candidates, target):
    dp = [[] for _ in range(target + 1)]
    dp[0] = [[]]  # base case: empty combination for sum 0

    for num in candidates:
        for t in range(num, target + 1):
            for comb in dp[t - num]:
                dp[t].append(comb + [num])

    return dp[target]
```

Time Complexity: O(n * target * k) where k is number of combinations | Space Complexity: O(target * k)
Idea: DP approach building combinations for each sum from 0 to target.

Solution 4: BFS/Queue Approach

```
def combinationSum(candidates, target):
    from collections import deque

    result = []
    queue = deque([([], 0, 0)])  # (combination, sum, start_index)

    while queue:
        comb, total, start = queue.popleft()

        if total == target:
            result.append(comb)
            continue
        if total > target:
            continue

        for i in range(start, len(candidates)):
            queue.append((comb + [candidates[i]], total + candidates[i], i))

    return result
```

Time Complexity: O(n^(target/min)) | Space Complexity: O(n^(target/min)) for queue
Idea: BFS exploring all combination possibilities level by level.

Solution 5: Memoized DFS

```
def combinationSum(candidates, target):
    from functools import lru_cache

    @lru_cache(None)
    def dfs(start, remaining):
        if remaining == 0:
            return [[]]
        if remaining < 0 or start >= len(candidates):
            return []

        result = []
        # Include current candidate (can reuse)
        for comb in dfs(start, remaining - candidates[start]):
            result.append([candidates[start]] + comb)

        # Skip current candidate
        result.extend(dfs(start + 1, remaining))

        return result

    return dfs(0, target)
```

Time Complexity: O(n * target) | Space Complexity: O(target)
Idea: Memoization reduces repeated calculations of same (start, remaining) states.

40. Combination Sum II

Given a collection of candidate numbers (candidates) and a target number (target), find all unique combinations in candidates where the candidate numbers sum to target.

Each number in candidates may only be used **once** in the combination.

Note: The solution set must not contain duplicate combinations.

Example 1:

Input: candidates = [10,1,2,7,6,1,5], target = 8
Output:
[
[1,1,6],
[1,2,5],
[1,7],
[2,6]
]

Example 2:

Input: candidates = [2,5,2,1,2], target = 5
Output:
[
[1,2,2],
[5]
]

Solution 1: Backtracking with Sorting & Skipping

```
def combinationSum2(candidates, target):
    def backtrack(start, path, remaining):
        if remaining == 0:
            result.append(path[:])
            return
        if remaining < 0:
            return

        for i in range(start, len(candidates)):
            # Skip duplicates: same as previous and not first in this level
            if i > start and candidates[i] == candidates[i-1]:
                continue

            if candidates[i] > remaining:  # prune
                break

            path.append(candidates[i])
            backtrack(i + 1, path, remaining - candidates[i])  # i+1: use once
            path.pop()

    candidates.sort()
    result = []
    backtrack(0, [], target)
    return result
```

Time Complexity: $O(2^n)$ | Space Complexity: O(n) for recursion
Idea: Sort to group duplicates, skip duplicates at same level, each element used once.

Solution 2: Counter-Based Backtracking

```
def combinationSum2(candidates, target):
    from collections import Counter

    def backtrack(index, path, remaining):
        if remaining == 0:
            result.append(path[:])
            return
        if remaining < 0 or index == len(unique_nums):
            return

        num, count = unique_nums[index]

        # Try 0 to count copies of current number
        max_copies = min(count, remaining // num)
        for i in range(max_copies + 1):
            # Add i copies
            for _ in range(i):
                path.append(num)

            backtrack(index + 1, path, remaining - i * num)

            # Remove i copies
            for _ in range(i):
                path.pop()

    counter = Counter(candidates)
    unique_nums = list(counter.items())
    result = []
    backtrack(0, [], target)
    return result
```

Time Complexity: $O(2^n)$ | Space Complexity: O(n)
Idea: Use counter to handle duplicates elegantly, decide how many copies of each number to include.

Solution 3: DP with Set for Uniqueness

```
def combinationSum2(candidates, target):
    candidates.sort()
    dp = [set() for _ in range(target + 1)]
    dp[0].add(())  # empty tuple for sum 0

    for num in candidates:
        for t in range(target, num - 1, -1):  # backwards to
avoid reuse
            for comb in dp[t - num]:
                dp[t].add(comb + (num,))

    return [list(comb) for comb in dp[target]]
```

Time Complexity: O(n * target * k) | Space Complexity: O(target * k)
Idea: DP with sets to ensure uniqueness, iterate backwards to use each element once.

Solution 4: Bitmask with Sorting

```
def combinationSum2(candidates, target):
    candidates.sort()
    n = len(candidates)
    result_set = set()

    for mask in range(1 << n):
        total = 0
        comb = []
        valid = True

        for i in range(n):
            if mask & (1 << i):
                # Skip if duplicate and previous not included
                if i > 0 and candidates[i] == candidates[i-1] and
not (mask & (1 << (i-1))):
                    valid = False
                    break
                total += candidates[i]
                comb.append(candidates[i])

        if valid and total == target:
            result_set.add(tuple(comb))

    return [list(comb) for comb in result_set]
```

Time Complexity: $O(2^n * n)$ | Space Complexity: $O(2^n)$
Idea: Generate all subsets via bitmask, filter valid combinations with sum target.

Solution 5: Two-phase Approach

```
def combinationSum2(candidates, target):

    # Phase 1: Count frequencies
    counter = defaultdict(int)
    for num in candidates:
        counter[num] += 1

    # Phase 2: Generate combinations
    unique_nums = sorted(counter.keys())
    result = []

    def dfs(index, path, remaining):
        if remaining == 0:
            result.append(path[:])
            return
        if index >= len(unique_nums) or remaining < 0:
            return

        num = unique_nums[index]
        max_use = min(counter[num], remaining // num)

        # Try using 0 to max_use copies of current number
        for use in range(max_use + 1):
            for _ in range(use):
                path.append(num)
            dfs(index + 1, path, remaining - use * num)
            for _ in range(use):
                path.pop()

    dfs(0, [], target)
    return result
```

Time Complexity: $O(2^n)$ | Space Complexity: O(n)
Idea: Separate counting and combination generation for clarity.

77. Combinations

Given two integers n and k, return *all possible combinations of* k *numbers chosen from the range* [1, n].

You may return the answer in **any order**.

Example 1:

Input: n = 4, k = 2
Output: [[1,2],[1,3],[1,4],[2,3],[2,4],[3,4]]
Explanation: There are 4 choose 2 = 6 total combinations. Note that combinations are unordered, i.e., [1,2] and [2,1] are considered to be the same combination.

Example 2:

Input: n = 1, k = 1
Output: [[1]]
Explanation: There is 1 choose 1 = 1 total combination.

Solution 1: Standard Backtracking

```
def combine(n, k):
    def backtrack(start, path):
        if len(path) == k:
            result.append(path[:])
            return

        for i in range(start, n + 1):
            path.append(i)
            backtrack(i + 1, path)
            path.pop()

    result = []
    backtrack(1, [])
    return result
```

Time Complexity: O(C(n,k) * k) | Space Complexity: O(k) for recursion
Idea: Choose numbers in increasing order to avoid duplicates.

Solution 2: Backtracking with Pruning

```
def combine(n, k):
    def backtrack(start, path):
        if len(path) == k:
            result.append(path[:])
            return

        # Prune: need k-len(path) more elements, max available: n-start+1
        max_needed = k - len(path)
        max_available = n - start + 1

        for i in range(start, n - max_needed + 2):  # +2 for
inclusive range
            path.append(i)
            backtrack(i + 1, path)
            path.pop()

    result = []
    backtrack(1, [])
    return result
```

Time Complexity: O(C(n,k) * k) | Space Complexity: O(k)
Idea: Prune branches that cannot complete k elements.

Solution 3: Iterative (Lexicographic)

```
def combine(n, k):
    # Initialize first combination
    nums = list(range(1, k + 1)) + [n + 1]

    result = []
    j = 0

    while j < k:
        result.append(nums[:k])

        j = 0
        # Find first position that can be incremented
        while j < k and nums[j + 1] == nums[j] + 1:
            nums[j] = j + 1
            j += 1

        if j < k:
            nums[j] += 1

    return result
```

Time Complexity: O(C(n,k) * k) | Space Complexity: O(k)
Idea: Generate combinations in lexicographic order algorithmically.

Solution 4: Using itertools

```
def combine(n, k):
    from itertools import combinations
    return list(combinations(range(1, n + 1), k))
```

Time Complexity: O(C(n,k) * k) | Space Complexity: O(C(n,k) * k)
Idea: Python built-in combinations generator.

Solution 5: DP (Combination Recurrence)

```
def combine(n, k):
    from functools import lru_cache

    @lru_cache(None)
    def dfs(start, k_remaining):
        if k_remaining == 0:
            return [[]]
        if start > n:
            return []

        # Combinations including start
        with_start = []
        for comb in dfs(start + 1, k_remaining - 1):
            with_start.append([start] + comb)

        # Combinations excluding start
        without_start = dfs(start + 1, k_remaining)

        return with_start + without_start

    return dfs(1, k)
```

Time Complexity: O(C(n,k) * k) | Space Complexity: O(C(n,k) * k)
Idea: DP using combination recurrence: C(n,k) = C(n-1,k-1) ∪ C(n-1,k).

Solution 6: Bitmask Generation

```
def combine(n, k):
    result = []

    for mask in range(1 << n):
        # Count bits
        if bin(mask).count('1') != k:
            continue

        # Convert bitmask to combination
        comb = []
        for i in range(n):
            if mask & (1 << i):
                comb.append(i + 1)

        result.append(comb)
    return result
```

Time Complexity: O(2^n * n) | Space Complexity: O(C(n,k) * k)
Idea: Generate all subsets of size k via bitmask filtering.

---

79. Word Search

Given an m x n grid of characters board and a string word, return true *if* word *exists in the grid*.

The word can be constructed from letters of sequentially adjacent cells, where adjacent cells are horizontally or vertically neighboring. The same letter cell may not be used more than once.

Example 1:

A	B	C	E
S	F	C	S
A	D	E	E

Input: board = [["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]], word = "ABCCED"
Output: true

Example 2:

A	B	C	E
S	F	C	S
A	D	E	E

Input: board = [["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]], word = "SEE"
Output: true

Example 3:

A	B	C	E
S	F	C	S
A	D	E	E

Input: board = [["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]], word = "ABCB"
Output: false

Solution 1: Standard Backtracking

```
def exist(board, word):
    def backtrack(i, j, index):
        if index == len(word):
            return True
        if i < 0 or i >= m or j < 0 or j >= n or board[i][j] !=
word[index]:
            return False

        # Mark as visited
        temp = board[i][j]
        board[i][j] = '#'

        # Explore 4 directions
        for di, dj in [(0,1), (1,0), (0,-1), (-1,0)]:
            if backtrack(i + di, j + dj, index + 1):
                board[i][j] = temp  # restore before return
                return True

        # Backtrack
        board[i][j] = temp
        return False

    m, n = len(board), len(board[0])

    # Optimization: check if all characters exist
    from collections import Counter
    board_counts = Counter(c for row in board for c in row)
    word_counts = Counter(word)
    for c, count in word_counts.items():
        if board_counts[c] < count:
            return False

    # Start from each cell
    for i in range(m):
        for j in range(n):
            if backtrack(i, j, 0):
                return True

    return False
```

Time Complexity: O(m*n * 3^L) where L = len(word) | Space Complexity: O(L)
Idea: DFS from each cell, backtrack with visited marking, prune with character count check.

Solution 2: Backtracking with Visited Set

```
def exist(board, word):
    def backtrack(i, j, index, visited):
        if index == len(word):
            return True
        if (i, j) in visited or i < 0 or i >= m or j < 0 or j >= n or
board[i][j] != word[index]:
            return False

        visited.add((i, j))

        for di, dj in [(0,1), (1,0), (0,-1), (-1,0)]:
            if backtrack(i + di, j + dj, index + 1, visited):
                return True

        visited.remove((i, j))
        return False

    m, n = len(board), len(board[0])

    for i in range(m):
        for j in range(n):
            if backtrack(i, j, 0, set()):
                return True

    return False
```

Time Complexity: O(m*n * 3^L) | Space Complexity: O(L)
Idea: Use set to track visited positions instead of modifying board.

Solution 3: Trie + Backtracking (for multiple words)

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_word = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):
        node = self.root
        for char in word:
            if char not in node.children:
                node.children[char] = TrieNode()
            node = node.children[char]
        node.is_word = True

def exist(board, word):
    # Build trie (useful when searching multiple words)
    trie = Trie()
    trie.insert(word)

    def backtrack(i, j, node):
        char = board[i][j]
        if char not in node.children:
            return False

        node = node.children[char]
        if node.is_word:
            return True

        # Mark visited
        board[i][j] = '#'

        for di, dj in [(0,1), (1,0), (0,-1), (-1,0)]:
            ni, nj = i + di, j + dj
            if 0 <= ni < m and 0 <= nj < n and board[ni][nj] != '#':
                if backtrack(ni, nj, node):
                    board[i][j] = char  # restore
                    return True

        # Backtrack
        board[i][j] = char
        return False

    m, n = len(board), len(board[0])

    for i in range(m):
        for j in range(n):
            if backtrack(i, j, trie.root):
                return True
    return False
```

Time Complexity: $O(m*n * 3^L)$ | Space Complexity: O(L) for trie
Idea: Use trie to efficiently check prefix existence during search.

Solution 4: BFS Approach

```
def exist(board, word):
    from collections import deque

    m, n = len(board), len(board[0])

    # Find all starting positions
    starts = []
    for i in range(m):
        for j in range(n):
            if board[i][j] == word[0]:
                starts.append((i, j))

    for start_i, start_j in starts:
        queue = deque([(start_i, start_j, 0, set())])

        while queue:
            i, j, index, visited = queue.popleft()

            if index == len(word) - 1:
                return True

            # Add to visited for this path
            new_visited = visited | {(i, j)}

            for di, dj in [(0,1), (1,0), (0,-1), (-1,0)]:
                ni, nj = i + di, j + dj
                if 0 <= ni < m and 0 <= nj < n and (ni, nj) not in visited:
                    if board[ni][nj] == word[index + 1]:
                        queue.append((ni, nj, index + 1, new_visited))

    return False
```

Time Complexity: $O(m*n * 4^L)$ |
Space Complexity: O(m*n * L) for queue
Idea: BFS exploring all paths level by level.

Solution 5: Optimized with Early Exit

```
def exist(board, word):
    m, n = len(board), len(board[0])

    # Prune 1: word longer than board cells
    if len(word) > m * n:
        return False

    # Prune 2: check character frequencies
    from collections import Counter
    board_counts = Counter(ch for row in board for ch in row)
    word_counts = Counter(word)
    if any(board_counts[ch] < word_counts[ch] for ch in word_counts):
        return False

    # Prune 3: check if first and last chars are rare
    if word_counts[word[0]] > word_counts[word[-1]]:
        word = word[::-1]  # reverse to start with rarer character

    def backtrack(i, j, index):
        if index == len(word):
            return True

        if i < 0 or i >= m or j < 0 or j >= n or board[i][j] != word[index]:
            return False

        temp = board[i][j]
        board[i][j] = '#'

        # Order directions based on likelihood
        directions = [(0,1), (1,0), (0,-1), (-1,0)]

        for di, dj in directions:
            if backtrack(i + di, j + dj, index + 1):
                board[i][j] = temp
                return True

        board[i][j] = temp
        return False

    for i in range(m):
        for j in range(n):
            if board[i][j] == word[0]:
                if backtrack(i, j, 0):
                    return True

    return False
```

Time Complexity: $O(m*n * 3^L)$ | Space Complexity: O(L)
Idea: Multiple optimizations: length check, frequency check, start with rarer character.

131. Palindrome Partitioning

Given a string s, partition s such that every substring of the partition is a **palindrome**. Return *all possible palindrome partitioning of* s.

Example 1:

Input: s = "aab"
Output: [["a","a","b"],["aa","b"]]

Example 2:

Input: s = "a"
Output: [["a"]]

Solution 1: Standard Backtracking

```
def partition(s):
    def backtrack(start, path):
        if start == len(s):
            result.append(path[:])
            return

        for end in range(start + 1, len(s) + 1):
            substring = s[start:end]
            if substring == substring[::-1]:  # Check palindrome
                path.append(substring)
                backtrack(end, path)
                path.pop()

    result = []
    backtrack(0, [])
    return result
```

Time Complexity: $O(n * 2^n)$ | Space Complexity: $O(n^2)$ for palindrome checks
Idea: Try all possible partitions, only continue if current substring is palindrome.

Solution 2: Backtracking with DP Palindrome Table

```
def partition(s):
    n = len(s)
    # dp[i][j] = True if s[i:j+1] is palindrome
    dp = [[False] * n for _ in range(n)]

    # Precompute palindrome table
    for i in range(n):
        dp[i][i] = True

    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1
            if s[i] == s[j]:
                if length == 2 or dp[i+1][j-1]:
                    dp[i][j] = True

    def backtrack(start, path):
        if start == n:
            result.append(path[:])
            return

        for end in range(start, n):
            if dp[start][end]:
                path.append(s[start:end+1])
                backtrack(end + 1, path)
                path.pop()

    result = []
    backtrack(0, [])
    return result
```

Time Complexity: $O(n * 2^n)$ | Space Complexity: $O(n^2)$
Idea: Precompute palindrome table for O(1) palindrome checks during backtracking.

Solution 3: Backtracking with Memoization

```
def partition(s):
    from functools import lru_cache

    @lru_cache(None)
    def is_palindrome(i, j):
        if i >= j:
            return True
        return s[i] == s[j] and is_palindrome(i + 1, j - 1)

    def backtrack(start, path):
        if start == len(s):
            result.append(path[:])
            return

        for end in range(start + 1, len(s) + 1):
            if is_palindrome(start, end - 1):
                path.append(s[start:end])
                backtrack(end, path)
                path.pop()

    result = []
    backtrack(0, [])
    return result
```

Time Complexity: $O(n * 2^n)$ | Space Complexity: $O(n^2)$
Idea: Memoize palindrome checks to avoid repeated computations.

Solution 4: Iterative DP (Count Partitions)

```
def partition(s):
    n = len(s)
    dp = [[] for _ in range(n + 1)]
    dp[0].append([])  # empty partition for empty string

    # Precompute palindrome table
    is_pal = [[False] * n for _ in range(n)]
    for i in range(n):
        is_pal[i][i] = True

    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1
            if s[i] == s[j] and (length == 2 or is_pal[i+1][j-1]):
                is_pal[i][j] = True

    # Build partitions iteratively
    for end in range(1, n + 1):
        for start in range(end):
            if is_pal[start][end-1]:
                for partition_list in dp[start]:
                    dp[end].append(partition_list + [s[start:end]])

    return dp[n]
```

Time Complexity: $O(n * 2^n)$ | Space Complexity: $O(n * 2^n)$
Idea: DP approach building partitions iteratively using precomputed palindrome table.

Solution 5: BFS Approach

```
def partition(s):
    from collections import deque

    result = []
    n = len(s)

    # Precompute palindrome table
    is_pal = [[False] * n for _ in range(n)]
    for i in range(n):
        is_pal[i][i] = True

    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1
            if s[i] == s[j] and (length == 2 or is_pal[i+1][j-1]):
                is_pal[i][j] = True

    queue = deque([(0, [])])  # (start_index,
current_partition)

    while queue:
        start, path = queue.popleft()

        if start == n:
            result.append(path)
            continue

        for end in range(start + 1, n + 1):
            if is_pal[start][end-1]:
                queue.append((end, path + [s[start:end]]))

    return result
```

Time Complexity: $O(n * 2^n)$ | Space Complexity: $O(n * 2^n)$
Idea: BFS exploring all valid partitions level by level.

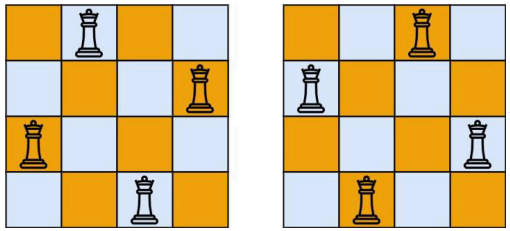
51. N-Queens

The **n-queens** puzzle is the problem of placing n queens on an n x n chessboard such that no two queens attack each other.

Given an integer n, return *all distinct solutions to the **n-queens puzzle***. You may return the answer in **any order**.

Each solution contains a distinct board configuration of the n-queens' placement, where 'Q' and '.' both indicate a queen and an empty space, respectively.

Example 1:



Input: n = 4
Output: [[".Q..","...Q","Q...","..Q."],["..Q.","Q...","...Q",".Q.."]]
Explanation: There exist two distinct solutions to the 4-queens puzzle as shown above

Example 2:

Input: n = 1
Output: [["Q"]]

Solution 1: Standard Backtracking

```
def solveNQueens(n):
    def backtrack(row, cols, diag1, diag2, board):
        if row == n:
            result.append(["".join(row) for row in board])
            return

        for col in range(n):
            d1 = row - col  # diagonal 1 (top-left to
bottom-right)
            d2 = row + col  # diagonal 2 (top-right to
bottom-left)

            if col not in cols and d1 not in diag1 and d2 not in
diag2:
                # Place queen
                board[row][col] = 'Q'
                cols.add(col)
                diag1.add(d1)
                diag2.add(d2)

                backtrack(row + 1, cols, diag1, diag2, board)

                # Remove queen
                board[row][col] = '.'
                cols.remove(col)
                diag1.remove(d1)
                diag2.remove(d2)

    result = []
    board = [['.'] * n for _ in range(n)]
    backtrack(0, set(), set(), set(), board)
    return result
```

Time Complexity: O(n!) | Space Complexity: O(n)
Idea: Place queens row by row, track attacked columns and diagonals.

Solution 2: Backtracking with Bitmask

```
def solveNQueens(n):
    def backtrack(row, cols, diag1, diag2, board):
        if row == n:
            result.append(["".join(row) for row in board])
            return

        # Generate available positions using bitmask
        available = ((1 << n) - 1) & ~(cols | diag1 | diag2)

        while available:
            # Get least significant set bit
            col = (available & -available).bit_length() - 1
            available &= available - 1  # remove the bit

            # Place queen
            board[row][col] = 'Q'

            # Recurse with updated bitmasks
            backtrack(row + 1,
                      cols | (1 << col),
                      (diag1 | (1 << (row - col + n - 1))) if row >=
col else diag1,
                      (diag2 | (1 << (row + col))) if row + col < 2*n -
1 else diag2,
                      board)

            # Remove queen
            board[row][col] = '.'

    result = []
    board = [['.'] * n for _ in range(n)]
    backtrack(0, 0, 0, 0, board)
    return result
```

Time Complexity: O(n!) | Space Complexity: O(n)
Idea: Use bitmask to represent available positions for O(1) operations.

Solution 3: Iterative Backtracking

```
def solveNQueens(n):
    result = []
    stack = [(0, ['.' * n for _ in range(n)], set(), set(), set())]  #
(row, board, cols, diag1, diag2)

    while stack:
        row, board, cols, diag1, diag2 = stack.pop()

        if row == n:
            result.append(board)
            continue

        for col in range(n):
            d1 = row - col
            d2 = row + col

            if col not in cols and d1 not in diag1 and d2 not in
diag2:
                new_board = board[:]
                new_board[row] = new_board[row][:col] + 'Q' +
new_board[row][col+1:]

                new_cols = cols.copy()
                new_diag1 = diag1.copy()
                new_diag2 = diag2.copy()

                new_cols.add(col)
                new_diag1.add(d1)
                new_diag2.add(d2)

                stack.append((row + 1, new_board, new_cols,
new_diag1, new_diag2))

    return result
```

Time Complexity: O(n!) | Space Complexity: O(n!) for stack
Idea: Iterative DFS using stack instead of recursion.

Solution 4: Minimum Remaining Values Heuristic

```
def solveNQueens(n):
    def backtrack(row, cols, diag1, diag2, board, rows_left):
        if not rows_left:
            result.append(["".join(row) for row in board])
            return

        # MRV heuristic: choose row with fewest available
columns
        row = min(rows_left, key=lambda r: sum(
            1 for c in range(n)
            if c not in cols and (r-c) not in diag1 and (r+c) not
in diag2
        ))
        rows_left.remove(row)

        for col in range(n):
            d1 = row - col
            d2 = row + col

            if col not in cols and d1 not in diag1 and d2 not in
diag2:
                board[row][col] = 'Q'
                cols.add(col)
                diag1.add(d1)
                diag2.add(d2)

                backtrack(row + 1, cols, diag1, diag2, board,
rows_left.copy())

                board[row][col] = '.'
                cols.remove(col)
                diag1.remove(d1)
                diag2.remove(d2)

        rows_left.add(row)

    result = []
    board = [['.'] * n for _ in range(n)]
    backtrack(0, set(), set(), set(), board, set(range(n)))
    return result
```

Time Complexity: O(n!) | Space Complexity: O(n)
Idea: Use heuristic to choose most constrained row first (MRV).

Solution 5: Symmetry Breaking

```
def solveNQueens(n):
    def backtrack(row, cols, diag1, diag2, board, solutions_set):
        if row == n:
            board_str = "".join("".join(row) for row in board)
            if board_str not in solutions_set:
                solutions_set.add(board_str)
                result.append(["".join(row) for row in board])
            return

        for col in range(n):
            d1 = row - col
            d2 = row + col

            if col not in cols and d1 not in diag1 and d2 not in diag2:
                board[row][col] = 'Q'
                cols.add(col)
                diag1.add(d1)
                diag2.add(d2)

                backtrack(row + 1, cols, diag1, diag2, board, solutions_set)

                board[row][col] = '.'
                cols.remove(col)
                diag1.remove(d1)
                diag2.remove(d2)

    result = []
    solutions_set = set()
    board = [['.'] * n for _ in range(n)]

    # Only try first half of columns for first row (symmetry)
    for col in range((n + 1) // 2):
        board[0][col] = 'Q'
        backtrack(1, {col}, {0-col}, {0+col}, board, solutions_set)
        board[0][col] = '.'

    return result
```

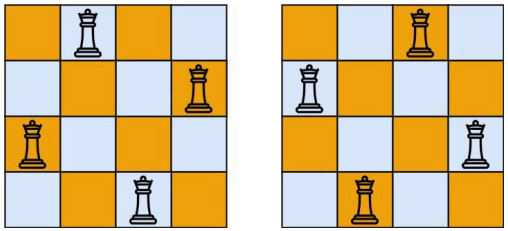
Time Complexity: O(n!/2) | Space Complexity: O(n!)
Idea: Exploit symmetry to reduce search space by half.

52. N-Queens II

The **n-queens** puzzle is the problem of placing n queens on an n x n chessboard such that no two queens attack each other.

Given an integer n, return *the number of distinct solutions to the **n-queens puzzle***.

Example 1:



Input: n = 4
Output: 2
Explanation: There are two distinct solutions to the 4-queens puzzle as shown.

Example 2:

Input: n = 1
Output: 1

Solution 1: Backtracking with Count

```
def totalNQueens(n):
    def backtrack(row, cols, diag1, diag2):
        if row == n:
            return 1

        count = 0
        for col in range(n):
            d1 = row - col
            d2 = row + col

            if col not in cols and d1 not in diag1 and d2 not in diag2:
                cols.add(col)
                diag1.add(d1)
                diag2.add(d2)

                count += backtrack(row + 1, cols, diag1, diag2)

                cols.remove(col)
                diag1.remove(d1)
                diag2.remove(d2)

        return count

    return backtrack(0, set(), set(), set())
```

Time Complexity: O(n!) | Space Complexity: O(n)

Solution 2: Bitmask Counting

```
def totalNQueens(n):
    def backtrack(row, cols, diag1, diag2):
        if row == n:
            return 1

        count = 0
        available = ((1 << n) - 1) & ~(cols | diag1 | diag2)

        while available:
            # Get least significant set bit
            col = (available & -available).bit_length() - 1
            available &= available - 1

            count += backtrack(row + 1,
                               cols | (1 << col),
                               (diag1 | (1 << (row - col + n - 1))) >> 1,
                               (diag2 | (1 << (row + col))) >> 1)

        return count

    return backtrack(0, 0, 0, 0)
```

Time Complexity: O(n!) | Space Complexity: O(n)
Idea: Bitmask version for faster operations.

Solution 3: Iterative Counting

```
def totalNQueens(n):
    count = 0
    stack = [(0, 0, 0, 0)]  # (row, cols, diag1, diag2)

    while stack:
        row, cols, diag1, diag2 = stack.pop()

        if row == n:
            count += 1
            continue

        available = ((1 << n) - 1) & ~(cols | diag1 | diag2)

        while available:
            col = (available & -available).bit_length() - 1
            available &= available - 1

            stack.append((row + 1,
                          cols | (1 << col),
                          (diag1 | (1 << (row - col + n - 1))) >> 1,
                          (diag2 | (1 << (row + col))) >> 1))

    return count
```

Time Complexity: O(n!) | Space Complexity: O(n!) for stack
Idea: Iterative version using stack.

Solution 4: Closed-form Formula (Small n)

```
def totalNQueens(n):
    # Known solutions for n ≤ 10
    solutions = [0, 1, 0, 0, 2, 10, 4, 40, 92, 352, 724]
    return solutions[n] if n < len(solutions) else -1
```

Time Complexity: O(1) | Space Complexity: O(1)
Idea: Precomputed known values (not general solution).

Solution 5: Backtracking with Pruning Enhancement

```
def totalNQueens(n):
    def backtrack(row, cols, diag1, diag2, left_cols):
        if row == n:
            return 1

        count = 0
        # Only try columns that haven't been tried at this row
        for col in left_cols:
            d1 = row - col
            d2 = row + col

            if d1 not in diag1 and d2 not in diag2:
                # Create new list without current column
                new_left_cols = [c for c in left_cols if c != col]

                cols.add(col)
                diag1.add(d1)
                diag2.add(d2)

                count += backtrack(row + 1, cols, diag1, diag2, new_left_cols)

                cols.remove(col)
                diag1.remove(d1)
                diag2.remove(d2)

        return count

    return backtrack(0, set(), set(), set(), list(range(n)))
```

Time Complexity: O(n!) | Space Complexity: O(n)
Idea: Track remaining columns to avoid checking already used columns.

17. Letter Combinations of a Phone Number

Given a string containing digits from 2-9 inclusive, return all possible letter combinations that the number could represent. Return the answer in **any order**.

A mapping of digits to letters (just like on the telephone buttons) is given below. Note that 1 does not map to any letters.

Example 1:

Input: digits = "23"
Output: ["ad","ae","af","bd","be","bf","cd","ce","cf"]

Example 2:

Input: digits = "2"
Output: ["a","b","c"]

Solution 1: Backtracking/DFS

```
def letterCombinations(digits):
    if not digits:
        return []

    phone_map = {
        '2': 'abc', '3': 'def', '4': 'ghi', '5': 'jkl',
        '6': 'mno', '7': 'pqrs', '8': 'tuv', '9': 'wxyz'
    }

    def backtrack(index, path):
        if index == len(digits):
            result.append("".join(path))
            return

        digit = digits[index]
        for letter in phone_map[digit]:
            path.append(letter)
            backtrack(index + 1, path)
            path.pop()

    result = []
    backtrack(0, [])
    return result
```

Time Complexity: $O(4^n * n)$ where n = len(digits) | Space Complexity: O(n)
Idea: Build combinations digit by digit, exploring all letters for each digit.

Solution 2: Iterative BFS

```
def letterCombinations(digits):
    if not digits:
        return []

    phone_map = {
        '2': 'abc', '3': 'def', '4': 'ghi', '5': 'jkl',
        '6': 'mno', '7': 'pqrs', '8': 'tuv', '9': 'wxyz'
    }

    result = [""]

    for digit in digits:
        new_result = []
        for combination in result:
            for letter in phone_map[digit]:
                new_result.append(combination + letter)
        result = new_result

    return result
```

Time Complexity: $O(4^n * n)$ | Space Complexity: $O(4^n)$
Idea: BFS building combinations level by level.

Solution 3: Iterative using Queue

```
def letterCombinations(digits):
    from collections import deque

    if not digits:
        return []

    phone_map = {
        '2': 'abc', '3': 'def', '4': 'ghi', '5': 'jkl',
        '6': 'mno', '7': 'pqrs', '8': 'tuv', '9': 'wxyz'
    }

    queue = deque([""])

    for digit in digits:
        level_size = len(queue)
        for _ in range(level_size):
            current = queue.popleft()
            for letter in phone_map[digit]:
                queue.append(current + letter)

    return list(queue)
```

Time Complexity: $O(4^n * n)$ | Space Complexity: $O(4^n)$
Idea: BFS using queue for level-order traversal.

Solution 4: Product from itertools

```
def letterCombinations(digits):
    if not digits:
        return []

    phone_map = {
        '2': 'abc', '3': 'def', '4': 'ghi', '5': 'jkl',
        '6': 'mno', '7': 'pqrs', '8': 'tuv', '9': 'wxyz'
    }

    from itertools import product

    # Get list of letter lists for each digit
    letter_lists = [phone_map[digit] for digit in digits]

    # Generate all combinations using Cartesian product
    return ["".join(comb) for comb in product(*letter_lists)]
```

Time Complexity: $O(4^n * n)$ | Space Complexity: $O(4^n)$
Idea: Use Python's built-in Cartesian product.

Solution 5: Recursive with Memoization

```
def letterCombinations(digits):
    if not digits:
        return []

    phone_map = {
        '2': 'abc', '3': 'def', '4': 'ghi', '5': 'jkl',
        '6': 'mno', '7': 'pqrs', '8': 'tuv', '9': 'wxyz'
    }

    from functools import lru_cache

    @lru_cache(None)
    def dfs(index):
        if index == len(digits):
            return [""]

        digit = digits[index]
        result = []
        for letter in phone_map[digit]:
            for suffix in dfs(index + 1):
                result.append(letter + suffix)
        return result

    return dfs(0)
```

Time Complexity: $O(4^n * n)$ | Space Complexity: $O(4^n)$
Idea: Memoized recursion to avoid recomputation (though not much benefit here).

Solution 6: Backtracking with StringBuilder Optimization

```
def letterCombinations(digits):
    if not digits:
        return []

    phone_map = {
        '2': 'abc', '3': 'def', '4': 'ghi', '5': 'jkl',
        '6': 'mno', '7': 'pqrs', '8': 'tuv', '9': 'wxyz'
    }

    result = []
    n = len(digits)

    def backtrack(index, path):
        if index == n:
            result.append(path)
            return

        digit = digits[index]
        for letter in phone_map[digit]:
            backtrack(index + 1, path + letter)

    backtrack(0, "")
    return result
```

Time Complexity: $O(4^n * n)$ | Space Complexity: O(n) for recursion depth
Idea: Pass string directly instead of list, avoiding list operations.

1D DYNAMIC PROGRAMMING

70. Climbing Stairs

You are climbing a staircase. It takes n steps to reach the top.

Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Example 1:

Input: n = 2
Output: 2
Explanation: There are two ways to climb to the top.
1. 1 step + 1 step
2. 2 steps

Example 2:

Input: n = 3
Output: 3
Explanation: There are three ways to climb to the top.
1. 1 step + 1 step + 1 step
2. 1 step + 2 steps
3. 2 steps + 1 step

Solution 1: Recursive (Top-Down)

```
def climbStairs(self, n: int) -> int:
    def dfs(i):
        if i <= 2:
            return i
        return dfs(i-1) + dfs(i-2)

    return dfs(n)
```

Time Complexity: $O(2^n)$ | Space Complexity: O(n)
Idea: Fibonacci recurrence - ways to reach step i = ways to i-1 + ways to i-2.

Solution 2: Memoization (Top-Down DP)

```
def climbStairs(self, n: int) -> int:
    memo = {}

    def dfs(i):
        if i <= 2:
            return i
        if i in memo:
            return memo[i]
        memo[i] = dfs(i-1) + dfs(i-2)
        return memo[i]

    return dfs(n)
```

Time Complexity: O(n) | Space Complexity: O(n)

Solution 3: Dynamic Programming (Bottom-Up)

```
def climbStairs(self, n: int) -> int:
    if n <= 2:
        return n

    dp = [0] * (n + 1)
    dp[1] = 1
    dp[2] = 2

    for i in range(3, n + 1):
        dp[i] = dp[i-1] + dp[i-2]

    return dp[n]
```

Time Complexity: O(n) | Space Complexity: O(n)
Idea: Build solution iteratively from base cases.

Solution 4: Space Optimized DP

```
def climbStairs(self, n: int) -> int:
    if n <= 2:
        return n

    prev2, prev1 = 1, 2  # dp[i-2], dp[i-1]

    for i in range(3, n + 1):
        curr = prev1 + prev2
        prev2 = prev1
        prev1 = curr

    return prev1
```

Time Complexity: O(n) | Space Complexity: O(1)
Idea: Only need last two values.

Solution 5: Matrix Exponentiation

```
def climbStairs(self, n: int) -> int:
    def matrix_multiply(a, b):
        return [
            [a[0][0]*b[0][0] + a[0][1]*b[1][0], a[0][0]*b[0][1] + a[0][1]*b[1][1]],
            [a[1][0]*b[0][0] + a[1][1]*b[1][0], a[1][0]*b[0][1] + a[1][1]*b[1][1]]
        ]

    def matrix_power(matrix, n):
        if n == 1:
            return matrix
        if n % 2 == 0:
            half = matrix_power(matrix, n // 2)
            return matrix_multiply(half, half)
        else:
            return matrix_multiply(matrix, matrix_power(matrix, n - 1))

    if n <= 2:
        return n

    base = [[1, 1], [1, 0]]
    result = matrix_power(base, n - 1)
    return result[0][0] + result[0][1]
```

Time Complexity: O(log n) | Space Complexity: O(log n)
Idea: Use matrix exponentiation for Fibonacci numbers.

Solution 6: Binet's Formula

```
def climbStairs(self, n: int) -> int:
    sqrt5 = 5  0.5
    phi = (1 + sqrt5) / 2
    psi = (1 - sqrt5) / 2
    return int((phi(n+1) - psi(n+1)) / sqrt5)
```

Time Complexity: O(log n) | Space Complexity: O(1)
Idea: Direct mathematical formula for Fibonacci.

746. Min Cost Climbing Stairs

You are given an integer array cost where cost[i] is the cost of i_{th} step on a staircase. Once you pay the cost, you can either climb one or two steps.

You can either start from the step with index 0, or the step with index 1.Return *the minimum cost to reach the top of the floor*.

Example 1:

Input: cost = [10,15,20]
Output: 15
Explanation: You will start at index 1.
- Pay 15 and climb two steps to reach the top.
The total cost is 15.

Example 2:

Input: cost = [1,100,1,1,1,100,1,1,100,1]
Output: 6
Explanation: You will start at index 0.
- Pay 1 and climb two steps to reach index 2.
- Pay 1 and climb two steps to reach index 4.
- Pay 1 and climb two steps to reach index 6.
- Pay 1 and climb one step to reach index 7.
- Pay 1 and climb two steps to reach index 9.
- Pay 1 and climb one step to reach the top.
The total cost is 6.

Solution 1: Recursive

```
def minCostClimbingStairs(self, cost: List[int]) -> int:
    def dfs(i):
        if i <= 1:
            return 0
        return min(dfs(i-1) + cost[i-1], dfs(i-2) + cost[i-2])

    return dfs(len(cost))
```

Time Complexity: $O(2^n)$ | Space Complexity: O(n)
Idea: At each step, take minimum of coming from previous or second previous.

Solution 2: Memoization

```
def minCostClimbingStairs(self, cost: List[int]) -> int:
    memo = {}

    def dfs(i):
        if i <= 1:
            return 0
        if i in memo:
            return memo[i]
        memo[i] = min(dfs(i-1) + cost[i-1], dfs(i-2) + cost[i-2])
        return memo[i]

    return dfs(len(cost))
```

Time Complexity: O(n) | Space Complexity: O(n)
Idea: Cache results of subproblems.

Solution 3: Dynamic Programming

```
def minCostClimbingStairs(self, cost: List[int]) -> int:
    n = len(cost)
    dp = [0] * (n + 1)

    for i in range(2, n + 1):
        dp[i] = min(dp[i-1] + cost[i-1], dp[i-2] + cost[i-2])

    return dp[n]
```

Time Complexity: O(n) | Space Complexity: O(n)
Idea: dp[i] = min cost to reach step i.

Solution 4: Space Optimized DP

```
def minCostClimbingStairs(self, cost: List[int]) -> int:
    prev2 = prev1 = 0

    for i in range(2, len(cost) + 1):
        curr = min(prev1 + cost[i-1], prev2 + cost[i-2])
        prev2 = prev1
        prev1 = curr

    return prev1
```

Time Complexity: O(n) | Space Complexity: O(1)
Idea: Only need last two values.

Solution 5: In-Place DP

```
def minCostClimbingStairs(self, cost: List[int]) -> int:
    for i in range(2, len(cost)):
        cost[i] += min(cost[i-1], cost[i-2])
    return min(cost[-1], cost[-2])
```

Time Complexity: O(n) | Space Complexity: O(1)
Idea: Modify original array to store minimum costs.

198. House Robber

You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed, the only constraint stopping you from robbing each of them is that adjacent houses have security systems connected and **it will automatically contact the police if two adjacent houses were broken into on the same night**.

Given an integer array nums representing the amount of money of each house, return *the maximum amount of money you can rob tonight* ***without alerting the police***.

Example 1:

Input: nums = [1,2,3,1]
Output: 4
Explanation: Rob house 1 (money = 1) and then rob house 3 (money = 3).
Total amount you can rob = 1 + 3 = 4.

Example 2:

Input: nums = [2,7,9,3,1]
Output: 12
Explanation: Rob house 1 (money = 2), rob house 3 (money = 9) and rob house 5 (money = 1).
Total amount you can rob = 2 + 9 + 1 = 12.

Solution 1: Recursive

```
def rob(self, nums: List[int]) -> int:
    def dfs(i):
        if i < 0:
            return 0
        return max(dfs(i-1), dfs(i-2) + nums[i])

    return dfs(len(nums) - 1)
```

Time Complexity: $O(2^n)$ | Space Complexity: O(n)
Idea: At each house, either rob it (skip previous) or skip it.

Solution 2: Memoization

```
def rob(self, nums: List[int]) -> int:
    memo = {}

    def dfs(i):
        if i < 0:
            return 0
        if i in memo:
            return memo[i]
        memo[i] = max(dfs(i-1), dfs(i-2) + nums[i])
        return memo[i]

    return dfs(len(nums) - 1)
```

Time Complexity: O(n) | Space Complexity: O(n)
Idea: Cache results to avoid redundant calculations.

Solution 3: Dynamic Programming

```
def rob(self, nums: List[int]) -> int:
    if not nums:
        return 0
    if len(nums) == 1:
        return nums[0]

    dp = [0] * len(nums)
    dp[0] = nums[0]
    dp[1] = max(nums[0], nums[1])

    for i in range(2, len(nums)):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i])

    return dp[-1]
```

Time Complexity: O(n) | Space Complexity: O(n)
Idea: dp[i] = max money up to house i.

Solution 4: Space Optimized DP

```
def rob(self, nums: List[int]) -> int:
    prev2 = prev1 = 0

    for num in nums:
        curr = max(prev1, prev2 + num)
        prev2 = prev1
        prev1 = curr

    return prev1
```

Time Complexity: O(n) | Space Complexity: O(1)
Idea: Only need last two values.

Solution 5: Two Pointers

```
def rob(self, nums: List[int]) -> int:
    even = odd = 0

    for i in range(len(nums)):
        if i % 2 == 0:
            even = max(even + nums[i], odd)
        else:
            odd = max(odd + nums[i], even)

    return max(even, odd)
```

Time Complexity: O(n) | Space Complexity: O(1)
Idea: Alternate between even and odd indexed houses.

213. House Robber II

You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed. All houses at this place are **arranged in a circle.** That means the first house is the neighbor of the last one. Meanwhile, adjacent houses have a security system connected, and **it will automatically contact the police if two adjacent houses were broken into on the same night**.

Given an integer array nums representing the amount of money of each house, return *the maximum amount of money you can rob tonight* ***without alerting the police***.

Example 1:

Input: nums = [2,3,2]
Output: 3
Explanation: You cannot rob house 1 (money = 2) and then rob house 3 (money = 2), because they are adjacent houses.

Example 2:

Input: nums = [1,2,3,1]
Output: 4
Explanation: Rob house 1 (money = 1) and then rob house 3 (money = 3).
Total amount you can rob = 1 + 3 = 4.

Example 3:

Input: nums = [1,2,3]
Output: 3

Solution 1: Split and Combine

```
def rob(self, nums: List[int]) -> int:
    if len(nums) == 1:
        return nums[0]

    def rob_linear(houses):
        prev2 = prev1 = 0
        for money in houses:
            curr = max(prev1, prev2 + money)
            prev2 = prev1
            prev1 = curr
        return prev1

    # Either skip first house or skip last house
    return max(rob_linear(nums[1:]), rob_linear(nums[:-1]))
```

Time Complexity: O(n) | Space Complexity: O(1)
Idea: Solve two linear problems - excluding first or last.

Solution 2: Dynamic Programming with Two Cases

```
def rob(self, nums: List[int]) -> int:
    if len(nums) == 1:
        return nums[0]

    def rob_range(start, end):
        prev2 = prev1 = 0
        for i in range(start, end):
            curr = max(prev1, prev2 + nums[i])
            prev2 = prev1
            prev1 = curr
        return prev1

    return max(rob_range(0, len(nums)-1), rob_range(1, len(nums)))
```

Time Complexity: O(n) | Space Complexity: O(1)
Idea: Use range-based rob function.

Solution 3: Recursive with Memoization

```
def rob(self, nums: List[int]) -> int:
    if len(nums) == 1:
        return nums[0]

    memo1, memo2 = {}, {}

    def dfs1(i):
        if i < 1:
            return 0
        if i in memo1:
            return memo1[i]
        memo1[i] = max(dfs1(i-1), dfs1(i-2) + nums[i])
        return memo1[i]

    def dfs2(i):
        if i < 0:
            return 0
        if i == len(nums) - 1:
            return 0
        if i in memo2:
            return memo2[i]
        memo2[i] = max(dfs2(i-1), dfs2(i-2) + nums[i])
        return memo2[i]

    return max(dfs1(len(nums)-1), dfs2(len(nums)-2))
```

Time Complexity: O(n) | Space Complexity: O(n)
Idea: Two separate memoization functions.

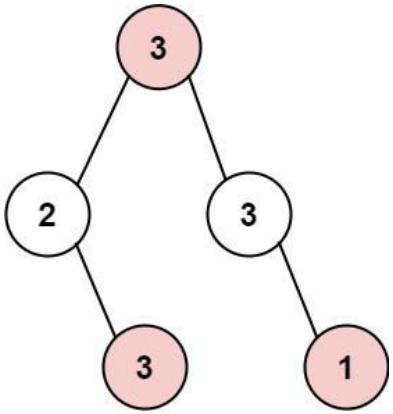
337. House Robber III

The thief has found himself a new place for his thievery again. There is only one entrance to this area, called root.

Besides the root, each house has one and only one parent house. After a tour, the smart thief realized that all houses in this place form a binary tree. It will automatically contact the police if **two directly-linked houses were broken into on the same night**.

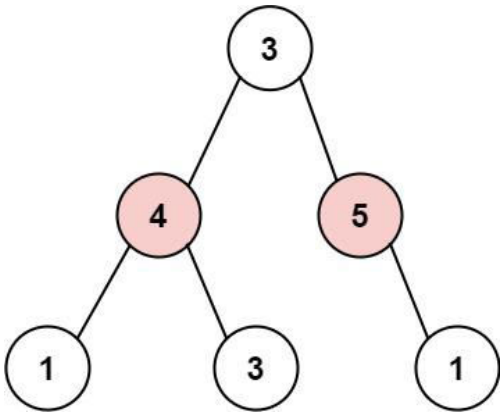
Given the root of the binary tree, return *the maximum amount of money the thief can rob* ***without alerting the police***.

Example 1:



Input: root = [3,2,3,null,3,null,1]
Output: 7
Explanation: Maximum amount of money the thief can rob = 3 + 3 + 1 = 7.

Example 2:



Input: root = [3,4,5,1,3,null,1]
Output: 9
Explanation: Maximum amount of money the thief can rob = 4 + 5 = 9.

Approach 1: Recursive with Memoization

```
def rob(root):
    memo = {}

    def dfs(node):
        if not node:
            return 0
        if node in memo:
            return memo[node]

        # Rob this house
        rob_current = node.val
        if node.left:
            rob_current += dfs(node.left.left) + dfs(node.left.right)
        if node.right:
            rob_current += dfs(node.right.left) + dfs(node.right.right)

        # Skip this house
        skip_current = dfs(node.left) + dfs(node.right)

        memo[node] = max(rob_current, skip_current)
        return memo[node]

    return dfs(root)
```

Time Complexity: O(n)
Space Complexity: O(n)

Approach 2: DP with Tuple Returns

```
def rob(root):
    def dfs(node):
        if not node:
            return (0, 0)  # (rob_current, skip_current)

        left = dfs(node.left)
        right = dfs(node.right)

        # If we rob this node, we cannot rob children
        rob_current = node.val + left[1] + right[1]

        # If we skip this node, we can rob or skip children
        skip_current = max(left) + max(right)

        return (rob_current, skip_current)

    return max(dfs(root))
```

Time Complexity: O(n)
Space Complexity: O(h) where h is height of tree

Approach 3: Greedy with Level Order

```
def rob(root):
    if not root:
        return 0

    # Level order to get sums at each level
    queue = deque([root])
    level_sums = []

    while queue:
        level_size = len(queue)
        level_sum = 0
        for _ in range(level_size):
            node = queue.popleft()
            level_sum += node.val
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        level_sums.append(level_sum)

    # Now it becomes House Robber I on level sums
    prev1 = prev2 = 0
    for num in level_sums:
        temp = prev1
        prev1 = max(prev2 + num, prev1)
        prev2 = temp

    return prev1
```

Time Complexity: O(n)
Space Complexity: O(n)

2560. House Robber IV

There are several consecutive houses along a street, each of which has some money inside. There is also a robber, who wants to steal money from the homes, but he **refuses to steal from adjacent homes**.

The **capability** of the robber is the maximum amount of money he steals from one house of all the houses he robbed.

You are given an integer array nums representing how much money is stashed in each house. More formally, the i_{th} house from the left has nums[i] dollars.

You are also given an integer k, representing the **minimum** number of houses the robber will steal from. It is always possible to steal at least k houses.

Return *the **minimum** capability of the robber out of all the possible ways to steal at least* k *houses*.

Example 1:

Input: nums = [2,3,5,9], k = 2
Output: 5
Explanation:
There are three ways to rob at least 2 houses:
- Rob the houses at indices 0 and 2. Capability is max(nums[0], nums[2]) = 5.
- Rob the houses at indices 0 and 3. Capability is max(nums[0], nums[3]) = 9.
- Rob the houses at indices 1 and 3. Capability is max(nums[1], nums[3]) = 9.
Therefore, we return min(5, 9, 9) = 5.

Example 2:

Input: nums = [2,7,9,3,1], k = 2
Output: 2
Explanation: There are 7 ways to rob the houses. The way which leads to minimum capability is to rob the house at index 0 and 4. Return max(nums[0], nums[4]) = 2.

Approach 1: Binary Search + Greedy

```
def minCapability(nums, k):
    def can_rob(capability):
        count = 0
        i = 0
        while i < len(nums):
            if nums[i] <= capability:
                count += 1
                i += 2  # Skip next house
            else:
                i += 1
        return count >= k

    left, right = min(nums), max(nums)
    result = right

    while left <= right:
        mid = (left + right) // 2
        if can_rob(mid):
            result = mid
            right = mid - 1
        else:
            left = mid + 1

    return result
```

Time Complexity: O(n log m) where m is max(nums)
Space Complexity: O(1)

Approach 2: DP with Binary Search

```
def minCapability(nums, k):
    def can_rob(capability):
        n = len(nums)
        dp = [0] * (n + 1)

        for i in range(1, n + 1):
            if nums[i-1] <= capability:
                dp[i] = max(dp[i-1], (dp[i-2] if i >= 2 else 0) + 1)
            else:
                dp[i] = dp[i-1]

        return dp[n] >= k

    values = sorted(set(nums))
    left, right = 0, len(values) - 1
    result = values[-1]

    while left <= right:
        mid = (left + right) // 2
        if can_rob(values[mid]):
            result = values[mid]
            right = mid - 1
        else:
            left = mid + 1

    return result
```

Time Complexity: O(n log n)
Space Complexity: O(n)

Approach 3: Greedy with Validation

```
def minCapability(nums, k):
    houses = [(val, i) for i, val in enumerate(nums)]
    houses.sort()

    left, right = 0, len(houses) - 1

    def can_rob_at_least_k(max_val):
        robbed = set()
        count = 0

        for val, idx in houses:
            if val > max_val:
                continue
            if idx - 1 in robbed or idx + 1 in robbed:
                continue
            robbed.add(idx)
            count += 1
            if count >= k:
                return True
        return False

    while left < right:
        mid = (left + right) // 2
        if can_rob_at_least_k(houses[mid][0]):
            right = mid
        else:
            left = mid + 1

    return houses[left][0]
```

Time Complexity: $O(n^2)$ worst case
Space Complexity: O(n)

3840. House Robber V

You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed and is protected by a security system with a color code.

You are given two integer arrays nums and colors, both of length n, where nums[i] is the amount of money in the i_{th} house and colors[i] is the color code of that house.

You **cannot rob two adjacent** houses if they share the **same color** code.

Return the **maximum** amount of money you can rob.

Example 1:

Input: nums = [1,4,3,5], colors = [1,1,2,2]

Output: 9

Explanation:

- Choose houses i = 1 with nums[1] = 4 and i = 3 with nums[3] = 5 because they are non-adjacent.
- Thus, the total amount robbed is 4 + 5 = 9.

Example 2:

Input: nums = [3,1,2,4], colors = [2,3,2,2]

Output: 8

Explanation:

- Choose houses i = 0 with nums[0] = 3, i = 1 with nums[1] = 1, and i = 3 with nums[3] = 4.
- This selection is valid because houses i = 0 and i = 1 have different colors, and house i = 3 is non-adjacent to i = 1.
- Thus, the total amount robbed is 3 + 1 + 4 = 8.

Example 3:

Input: nums = [10,1,3,9], colors = [1,1,1,2]

Output: 22

Explanation:

- Choose houses i = 0 with nums[0] = 10, i = 2 with nums[2] = 3, and i = 3 with nums[3] = 9.
- This selection is valid because houses i = 0 and i = 2 are non-adjacent, and houses i = 2 and i = 3 have different colors.
- Thus, the total amount robbed is 10 + 3 + 9 = 22.

Approach 1: Dynamic Programming with Two State Variables

Approach: This solution tracks two states for each house - whether we rob it or not. At each house, we make decisions based on the color comparison with the previous house .

```
def rob(nums, colors):
    n = len(nums)
    if n == 0:
        return 0
    if n == 1:
        return nums[0]

    # Initialize states for the first house
    prev_not_rob = 0          # Max amount when not robbing previous house
    prev_rob = nums[0]        # Max amount when robbing previous house

    for i in range(1, n):
        # Current state when not robbing this house
        curr_not_rob = max(prev_not_rob, prev_rob)

        # Current state when robbing this house
        if colors[i] != colors[i-1]:
            # Different colors: previous house can be robbed or not
            curr_rob = max(prev_not_rob, prev_rob) + nums[i]
        else:
            # Same color: previous house must NOT be robbed
            curr_rob = prev_not_rob + nums[i]

        # Update states for next iteration
        prev_not_rob, prev_rob = curr_not_rob, curr_rob

    # Return maximum of robbing or not robbing the last house
    return max(prev_not_rob, prev_rob)
```

Time Complexity: O(n) - single pass through the array
Space Complexity: O(1) - only constant extra space

Approach 2: DP with Array (Clearer State Tracking)

Approach: Use two DP arrays to explicitly track the maximum amount when robbing or not robbing each house, making the state transitions more visible .

```
def rob(nums, colors):
    n = len(nums)
    if n == 0:
        return 0

    # dp[i][0] = max amount up to house i without robbing house i
    # dp[i][1] = max amount up to house i with robbing house i
    dp = [[0, 0] for _ in range(n)]

    # Initialize first house
    dp[0][0] = 0
    dp[0][1] = nums[0]

    for i in range(1, n):
        # Not robbing current house - take max of previous states
        dp[i][0] = max(dp[i-1][0], dp[i-1][1])

        # Robbing current house
        if colors[i] != colors[i-1]:
            # Different colors: can take max from previous
            dp[i][1] = max(dp[i-1][0], dp[i-1][1]) + nums[i]
        else:
            # Same color: must skip previous house
            dp[i][1] = dp[i-1][0] + nums[i]

    return max(dp[n-1][0], dp[n-1][1])
```

Time Complexity: O(n)
Space Complexity: O(n) for the DP array

Approach 3: Recursive with Memoization

Approach: Use recursion with memoization to solve subproblems. At each house, we make decisions based on whether we robbed the previous house and color comparison.

```
def rob(nums, colors):
    n = len(nums)
    memo = {}

    def dfs(i, prev_robbed, prev_color):
        if i >= n:
            return 0

        key = (i, prev_robbed, prev_color)
        if key in memo:
            return memo[key]

        # Option 1: Skip current house
        skip = dfs(i + 1, False, prev_color)

        # Option 2: Rob current house (if allowed)
        rob_current = 0
        if not (prev_robbed and colors[i] == prev_color):
            rob_current = nums[i] + dfs(i + 1, True, colors[i])

        memo[key] = max(skip, rob_current)
        return memo[key]

    return dfs(0, False, -1)
```

Time Complexity: O(n * 2 * k) where k is number of distinct colors
Space Complexity: O(n * 2 * k) for memoization

53. Maximum Subarray (Kadane's Algorithm)

Given an integer array nums, find the subarray with the largest sum, and return *its sum*.

Example 1:

Input: nums = [-2,1,-3,4,-1,2,1,-5,4]
Output: 6
Explanation: The subarray [4,-1,2,1] has the largest sum 6.

Example 2:

Input: nums = [1]
Output: 1
Explanation: The subarray [1] has the largest sum 1.

Example 3:

Input: nums = [5,4,-1,7,8]
Output: 23
Explanation: The subarray [5,4,-1,7,8] has the largest sum 23.

Solution 1: Brute Force

```
def maxSubArray(self, nums: List[int]) -> int:
    max_sum = float('-inf')

    for i in range(len(nums)):
        curr_sum = 0
        for j in range(i, len(nums)):
            curr_sum += nums[j]
            max_sum = max(max_sum, curr_sum)

    return max_sum
```

Time Complexity: $O(n^2)$ | Space Complexity: O(1)
Idea: Check all subarrays.

Solution 2: Divide and Conquer

```
def maxSubArray(self, nums: List[int]) -> int:
    def helper(left, right):
        if left == right:
            return nums[left]

        mid = (left + right) // 2

        # Max sum in left half
        left_sum = helper(left, mid)
        # Max sum in right half
        right_sum = helper(mid + 1, right)

        # Max sum crossing mid
        left_cross = float('-inf')
        curr = 0
        for i in range(mid, left - 1, -1):
            curr += nums[i]
            left_cross = max(left_cross, curr)

        right_cross = float('-inf')
        curr = 0
        for i in range(mid + 1, right + 1):
            curr += nums[i]
            right_cross = max(right_cross, curr)

        cross_sum = left_cross + right_cross

        return max(left_sum, right_sum, cross_sum)

    return helper(0, len(nums) - 1)
```

Time Complexity: O(n log n) | Space Complexity: O(log n)
Idea: Split array and combine results.

Solution 3: Kadane's Algorithm

```
def maxSubArray(self, nums: List[int]) -> int:
    curr_sum = max_sum = nums[0]

    for num in nums[1:]:
        curr_sum = max(num, curr_sum + num)
        max_sum = max(max_sum, curr_sum)

    return max_sum
```

Time Complexity: O(n) | Space Complexity: O(1)
Idea: Keep running sum, reset if negative.

Solution 4: Dynamic Programming

```
def maxSubArray(self, nums: List[int]) -> int:
    dp = [0] * len(nums)
    dp[0] = nums[0]
    max_sum = nums[0]

    for i in range(1, len(nums)):
        dp[i] = max(nums[i], dp[i-1] + nums[i])
        max_sum = max(max_sum, dp[i])

    return max_sum
```

Time Complexity: O(n) | Space Complexity: O(n)
Idea: dp[i] = max sum ending at i.

---

152. Maximum Product Subarray

Given an integer array nums, find a subarray that has the largest product, and return *the product*.

The test cases are generated so that the answer will fit in a **32-bit** integer.

Note that the product of an array with a single element is the value of that element.

Example 1:

Input: nums = [2,3,-2,4]
Output: 6
Explanation: [2,3] has the largest product 6.

Example 2:

Input: nums = [-2,0,-1]
Output: 0
Explanation: The result cannot be 2, because [-2,-1] is not a subarray.

Solution 1: Brute Force

```
def maxProduct(self, nums: List[int]) -> int:
    max_prod = float('-inf')

    for i in range(len(nums)):
        curr = 1
        for j in range(i, len(nums)):
            curr *= nums[j]
            max_prod = max(max_prod, curr)

    return max_prod
```

Time Complexity: $O(n^2)$ | Space Complexity: O(1)
Idea: Check all subarrays.

Solution 2: Two-Pass

```
def maxProduct(self, nums: List[int]) -> int:
    max_prod = float('-inf')

    # Forward pass
    curr = 1
    for num in nums:
        curr *= num
        max_prod = max(max_prod, curr)
        if curr == 0:
            curr = 1

    # Backward pass
    curr = 1
    for num in reversed(nums):
        curr *= num
        max_prod = max(max_prod, curr)
        if curr == 0:
            curr = 1

    return max_prod
```

Time Complexity: O(n) | Space Complexity: O(1)
Idea: Maximum product will be either prefix or suffix product.

Solution 3: Kadane's Variation

```
def maxProduct(self, nums: List[int]) -> int:
    max_prod = min_prod = result = nums[0]

    for num in nums[1:]:
        # Store max_prod before updating
        temp_max = max(num, max_prod * num, min_prod * num)
        min_prod = min(num, max_prod * num, min_prod * num)
        max_prod = temp_max

        result = max(result, max_prod)

    return result
```

Time Complexity: O(n) | Space Complexity: O(1)
Idea: Track both max and min products (negative can become positive).

---

322. Coin Change

You are given an integer array coins representing coins of different denominations and an integer amount representing a total amount of money.

Return *the fewest number of coins that you need to make up that amount*. If that amount of money cannot be made up by any combination of the coins, return -1.

You may assume that you have an infinite number of each kind of coin.

Example 1:

Input: coins = [1,2,5], amount = 11
Output: 3
Explanation: 11 = 5 + 5 + 1

Example 2:

Input: coins = [2], amount = 3
Output: -1

Example 3:

Input: coins = [1], amount = 0
Output: 0

Solution 1: Recursive

```
def coinChange(self, coins: List[int], amount: int) -> int:
    def dfs(remaining):
        if remaining < 0:
            return float('inf')
        if remaining == 0:
            return 0

        min_coins = float('inf')
        for coin in coins:
            min_coins = min(min_coins, 1 + dfs(remaining - coin))

        return min_coins

    result = dfs(amount)
    return result if result != float('inf') else -1
```

Time Complexity: O($amount^n$) | Space Complexity: O(amount)
Idea: Try all coin combinations recursively.

Solution 2: Memoization

```
def coinChange(self, coins: List[int], amount: int) -> int:
    memo = {}

    def dfs(remaining):
        if remaining < 0:
            return float('inf')
        if remaining == 0:
            return 0
        if remaining in memo:
            return memo[remaining]

        min_coins = float('inf')
        for coin in coins:
            min_coins = min(min_coins, 1 + dfs(remaining - coin))

        memo[remaining] = min_coins
        return min_coins

    result = dfs(amount)
    return result if result != float('inf') else -1
```

Time Complexity: O(amount × len(coins)) | Space Complexity: O(amount)
Idea: Cache results for each remaining amount.

Solution 3: Dynamic Programming (Bottom-Up)

```
def coinChange(self, coins: List[int], amount: int) -> int:
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0

    for i in range(1, amount + 1):
        for coin in coins:
            if coin <= i:
                dp[i] = min(dp[i], dp[i - coin] + 1)

    return dp[amount] if dp[amount] != float('inf') else -1
```

Time Complexity: O(amount × len(coins)) | Space Complexity: O(amount)
Idea: Build solution for each amount from 0 to target.

Solution 4: BFS

```
def coinChange(self, coins: List[int], amount: int) -> int:
    if amount == 0:
        return 0

    from collections import deque
    queue = deque([(0, 0)])  # (current_amount, steps)
    visited = set([0])

    while queue:
        curr, steps = queue.popleft()

        for coin in coins:
            next_amount = curr + coin
            if next_amount == amount:
                return steps + 1
            if next_amount < amount and next_amount not in visited:
                visited.add(next_amount)
                queue.append((next_amount, steps + 1))

    return -1
```

Time Complexity: O(amount × len(coins)) | Space Complexity: O(amount)
Idea: Treat as shortest path problem in amount space.

518. Coin Change II

You are given an integer array coins representing coins of different denominations and an integer amount representing a total amount of money.

Return *the number of combinations that make up that amount*. If that amount of money cannot be made up by any combination of the coins, return 0.

You may assume that you have an infinite number of each kind of coin.

The answer is **guaranteed** to fit into a signed **32-bit** integer.

Example 1:

Input: amount = 5, coins = [1,2,5]
Output: 4
Explanation: there are four ways to make up the amount:
5=5
5=2+2+1
5=2+1+1+1
5=1+1+1+1+1

Example 2:

Input: amount = 3, coins = [2]
Output: 0
Explanation: the amount of 3 cannot be made up just with coins of 2.

Example 3:

Input: amount = 10, coins = [10]
Output: 1

Solution 1: Recursive

```
def change(self, amount: int, coins: List[int]) -> int:
    def dfs(i, remaining):
        if remaining == 0:
            return 1
        if i == len(coins) or remaining < 0:
            return 0

        # Include coin i or skip it
        return dfs(i, remaining - coins[i]) + dfs(i + 1, remaining)

    return dfs(0, amount)
```

Time Complexity: O(2^n) | Space Complexity: O(n)
Idea: At each coin, either use it or skip it.

Solution 2: Memoization

```
def change(self, amount: int, coins: List[int]) -> int:
    memo = {}

    def dfs(i, remaining):
        if remaining == 0:
            return 1
        if i == len(coins) or remaining < 0:
            return 0
        if (i, remaining) in memo:
            return memo[(i, remaining)]

        memo[(i, remaining)] = dfs(i, remaining - coins[i]) +
dfs(i + 1, remaining)
        return memo[(i, remaining)]

    return dfs(0, amount)
```

Time Complexity: O(len(coins) × amount) | Space Complexity: O(len(coins) × amount)
Idea: Cache results for (coin index, remaining amount).

Solution 3: 2D Dynamic Programming

```
def change(self, amount: int, coins: List[int]) -> int:
    dp = [[0] * (amount + 1) for _ in range(len(coins) + 1)]
    dp[0][0] = 1

    for i in range(1, len(coins) + 1):
        dp[i][0] = 1
        for j in range(1, amount + 1):
            dp[i][j] = dp[i-1][j]  # skip coin
            if j >= coins[i-1]:
                dp[i][j] += dp[i][j - coins[i-1]]  # use coin

    return dp[len(coins)][amount]
```

Time Complexity: O(len(coins) × amount) | Space Complexity: O(len(coins) × amount)
Idea: dp[i][j] = combinations using first i coins to make amount j.

Solution 4: 1D Dynamic Programming

```
def change(self, amount: int, coins: List[int]) -> int:
    dp = [0] * (amount + 1)
    dp[0] = 1

    for coin in coins:
        for j in range(coin, amount + 1):
            dp[j] += dp[j - coin]

    return dp[amount]
```

Time Complexity: O(len(coins) × amount) | Space Complexity: O(amount)
Idea: Process coins one by one to avoid counting permutations.

300. Longest Increasing Subsequence

Given an integer array nums, return *the length of the longest* ***strictly increasing subsequence***.

Example 1:

Input: nums = [10,9,2,5,3,7,101,18]
Output: 4
Explanation: The longest increasing subsequence is [2,3,7,101], therefore the length is 4.

Example 2:

Input: nums = [0,1,0,3,2,3]
Output: 4

Example 3:

Input: nums = [7,7,7,7,7,7,7]
Output: 1

Solution 1: Recursive

```
def lengthOfLIS(self, nums: List[int]) -> int:
    def dfs(i, prev):
        if i == len(nums):
            return 0

        taken = 0
        if nums[i] > prev:
            taken = 1 + dfs(i + 1, nums[i])
        not_taken = dfs(i + 1, prev)

        return max(taken, not_taken)

    return dfs(0, float('-inf'))
```

Time Complexity: $O(2^n)$ | Space Complexity: O(n)
Idea: At each element, either take it if increasing or skip it.

Solution 2: Memoization

```
def lengthOfLIS(self, nums: List[int]) -> int:
    memo = {}

    def dfs(i, prev_index):
        if i == len(nums):
            return 0
        if (i, prev_index) in memo:
            return memo[(i, prev_index)]

        taken = 0
        if prev_index == -1 or nums[i] > nums[prev_index]:
            taken = 1 + dfs(i + 1, i)
        not_taken = dfs(i + 1, prev_index)

        memo[(i, prev_index)] = max(taken, not_taken)
        return memo[(i, prev_index)]

    return dfs(0, -1)
```

Time Complexity: $O(n^2)$ | Space Complexity: $O(n^2)$
Idea: Cache with previous index.

Solution 3: Dynamic Programming $O(n^2)$

```
def lengthOfLIS(self, nums: List[int]) -> int:
    dp = [1] * len(nums)

    for i in range(len(nums)):
        for j in range(i):
            if nums[i] > nums[j]:
                dp[i] = max(dp[i], dp[j] + 1)

    return max(dp)
```

Time Complexity: $O(n^2)$ | Space Complexity: O(n)
Idea: dp[i] = LIS ending at i.

Solution 4: Binary Search O(n log n)

```
def lengthOfLIS(self, nums: List[int]) -> int:
    tails = []

    for num in nums:
        left, right = 0, len(tails)
        while left < right:
            mid = (left + right) // 2
            if tails[mid] < num:
                left = mid + 1
            else:
                right = mid

        if left == len(tails):
            tails.append(num)
        else:
            tails[left] = num

    return len(tails)
```

Time Complexity: O(n log n) | Space Complexity: O(n)
Idea: Maintain tails of increasing subsequences.

Solution 5: Patience Sorting

```
def lengthOfLIS(self, nums: List[int]) -> int:
    piles = []

    for num in nums:
        pile = bisect_left(piles, num)
        if pile == len(piles):
            piles.append(num)
        else:
            piles[pile] = num

    return len(piles)
```

Time Complexity: O(n log n) | Space Complexity: O(n)
Idea: Simulate patience card game.

139. Word Break

Given a string s and a dictionary of strings wordDict, return true if s can be segmented into a space-separated sequence of one or more dictionary words.

Note that the same word in the dictionary may be reused multiple times in the segmentation.

Example 1:

Input: s = "leetcode", wordDict = ["leet","code"]
Output: true
Explanation: Return true because "leetcode" can be segmented as "leet code".

Example 2:

Input: s = "applepenapple", wordDict = ["apple","pen"]
Output: true
Explanation: Return true because "applepenapple" can be segmented as "apple pen apple".
Note that you are allowed to reuse a dictionary word.

Example 3:

Input: s = "catsandog", wordDict = ["cats","dog","sand","and","cat"]
Output: false

Solution 1: Recursive

```
def wordBreak(self, s: str, wordDict: List[str]) -> bool:
    word_set = set(wordDict)

    def dfs(start):
        if start == len(s):
            return True

        for end in range(start + 1, len(s) + 1):
            if s[start:end] in word_set and dfs(end):
                return True

        return False

    return dfs(0)
```

Time Complexity: $O(2^n)$ | Space Complexity: O(n)
Idea: Try all possible word breaks.

Solution 2: Memoization

```
def wordBreak(self, s: str, wordDict: List[str]) -> bool:
    word_set = set(wordDict)
    memo = {}

    def dfs(start):
        if start == len(s):
            return True
        if start in memo:
            return memo[start]

        for end in range(start + 1, len(s) + 1):
            if s[start:end] in word_set and dfs(end):
                memo[start] = True
                return True

        memo[start] = False
        return False

    return dfs(0)
```

Time Complexity: $O(n^2)$ | Space Complexity: O(n)
Idea: Cache results for each starting index.

Solution 3: Dynamic Programming

```
def wordBreak(self, s: str, wordDict: List[str]) -> bool:
    word_set = set(wordDict)
    dp = [False] * (len(s) + 1)
    dp[0] = True

    for i in range(1, len(s) + 1):
        for j in range(i):
            if dp[j] and s[j:i] in word_set:
                dp[i] = True
                break

    return dp[len(s)]
```

Time Complexity: $O(n^2)$ | Space Complexity: O(n)
Idea: dp[i] = can break s[0:i].

Solution 4: BFS

```
def wordBreak(self, s: str, wordDict: List[str]) -> bool:
    word_set = set(wordDict)
    from collections import deque

    queue = deque([0])
    visited = set([0])

    while queue:
        start = queue.popleft()

        for end in range(start + 1, len(s) + 1):
            if end in visited:
                continue

            if s[start:end] in word_set:
                if end == len(s):
                    return True
                queue.append(end)
                visited.add(end)

    return False
```

Time Complexity: $O(n^2)$ | Space Complexity: O(n)
Idea: BFS over possible break positions.

Solution 5: Trie Optimization

```
def wordBreak(self, s: str, wordDict: List[str]) -> bool:
    class TrieNode:
        def __init__(self):
            self.children = {}
            self.is_word = False

    # Build Trie
    root = TrieNode()
    for word in wordDict:
        node = root
        for c in word:
            if c not in node.children:
                node.children[c] = TrieNode()
            node = node.children[c]
        node.is_word = True

    dp = [False] * (len(s) + 1)
    dp[0] = True

    for i in range(len(s)):
        if not dp[i]:
            continue

        node = root
        for j in range(i, len(s)):
            if s[j] not in node.children:
                break
            node = node.children[s[j]]
            if node.is_word:
                dp[j + 1] = True

    return dp[len(s)]
```

Time Complexity: $O(n^2)$ | Space Complexity: O(n + total chars in dict)
Idea: Use Trie for faster word lookup.

---

91. Decode Ways

You have intercepted a secret message encoded as a string of numbers. The message is **decoded** via the following mapping:

"1" -> 'A'

"2" -> 'B'

...

"25" -> 'Y'

"26" -> 'Z'

However, while decoding the message, you realize that there are many different ways you can decode the message because some codes are contained in other codes ("2" and "5" vs "25").

For example, "11106" can be decoded into:

- "AAJF" with the grouping (1, 1, 10, 6)
- "KJF" with the grouping (11, 10, 6)
- The grouping (1, 11, 06) is invalid because "06" is not a valid code (only "6" is valid).

Note: there may be strings that are impossible to decode.

Given a string s containing only digits, return the **number of ways** to **decode** it. If the entire string cannot be decoded in any valid way, return 0.

The test cases are generated so that the answer fits in a **32-bit** integer.

Example 1:

Input: s = "12"

Output: 2

Explanation:

"12" could be decoded as "AB" (1 2) or "L" (12).

Example 2:

Input: s = "226"

Output: 3

Explanation:

"226" could be decoded as "BZ" (2 26), "VF" (22 6), or "BBF" (2 2 6).

Example 3:

Input: s = "06"

Output: 0

Explanation:

"06" cannot be mapped to "F" because of the leading zero ("6" is different from "06"). In this case, the string is not a valid encoding, so return 0.

Solution 1: Recursive

```
def numDecodings(self, s: str) -> int:
    def dfs(i):
        if i == len(s):
            return 1
        if s[i] == '0':
            return 0

        # Take one digit
        ways = dfs(i + 1)

        # Take two digits if valid
        if i + 1 < len(s) and int(s[i:i+2]) <= 26:
            ways += dfs(i + 2)

        return ways

    return dfs(0)
```

Time Complexity: $O(2^n)$ | Space Complexity: O(n)
Idea: Decode one or two digits at each step.

Solution 2: Memoization

```
def numDecodings(self, s: str) -> int:
    memo = {}

    def dfs(i):
        if i == len(s):
            return 1
        if s[i] == '0':
            return 0
        if i in memo:
            return memo[i]

        ways = dfs(i + 1)

        if i + 1 < len(s) and int(s[i:i+2]) <= 26:
            ways += dfs(i + 2)

        memo[i] = ways
        return ways

    return dfs(0)
```

Time Complexity: O(n) | Space Complexity: O(n)
Idea: Cache results for each index.

Solution 3: Dynamic Programming

```
def numDecodings(self, s: str) -> int:
    if not s or s[0] == '0':
        return 0

    n = len(s)
    dp = [0] * (n + 1)
    dp[0] = 1
    dp[1] = 1

    for i in range(2, n + 1):
        # One digit
        if s[i-1] != '0':
            dp[i] += dp[i-1]

        # Two digits
        two_digit = int(s[i-2:i])
        if 10 <= two_digit <= 26:
            dp[i] += dp[i-2]

    return dp[n]
```

Time Complexity: O(n) | Space Complexity: O(n)
Idea: dp[i] = ways to decode first i characters.

Solution 4: Space Optimized DP

```
def numDecodings(self, s: str) -> int:
    if not s or s[0] == '0':
        return 0

    prev2 = 1  # dp[i-2]
    prev1 = 1  # dp[i-1]

    for i in range(1, len(s)):
        curr = 0

        # One digit
        if s[i] != '0':
            curr += prev1

        # Two digits
        two_digit = int(s[i-1:i+1])
        if 10 <= two_digit <= 26:
            curr += prev2

        prev2 = prev1
        prev1 = curr

        if prev1 == 0:  # Early termination if no ways
            return 0

    return prev1
```

Time Complexity: O(n) | Space Complexity: O(1)
Idea: Only need last two values.

Solution 5: Constant Space with Single Variable

```
def numDecodings(self, s: str) -> int:
    if not s or s[0] == '0':
        return 0

    prev = 1  # dp[i-1]
    prev_prev = 1  # dp[i-2]

    for i in range(1, len(s)):
        current = 0

        if s[i] != '0':
            current = prev

        two_digit = int(s[i-1:i+1])
        if 10 <= two_digit <= 26:
            current += prev_prev

        prev_prev = prev
        prev = current

    return prev
```

Time Complexity: O(n) | Space Complexity: O(1)

416. Partition Equal Subset Sum

Given an integer array nums, return true *if you can partition the array into two subsets such that the sum of the elements in both subsets is equal or* false *otherwise*.

Example 1:

Input: nums = [1,5,11,5]
Output: true
Explanation: The array can be partitioned as [1, 5, 5] and [11].

Example 2:

Input: nums = [1,2,3,5]
Output: false
Explanation: The array cannot be partitioned into equal sum subsets.

Solution 1: Recursive

```
def canPartition(self, nums: List[int]) -> bool:
    total = sum(nums)
    if total % 2:
        return False

    target = total // 2

    def dfs(i, remaining):
        if remaining == 0:
            return True
        if i == len(nums) or remaining < 0:
            return False

        # Include or exclude current number
        return dfs(i + 1, remaining - nums[i]) or dfs(i + 1,
remaining)

    return dfs(0, target)
```

Time Complexity: $O(2^n)$ | Space Complexity: O(n)
Idea: Try all subsets to reach target sum.

Solution 2: Memoization

```
def canPartition(self, nums: List[int]) -> bool:
    total = sum(nums)
    if total % 2:
        return False

    target = total // 2
    memo = {}

    def dfs(i, remaining):
        if remaining == 0:
            return True
        if i == len(nums) or remaining < 0:
            return False
        if (i, remaining) in memo:
            return memo[(i, remaining)]

        memo[(i, remaining)] = dfs(i + 1, remaining - nums[i])
or dfs(i + 1, remaining)
        return memo[(i, remaining)]

    return dfs(0, target)
```

Time Complexity: O(n × target) | Space Complexity: O(n × target)
Idea: Cache (index, remaining) pairs.

Solution 3: 2D Dynamic Programming

```
def canPartition(self, nums: List[int]) -> bool:
    total = sum(nums)
    if total % 2:
        return False

    target = total // 2
    n = len(nums)
    dp = [[False] * (target + 1) for _ in range(n + 1)]
    dp[0][0] = True

    for i in range(1, n + 1):
        dp[i][0] = True
        for j in range(1, target + 1):
            dp[i][j] = dp[i-1][j]
            if j >= nums[i-1]:
                dp[i][j] = dp[i][j] or dp[i-1][j - nums[i-1]]

    return dp[n][target]
```

Time Complexity: O(n × target) | Space Complexity: O(n × target)
Idea: dp[i][j] = can sum j using first i numbers.

Solution 4: 1D Dynamic Programming

```
def canPartition(self, nums: List[int]) -> bool:
    total = sum(nums)
    if total % 2:
        return False

    target = total // 2
    dp = [False] * (target + 1)
    dp[0] = True

    for num in nums:
        for j in range(target, num - 1, -1):
            if dp[j - num]:
                dp[j] = True

    return dp[target]
```

Time Complexity: O(n × target) | Space Complexity: O(target)
Idea: Process from right to left to avoid reusing same number.

Solution 5: Bitmask Optimization

```
def canPartition(self, nums: List[int]) -> bool:
    total = sum(nums)
    if total % 2:
        return False

    target = total // 2
    bits = 1  # bitmask representing achievable sums

    for num in nums:
        bits |= bits << num

    # Check if target bit is set
    return (bits >> target) & 1 == 1
```

Time Complexity: O(n × target / word_size) | Space Complexity: O(target / word_size)
Idea: Use bitmask for faster subset sum computation.

343. Integer Break

Given an integer n, break it into the sum of k **positive integers**, where k >= 2, and maximize the product of those integers.

Return *the maximum product you can get*.

Example 1:

Input: n = 2
Output: 1
Explanation: 2 = 1 + 1, 1 × 1 = 1.

Example 2:

Input: n = 10
Output: 36
Explanation: 10 = 3 + 3 + 4, 3 × 3 × 4 = 36.

Solution 1: Recursive

```
def integerBreak(self, n: int) -> int:
    def dfs(num):
        if num <= 3:
            return num

        max_product = 0
        for i in range(1, num):
            max_product = max(max_product, i * dfs(num - i))

        return max_product

    if n <= 3:
        return n - 1
    return dfs(n)
```

Time Complexity: $O(n^n)$ | Space Complexity: O(n)
Idea: Try all possible first splits.

Solution 2: Memoization

```
def integerBreak(self, n: int) -> int:
    memo = {}

    def dfs(num):
        if num <= 3:
            return num
        if num in memo:
            return memo[num]

        max_product = 0
        for i in range(1, num):
            max_product = max(max_product, i * dfs(num - i))

        memo[num] = max_product
        return max_product

    if n <= 3:
        return n - 1
    return dfs(n)
```

Time Complexity: $O(n^2)$ | Space Complexity: O(n)
Idea: Cache results for each number.

Solution 3: Dynamic Programming

```
def integerBreak(self, n: int) -> int:
    if n <= 3:
        return n - 1

    dp = [0] * (n + 1)
    dp[1] = 1
    dp[2] = 2
    dp[3] = 3

    for i in range(4, n + 1):
        for j in range(1, i):
            dp[i] = max(dp[i], j * dp[i - j])

    return dp[n]
```

Time Complexity: $O(n^2)$ | Space Complexity: O(n)
Idea: dp[i] = max product for number i.

Solution 4: Mathematical (Using 3's)

```
def integerBreak(self, n: int) -> int:
    if n <= 3:
        return n - 1

    quotient, remainder = divmod(n, 3)

    if remainder == 0:
        return 3  quotient
    elif remainder == 1:
        return 3  (quotient - 1) * 4
    else:  # remainder == 2
        return 3  quotient * 2
```

Time Complexity: O(log n) | Space Complexity: O(1)
Idea: Break into 3's as much as possible (3 gives best product).

Solution 5: Mathematical with Exponentiation Optimization

```
def integerBreak(self, n: int) -> int:
    if n <= 3:
        return n - 1

    if n % 3 == 0:
        return 3  (n // 3)
    elif n % 3 == 1:
        return 3  (n // 3 - 1) * 4
    else:
        return 3  (n // 3) * 2
```

Time Complexity: O(log n) | Space Complexity: O(1)
Idea: Optimized version of mathematical approach.

These solutions cover all major approaches for each problem, from basic brute force to optimized solutions with various trade-offs in time and space complexity.

2D DYNAMIC PROGRAMMING

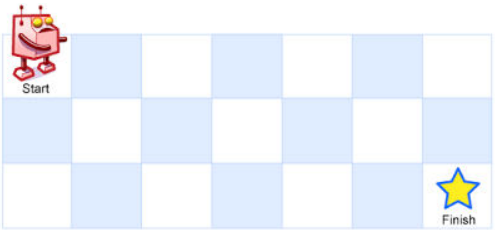
62. Unique Paths

There is a robot on an m x n grid. The robot is initially located at the **top-left corner** (i.e., grid[0][0]). The robot tries to move to the **bottom-right corner** (i.e., grid[m - 1][n - 1]). The robot can only move either down or right at any point in time.

Given the two integers m and n, return *the number of possible unique paths that the robot can take to reach the bottom-right corner.*

The test cases are generated so that the answer will be less than or equal to $2 * 10^9$.

Example 1:



Input: m = 3, n = 7
Output: 28

Example 2:

Input: m = 3, n = 2
Output: 3
Explanation: From the top-left corner, there are a total of 3 ways to reach the bottom-right corner:
1. Right -> Down -> Down
2. Down -> Down -> Right
3. Down -> Right -> Down

Approach 1. Math (Combinatorics)

```
def uniquePaths(self, m: int, n: int) -> int:
    # Total steps: (m-1) down + (n-1) right = m+n-2
    # Choose positions for down moves: C(m+n-2, m-1)

    return comb(m + n - 2, m - 1)
```

- Time: O(min(m,n))
- Space: O(1)

Approach 2. 2D DP

```
def uniquePaths(self, m: int, n: int) -> int:
    dp = [[1] * n for _ in range(m)]

    for i in range(1, m):
        for j in range(1, n):
            dp[i][j] = dp[i-1][j] + dp[i][j-1]

    return dp[m-1][n-1]
```

- Time: O(m×n)
- Space: O(m×n)

Approach 3. 1D DP (Space Optimized)

```
def uniquePaths(self, m: int, n: int) -> int:
    dp = [1] * n

    for i in range(1, m):
        for j in range(1, n):
            dp[j] += dp[j-1]

    return dp[n-1]
```

- Time: O(m×n)
- Space: O(n)

---

63. Unique Paths II

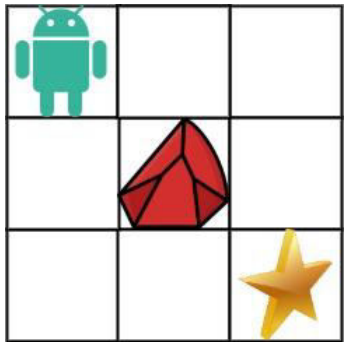
You are given an m x n integer array grid. There is a robot initially located at the **top-left corner** (i.e., grid[0][0]). The robot tries to move to the **bottom-right corner** (i.e., grid[m - 1][n - 1]). The robot can only move either down or right at any point in time.

An obstacle and space are marked as 1 or 0 respectively in grid. A path that the robot takes cannot include **any** square that is an obstacle.

Return *the number of possible unique paths that the robot can take to reach the bottom-right corner.*

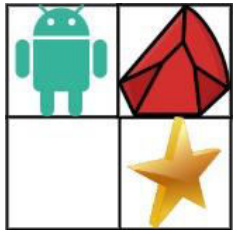
The testcases are generated so that the answer will be less than or equal to $2 * 10^9$.

Example 1:



Input: obstacleGrid = [[0,0,0],[0,1,0],[0,0,0]]
Output: 2
Explanation: There is one obstacle in the middle of the 3x3 grid above.
There are two ways to reach the bottom-right corner:
1. Right -> Right -> Down -> Down
2. Down -> Down -> Right -> Right

Example 2:



Input: obstacleGrid = [[0,1],[0,0]]
Output: 1

Approach 1. 2D DP with Obstacles

```
def uniquePathsWithObstacles(self, obstacleGrid: List[List[int]]) -> int:
    m, n = len(obstacleGrid), len(obstacleGrid[0])

    # If start or end is obstacle
    if obstacleGrid[0][0] == 1 or obstacleGrid[m-1][n-1] == 1:
        return 0

    dp = [[0] * n for _ in range(m)]
    dp[0][0] = 1

    # Initialize first row
    for j in range(1, n):
        dp[0][j] = dp[0][j-1] if obstacleGrid[0][j] == 0 else 0

    # Initialize first column
    for i in range(1, m):
        dp[i][0] = dp[i-1][0] if obstacleGrid[i][0] == 0 else 0

    # Fill DP table
    for i in range(1, m):
        for j in range(1, n):
            if obstacleGrid[i][j] == 0:
                dp[i][j] = dp[i-1][j] + dp[i][j-1]

    return dp[m-1][n-1]
```

- Time: O(m×n)
- Space: O(m×n)

Approach 2. 1D Space Optimized

```
def uniquePathsWithObstacles(self, obstacleGrid: List[List[int]]) -> int:
    m, n = len(obstacleGrid), len(obstacleGrid[0])

    if obstacleGrid[0][0] == 1:
        return 0

    dp = [0] * n
    dp[0] = 1

    for i in range(m):
        for j in range(n):
            if obstacleGrid[i][j] == 1:
                dp[j] = 0
            elif j > 0:
                dp[j] += dp[j-1]

    return dp[n-1]
```

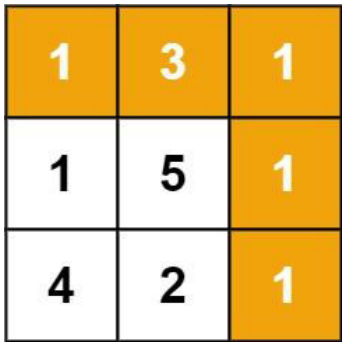
- Time: O(m×n)
- Space: O(n)

64. Minimum Path Sum

Given a m x n grid filled with non-negative numbers, find a path from top left to bottom right, which minimizes the sum of all numbers along its path.

Note: You can only move either down or right at any point in time.

Example 1:



Input: grid = [[1,3,1],[1,5,1],[4,2,1]]
Output: 7
Explanation: Because the path 1 → 3 → 1 → 1 → 1 minimizes the sum.

Example 2:

Input: grid = [[1,2,3],[4,5,6]]
Output: 12

Approach 1. 2D DP

```
def minPathSum(self, grid: List[List[int]]) -> int:
    m, n = len(grid), len(grid[0])
    dp = [[0] * n for _ in range(m)]
    dp[0][0] = grid[0][0]

    # Initialize first row
    for j in range(1, n):
        dp[0][j] = dp[0][j-1] + grid[0][j]

    # Initialize first column
    for i in range(1, m):
        dp[i][0] = dp[i-1][0] + grid[i][0]

    # Fill DP table
    for i in range(1, m):
        for j in range(1, n):
            dp[i][j] = grid[i][j] + min(dp[i-1][j], dp[i][j-1])

    return dp[m-1][n-1]
```

- Time: O(m×n)
- Space: O(m×n)

Approach 2. 1D Space Optimized

```
def minPathSum(self, grid: List[List[int]]) -> int:
    m, n = len(grid), len(grid[0])
    dp = [0] * n
    dp[0] = grid[0][0]

    # Initialize first row
    for j in range(1, n):
        dp[j] = dp[j-1] + grid[0][j]

    # Process remaining rows
    for i in range(1, m):
        dp[0] += grid[i][0]
        for j in range(1, n):
            dp[j] = grid[i][j] + min(dp[j], dp[j-1])

    return dp[n-1]
```

- Time: O(m×

1143. Longest Common Subsequence

Given two strings text1 and text2, return *the length of their longest **common subsequence***. If there is no **common subsequence**, return 0.

A **subsequence** of a string is a new string generated from the original string with some characters (can be none) deleted without changing the relative order of the remaining characters.

- For example, "ace" is a subsequence of "abcde".

A **common subsequence** of two strings is a subsequence that is common to both strings.

Example 1:

Input: text1 = "abcde", text2 = "ace"
Output: 3
Explanation: The longest common subsequence is "ace" and its length is 3.

Example 2:

Input: text1 = "abc", text2 = "abc"
Output: 3
Explanation: The longest common subsequence is "abc" and its length is 3.

Example 3:

Input: text1 = "abc", text2 = "def"
Output: 0
Explanation: There is no such common subsequence, so the result is 0.

Solution 1: Recursive (Top-Down)

```
def longestCommonSubsequence(self, text1: str, text2: str) -> int:
    def dfs(i, j):
        if i == len(text1) or j == len(text2):
            return 0

        if text1[i] == text2[j]:
            return 1 + dfs(i + 1, j + 1)
        else:
            return max(dfs(i + 1, j), dfs(i, j + 1))

    return dfs(0, 0)
```

Time Complexity: O(2^(m+n)) | Space Complexity: O(m+n)
Idea: Recursively explore all possibilities - match characters or skip from either string.

Solution 2: Memoization (Top-Down DP)

```
def longestCommonSubsequence(self, text1: str, text2: str) -> int:
    memo = {}

    def dfs(i, j):
        if i == len(text1) or j == len(text2):
            return 0

        if (i, j) in memo:
            return memo[(i, j)]

        if text1[i] == text2[j]:
            memo[(i, j)] = 1 + dfs(i + 1, j + 1)
        else:
            memo[(i, j)] = max(dfs(i + 1, j), dfs(i, j + 1))

        return memo[(i, j)]

    return dfs(0, 0)
```

Time Complexity: O(m×n) | Space Complexity: O(m×n)
Idea: Cache results of subproblems to avoid redundant calculations.

Solution 3: 2D Dynamic Programming (Bottom-Up)

```
def longestCommonSubsequence(self, text1: str, text2: str) -> int:
    m, n = len(text1), len(text2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if text1[i-1] == text2[j-1]:
                dp[i][j] = 1 + dp[i-1][j-1]
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])

    return dp[m][n]
```

Time Complexity: O(m×n) | Space Complexity: O(m×n)
Idea: Build solution iteratively using DP table where dp[i][j] represents LCS of first i chars of text1 and first j chars of text2.

Solution 4: Space Optimized DP

```
def longestCommonSubsequence(self, text1: str, text2: str) -> int:
    m, n = len(text1), len(text2)

    # Use only two rows
    prev = [0] * (n + 1)
    curr = [0] * (n + 1)

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if text1[i-1] == text2[j-1]:
                curr[j] = 1 + prev[j-1]
            else:
                curr[j] = max(prev[j], curr[j-1])
        prev, curr = curr, [0] * (n + 1)

    return prev[n]
```

Time Complexity: O(m×n) | Space Complexity: O(n)
Idea: Since we only need previous row, maintain just two arrays.

72. Edit Distance

Given two strings word1 and word2, return *the minimum number of operations required to convert word1 to word2*.

You have the following three operations permitted on a word:

- Insert a character
- Delete a character
- Replace a character

Example 1:

Input: word1 = "horse", word2 = "ros"
Output: 3
Explanation:
horse -> rorse (replace 'h' with 'r')
rorse -> rose (remove 'r')
rose -> ros (remove 'e')

Example 2:

Input: word1 = "intention", word2 = "execution"
Output: 5
Explanation:
intention -> inention (remove 't')
inention -> enention (replace 'i' with 'e')
enention -> exention (replace 'n' with 'x')
exention -> exection (replace 'n' with 'c')
execution -> execution (insert 'u')

Solution 1: Recursive

```
def minDistance(self, word1: str, word2: str) -> int:
    def dfs(i, j):
        if i == len(word1):
            return len(word2) - j  # insert remaining chars
        if j == len(word2):
            return len(word1) - i  # delete remaining chars

        if word1[i] == word2[j]:
            return dfs(i + 1, j + 1)
        else:
            insert = dfs(i, j + 1)      # insert word2[j] into word1
            delete = dfs(i + 1, j)      # delete word1[i]
            replace = dfs(i + 1, j + 1) # replace word1[i] with word2[j]
            return 1 + min(insert, delete, replace)

    return dfs(0, 0)
```

Time Complexity: O(3^(m+n)) | Space Complexity: O(m+n)
Idea: Try all three operations at each mismatch.

Solution 2: Memoization

```
def minDistance(self, word1: str, word2: str) -> int:
    memo = {}

    def dfs(i, j):
        if i == len(word1):
            return len(word2) - j
        if j == len(word2):
            return len(word1) - i

        if (i, j) in memo:
            return memo[(i, j)]

        if word1[i] == word2[j]:
            memo[(i, j)] = dfs(i + 1, j + 1)
        else:
            insert = dfs(i, j + 1)
            delete = dfs(i + 1, j)
            replace = dfs(i + 1, j + 1)
            memo[(i, j)] = 1 + min(insert, delete, replace)

        return memo[(i, j)]

    return dfs(0, 0)
```

Time Complexity: O(m×n) | Space Complexity: O(m×n)
Idea: Cache results of (i,j) pairs.

Solution 3: 2D Dynamic Programming

```
def minDistance(self, word1: str, word2: str) -> int:
    m, n = len(word1), len(word2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    # Base cases
    for i in range(m + 1):
        dp[i][0] = i  # deleting all chars
    for j in range(n + 1):
        dp[0][j] = j  # inserting all chars

    # Fill DP table
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if word1[i-1] == word2[j-1]:
                dp[i][j] = dp[i-1][j-1]
            else:
                dp[i][j] = 1 + min(
                    dp[i-1][j],    # delete
                    dp[i][j-1],    # insert
                    dp[i-1][j-1]   # replace
                )

    return dp[m][n]
```

Time Complexity: O(m×n) | Space Complexity: O(m×n)
Idea: dp[i][j] = min edits for first i chars of word1 and first j chars of word2.

Solution 4: Space Optimized DP

```
def minDistance(self, word1: str, word2: str) -> int:
    m, n = len(word1), len(word2)

    # Use two rows
    prev = [j for j in range(n + 1)]
    curr = [0] * (n + 1)

    for i in range(1, m + 1):
        curr[0] = i  # delete all chars
        for j in range(1, n + 1):
            if word1[i-1] == word2[j-1]:
                curr[j] = prev[j-1]
            else:
                curr[j] = 1 + min(prev[j], curr[j-1], prev[j-1])
        prev, curr = curr, [0] * (n + 1)

    return prev[n]
```

Time Complexity: O(m×n) | Space Complexity: O(n)
Idea: Only need previous row for calculations.

5. Longest Palindromic Substring

Given a string s, return *the longest palindromic substring* in s.

Example 1:

Input: s = "babad"
Output: "bab"
Explanation: "aba" is also a valid answer.

Example 2:

Input: s = "cbbd"
Output: "bb"

Solution 1: Brute Force

```
def longestPalindrome(self, s: str) -> str:
    def is_palindrome(i, j):
        while i < j:
            if s[i] != s[j]:
                return False
            i += 1
            j -= 1
        return True

    n = len(s)
    longest = ""

    for i in range(n):
        for j in range(i, n):
            if is_palindrome(i, j) and j - i + 1 > len(longest):
                longest = s[i:j+1]

    return longest
```

Time Complexity: $O(n^3)$ | Space Complexity: $O(1)$
Idea: Check all substrings if they're palindrome.

Solution 2: Expand Around Center

```
def longestPalindrome(self, s: str) -> str:
    def expand_around_center(left, right):
        while left >= 0 and right < len(s) and s[left] == s[right]:
            left -= 1
            right += 1
        return s[left+1:right]

    longest = ""

    for i in range(len(s)):
        # Odd length palindrome
        odd = expand_around_center(i, i)
        if len(odd) > len(longest):
            longest = odd

        # Even length palindrome
        even = expand_around_center(i, i + 1)
        if len(even) > len(longest):
            longest = even

    return longest
```

Time Complexity: $O(n^2)$ | Space Complexity: $O(1)$
Idea: Expand from each center (n centers for odd, n-1 for even).

Solution 3: Dynamic Programming

```
def longestPalindrome(self, s: str) -> str:
    n = len(s)
    dp = [[False] * n for _ in range(n)]
    longest = s[0]

    # All single characters are palindromes
    for i in range(n):
        dp[i][i] = True

    # Check length 2 palindromes
    for i in range(n-1):
        if s[i] == s[i+1]:
            dp[i][i+1] = True
            longest = s[i:i+2]

    # Check lengths >= 3
    for length in range(3, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1
            if s[i] == s[j] and dp[i+1][j-1]:
                dp[i][j] = True
                if length > len(longest):
                    longest = s[i:j+1]

    return longest
```

Time Complexity: $O(n^2)$ | Space Complexity: $O(n^2)$
Idea: dp[i][j] = True if s[i:j+1] is palindrome.

Solution 4: Manacher's Algorithm

```
def longestPalindrome(self, s: str) -> str:
    # Transform string to handle even length palindromes
    T = '#'.join('^{}$'.format(s))
    n = len(T)
    P = [0] * n
    center = right = 0

    for i in range(1, n-1):
        mirror = 2 * center - i

        if i < right:
            P[i] = min(right - i, P[mirror])

        # Expand around center i
        while T[i + 1 + P[i]] == T[i - 1 - P[i]]:
            P[i] += 1

        # Adjust center and right bound
        if i + P[i] > right:
            center = i
            right = i + P[i]

    # Find maximum
    max_len, center_index = max((n, i) for i, n in
enumerate(P))
    start = (center_index - max_len) // 2
    return s[start:start + max_len]
```

Time Complexity: $O(n)$ | Space Complexity: $O(n)$
Idea: Linear time algorithm using palindrome symmetry.

516. Longest Palindromic Subsequence

Given a string s, find *the longest palindromic **subsequence**'s length in* s.

A **subsequence** is a sequence that can be derived from another sequence by deleting some or no elements without changing the order of the remaining elements.

Example 1:

Input: s = "bbbab"
Output: 4
Explanation: One possible longest palindromic subsequence is "bbbb".

Example 2:

Input: s = "cbbd"
Output: 2
Explanation: One possible longest palindromic subsequence is "bb".

Solution 1: Recursive

```
def longestPalindromeSubseq(self, s: str) -> int:
    def dfs(i, j):
        if i > j:
            return 0
        if i == j:
            return 1

        if s[i] == s[j]:
            return 2 + dfs(i + 1, j - 1)
        else:
            return max(dfs(i + 1, j), dfs(i, j - 1))

    return dfs(0, len(s) - 1)
```

Time Complexity: $O(2^n)$ | Space Complexity: $O(n)$
Idea: Recursively compare ends of string.

Solution 2: Memoization

```
def longestPalindromeSubseq(self, s: str) -> int:
    memo = {}

    def dfs(i, j):
        if i > j:
            return 0
        if i == j:
            return 1

        if (i, j) in memo:
            return memo[(i, j)]

        if s[i] == s[j]:
            memo[(i, j)] = 2 + dfs(i + 1, j - 1)
        else:
            memo[(i, j)] = max(dfs(i + 1, j), dfs(i, j - 1))

        return memo[(i, j)]

    return dfs(0, len(s) - 1)
```

Time Complexity: $O(n^2)$ | Space Complexity: $O(n^2)$
Idea: Cache results of subproblems.

Solution 3: 2D Dynamic Programming

```
def longestPalindromeSubseq(self, s: str) -> int:
    n = len(s)
    dp = [[0] * n for _ in range(n)]

    # Base case: single characters
    for i in range(n):
        dp[i][i] = 1

    # Build table for increasing lengths
    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1

            if s[i] == s[j]:
                dp[i][j] = 2 + (dp[i+1][j-1] if length > 2 else 0)
            else:
                dp[i][j] = max(dp[i+1][j], dp[i][j-1])

    return dp[0][n-1]
```

Time Complexity: $O(n^2)$ | Space Complexity: $O(n^2)$
Idea: dp[i][j] = LPS length for s[i:j+1].

Solution 4: Space Optimized DP

```
def longestPalindromeSubseq(self, s: str) -> int:
    n = len(s)
    dp = [0] * n

    for i in range(n-1, -1, -1):
        dp[i] = 1  # length 1 palindrome
        prev = 0

        for j in range(i+1, n):
            temp = dp[j]

            if s[i] == s[j]:
                dp[j] = 2 + prev
            else:
                dp[j] = max(dp[j], dp[j-1])

            prev = temp

    return dp[n-1]
```

Time Complexity: $O(n^2)$ | Space Complexity: $O(n)$
Idea: Use single array with careful state management.

221. Maximal Square

Given an m x n binary matrix filled with 0's and 1's, *find the largest square containing only* 1's *and return its area.*

Example 1:

1	0	1	0	0
1	0	1	1	1
1	1	1	1	1
1	0	0	1	0

Input: matrix = [["1","0","1","0","0"],["1","0","1","1","1"],["1","1","1","1","1"],["1","0","0","1","0"]]
Output: 4

Example 2:

0	1
1	0

Input: matrix = [["0","1"],["1","0"]]
Output: 1

Example 3:

Input: matrix = [["0"]]
Output: 0

Solution 1: Brute Force

```
def maximalSquare(self, matrix: List[List[str]]) -> int:
    if not matrix:
        return 0

    m, n = len(matrix), len(matrix[0])
    max_side = 0

    for i in range(m):
        for j in range(n):
            if matrix[i][j] == '1':
                # Try expanding square
                side = 1
                flag = True

                while i + side < m and j + side < n and flag:
                    for k in range(j, j + side + 1):
                        if matrix[i + side][k] == '0':
                            flag = False
                            break

                    for k in range(i, i + side):
                        if matrix[k][j + side] == '0':
                            flag = False
                            break

                    if flag:
                        side += 1

                max_side = max(max_side, side)

    return max_side * max_side
```

Time Complexity: $O(m^2n^2)$ | Space Complexity: $O(1)$
Idea: For each 1, try to expand square.

Solution 2: Dynamic Programming

```
def maximalSquare(self, matrix: List[List[str]]) -> int:
    if not matrix:
        return 0

    m, n = len(matrix), len(matrix[0])
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    max_side = 0

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if matrix[i-1][j-1] == '1':
                dp[i][j] = min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) + 1
                max_side = max(max_side, dp[i][j])

    return max_side * max_side
```

Time Complexity: $O(m\times n)$ | Space Complexity: $O(m\times n)$
Idea: dp[i][j] = side length of largest square ending at (i-1,j-1).

Solution 3: Space Optimized DP

```
def maximalSquare(self, matrix: List[List[str]]) -> int:
    if not matrix:
        return 0

    m, n = len(matrix), len(matrix[0])
    dp = [0] * (n + 1)
    max_side = 0
    prev = 0

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            temp = dp[j]

            if matrix[i-1][j-1] == '1':
                dp[j] = min(prev, dp[j-1], dp[j]) + 1
                max_side = max(max_side, dp[j])
            else:
                dp[j] = 0

            prev = temp

    return max_side * max_side
```

Time Complexity: $O(m\times n)$ | Space Complexity: $O(n)$
Idea: Use single array with prev variable for diagonal.

97. Interleaving String

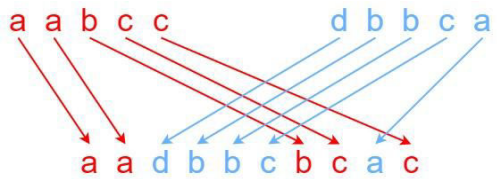
Given strings s1, s2, and s3, find whether s3 is formed by an **interleaving** of s1 and s2.

An **interleaving** of two strings s and t is a configuration where s and t are divided into n and m substrings respectively, such that:

- $s = s_1 + s_2 + \ldots + s_n$
- $t = t_1 + t_2 + \ldots + t_m$
- $|n - m| <= 1$
- The **interleaving** is $s_1 + t_1 + s_2 + t_2 + s_3 + t_3 + \ldots$ or $t_1 + s_1 + t_2 + s_2 + t_3 + s_3 + \ldots$

Note: a + b is the concatenation of strings a and b.

Example 1:



Input: s1 = "aabcc", s2 = "dbbca", s3 = "aadbbcbcac"
Output: true
Explanation: One way to obtain s3 is:
Split s1 into s1 = "aa" + "bc" + "c", and s2 into s2 = "dbbc" + "a".
Interleaving the two splits, we get "aa" + "dbbc" + "bc" + "a" + "c" = "aadbbcbcac".
Since s3 can be obtained by interleaving s1 and s2, we return true.

Example 2:

Input: s1 = "aabcc", s2 = "dbbca", s3 = "aadbbbaccc"
Output: false
Explanation: Notice how it is impossible to interleave s2 with any other string to obtain s3.

Example 3:

Input: s1 = "", s2 = "", s3 = ""
Output: true

Solution 1: Recursive

```
def isInterleave(self, s1: str, s2: str, s3: str) -> bool:
    if len(s1) + len(s2) != len(s3):
        return False

    def dfs(i, j):
        if i == len(s1) and j == len(s2):
            return True

        if i < len(s1) and s1[i] == s3[i+j] and dfs(i+1, j):
            return True

        if j < len(s2) and s2[j] == s3[i+j] and dfs(i, j+1):
            return True

        return False

    return dfs(0, 0)
```

Time Complexity: O(2^(m+n)) | Space Complexity: O(m+n)
Idea: Try taking from either string at each step.

Solution 2: Memoization

```
def isInterleave(self, s1: str, s2: str, s3: str) -> bool:
    if len(s1) + len(s2) != len(s3):
        return False

    memo = {}

    def dfs(i, j):
        if i == len(s1) and j == len(s2):
            return True

        if (i, j) in memo:
            return memo[(i, j)]

        if i < len(s1) and s1[i] == s3[i+j] and dfs(i+1, j):
            memo[(i, j)] = True
            return True

        if j < len(s2) and s2[j] == s3[i+j] and dfs(i, j+1):
            memo[(i, j)] = True
            return True

        memo[(i, j)] = False
        return False

    return dfs(0, 0)
```

Time Complexity: O(m×n) | Space Complexity: O(m×n)
Idea: Cache results of (i,j) pairs.

Solution 3: 2D Dynamic Programming

```
def isInterleave(self, s1: str, s2: str, s3: str) -> bool:
    m, n = len(s1), len(s2)

    if m + n != len(s3):
        return False

    dp = [[False] * (n + 1) for _ in range(m + 1)]
    dp[0][0] = True

    # First row - using only s2
    for j in range(1, n + 1):
        dp[0][j] = dp[0][j-1] and s2[j-1] == s3[j-1]

    # First column - using only s1
    for i in range(1, m + 1):
        dp[i][0] = dp[i-1][0] and s1[i-1] == s3[i-1]

    # Fill DP table
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            k = i + j - 1
            dp[i][j] = (dp[i-1][j] and s1[i-1] == s3[k]) or \
                       (dp[i][j-1] and s2[j-1] == s3[k])

    return dp[m][n]
```

Time Complexity: O(m×n) | Space Complexity: O(m×n)
Idea: dp[i][j] = True if s3[0:i+j] is interleaving of s1[0:i] and s2[0:j].

Solution 4: 1D Dynamic Programming

```
def isInterleave(self, s1: str, s2: str, s3: str) -> bool:
    m, n = len(s1), len(s2)

    if m + n != len(s3):
        return False

    dp = [False] * (n + 1)

    for i in range(m + 1):
        for j in range(n + 1):
            if i == 0 and j == 0:
                dp[j] = True
            elif i == 0:
                dp[j] = dp[j-1] and s2[j-1] == s3[i+j-1]
            elif j == 0:
                dp[j] = dp[j] and s1[i-1] == s3[i+j-1]
            else:
                dp[j] = (dp[j] and s1[i-1] == s3[i+j-1]) or \
                        (dp[j-1] and s2[j-1] == s3[i+j-1])

    return dp[n]
```

Time Complexity: O(m×n) | Space Complexity: O(n)
Idea: Use single array, updating in place.

312. Burst Balloons

You are given n balloons, indexed from 0 to n - 1. Each balloon is painted with a number on it represented by an array nums. You are asked to burst all the balloons.

If you burst the i_{th} balloon, you will get nums[i - 1] * nums[i] * nums[i + 1] coins. If i - 1 or i + 1 goes out of bounds of the array, then treat it as if there is a balloon with a 1 painted on it.

Return *the maximum coins you can collect by bursting the balloons wisely.*

Example 1:

Input: nums = [3,1,5,8]
Output: 167
Explanation:
nums = [3,1,5,8] --> [3,5,8] --> [3,8] --> [8] --> []
coins = 3*1*5 + 3*5*8 + 1*3*8 + 1*8*1 = 167
Example 2:

Input: nums = [1,5]
Output: 10

Solution 1: Recursive with Memoization

```
def maxCoins(self, nums: List[int]) -> int:
    # Add 1 at both ends for boundary
    nums = [1] + nums + [1]
    n = len(nums)
    memo = {}

    def dfs(left, right):
        if left + 1 == right:
            return 0

        if (left, right) in memo:
            return memo[(left, right)]

        max_coins = 0
        for i in range(left + 1, right):
            # Burst i last in this range
            coins = nums[left] * nums[i] * nums[right]
            coins += dfs(left, i) + dfs(i, right)
            max_coins = max(max_coins, coins)

        memo[(left, right)] = max_coins
        return max_coins

    return dfs(0, n-1)
```

Time Complexity: $O(n^3)$ | Space Complexity: $O(n^2)$
Idea: Divide and conquer - burst last balloon in range.

Solution 2: Dynamic Programming

```
def maxCoins(self, nums: List[int]) -> int:
    nums = [1] + nums + [1]
    n = len(nums)
    dp = [[0] * n for _ in range(n)]

    for length in range(2, n):
        for left in range(n - length):
            right = left + length
            for i in range(left + 1, right):
                dp[left][right] = max(
                    dp[left][right],
                    nums[left] * nums[i] * nums[right] +
                    dp[left][i] + dp[i][right]
                )

    return dp[0][n-1]
```

Time Complexity: $O(n^3)$ | Space Complexity: $O(n^2)$
Idea: Build solution for increasing range lengths.

---

115. Distinct Subsequences

Given two strings s and t, return *the number of distinct* ***subsequences*** *of* s *which equals* t.

The test cases are generated so that the answer fits on a 32-bit signed integer.

Example 1:

Input: s = "rabbbit", t = "rabbit"
Output: 3
Explanation:
As shown below, there are 3 ways you can generate "rabbit" from s.
rabbbit
rabbbit
rabbbit

Example 2:

Input: s = "babgbag", t = "bag"
Output: 5
Explanation:
As shown below, there are 5 ways you can generate "bag" from s.
bab**g**bag
babgba**g**
babgba**g**
ba**b**gba**g**
babg**bag**

Solution 1: Recursive

```
def numDistinct(self, s: str, t: str) -> int:
    def dfs(i, j):
        if j == len(t):
            return 1
        if i == len(s):
            return 0

        if s[i] == t[j]:
            return dfs(i + 1, j + 1) + dfs(i + 1, j)
        else:
            return dfs(i + 1, j)

    return dfs(0, 0)
```

Time Complexity: O(2^n) | Space Complexity: O(n)
Idea: For matching chars, either use it or skip it.

Solution 2: Memoization

```
def numDistinct(self, s: str, t: str) -> int:
    memo = {}

    def dfs(i, j):
        if j == len(t):
            return 1
        if i == len(s):
            return 0

        if (i, j) in memo:
            return memo[(i, j)]

        if s[i] == t[j]:
            memo[(i, j)] = dfs(i + 1, j + 1) + dfs(i + 1, j)
        else:
            memo[(i, j)] = dfs(i + 1, j)

        return memo[(i, j)]

    return dfs(0, 0)
```

Time Complexity: O(m×n) | Space Complexity: O(m×n)
Idea: Cache results of (i,j) pairs.

Solution 3: 2D Dynamic Programming

```
def numDistinct(self, s: str, t: str) -> int:
    m, n = len(s), len(t)
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    # Empty string is subsequence of any string
    for i in range(m + 1):
        dp[i][0] = 1

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            dp[i][j] = dp[i-1][j]  # don't use s[i-1]
            if s[i-1] == t[j-1]:
                dp[i][j] += dp[i-1][j-1]  # use s[i-1]

    return dp[m][n]
```

Time Complexity: O(m×n) | Space Complexity: O(m×n)
Idea: dp[i][j] = distinct subsequences of s[:i] that equal t[:j].

Solution 4: 1D Dynamic Programming

```
def numDistinct(self, s: str, t: str) -> int:
    m, n = len(s), len(t)
    dp = [0] * (n + 1)
    dp[0] = 1

    for i in range(1, m + 1):
        prev = 1  # dp[i-1][0]
        for j in range(1, n + 1):
            temp = dp[j]  # store dp[i-1][j]

            if s[i-1] == t[j-1]:
                dp[j] += prev  # add dp[i-1][j-1]

            prev = temp

    return dp[n]
```

Time Complexity: O(m×n) | Space Complexity: O(n)
Idea: Update from right to left to avoid overwriting.

10. Regular Expression Matching

Given an input string s and a pattern p, implement regular expression matching with support for '.' and '*' where:

- '.' Matches any single character.
- '*' Matches zero or more of the preceding element.

The matching should cover the **entire** input string (not partial).

Example 1:

Input: s = "aa", p = "a"
Output: false
Explanation: "a" does not match the entire string "aa".

Example 2:

Input: s = "aa", p = "a*"
Output: true
Explanation: '*' means zero or more of the preceding element, 'a'. Therefore, by repeating 'a' once, it becomes "aa".

Example 3:

Input: s = "ab", p = ".*"
Output: true
Explanation: ".*" means "zero or more (*) of any character (.)".

Solution 1: Recursive

```
def isMatch(self, s: str, p: str) -> bool:
    if not p:
        return not s

    first_match = bool(s) and (p[0] == s[0] or p[0] == '.')

    if len(p) >= 2 and p[1] == '*':
        # zero occurrences of preceding char OR one/more occurrences
        return (self.isMatch(s, p[2:]) or
                (first_match and self.isMatch(s[1:], p)))
    else:
        return first_match and self.isMatch(s[1:], p[1:])
```

Time Complexity: O(2^(m+n)) | Space Complexity: O(m+n)
Idea: Handle '*' by either skipping or consuming one char.

Solution 2: Memoization

```
def isMatch(self, s: str, p: str) -> bool:
    memo = {}

    def dfs(i, j):
        if (i, j) in memo:
            return memo[(i, j)]

        if j == len(p):
            return i == len(s)

        first_match = i < len(s) and (p[j] == s[i] or p[j] == '.')

        if j + 1 < len(p) and p[j+1] == '*':
            memo[(i, j)] = (dfs(i, j+2) or
                            (first_match and dfs(i+1, j)))
        else:
            memo[(i, j)] = first_match and dfs(i+1, j+1)

        return memo[(i, j)]

    return dfs(0, 0)
```

Time Complexity: O(m×n) | Space Complexity: O(m×n)
Idea: Cache results of (i,j) pairs.

Solution 3: 2D Dynamic Programming

```
def isMatch(self, s: str, p: str) -> bool:
    m, n = len(s), len(p)
    dp = [[False] * (n + 1) for _ in range(m + 1)]
    dp[0][0] = True

    # Handle patterns like a*, a*b*, a*b*c* that can match
empty string
    for j in range(2, n + 1):
        if p[j-1] == '*':
            dp[0][j] = dp[0][j-2]

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if p[j-1] == '*':
                # zero occurrences or one/more occurrences
                dp[i][j] = dp[i][j-2]
                if p[j-2] == '.' or p[j-2] == s[i-1]:
                    dp[i][j] = dp[i][j] or dp[i-1][j]
            else:
                if p[j-1] == '.' or p[j-1] == s[i-1]:
                    dp[i][j] = dp[i-1][j-1]

    return dp[m][n]
```

Time Complexity: O(m×n) | Space Complexity: O(m×n)
Idea: Bottom-up DP handling all cases.

Solution 4: 1D Dynamic Programming

```
def isMatch(self, s: str, p: str) -> bool:
    m, n = len(s), len(p)
    dp = [False] * (n + 1)
    dp[0] = True

    # Initialize for empty string
    for j in range(2, n + 1):
        if p[j-1] == '*':
            dp[j] = dp[j-2]

    for i in range(1, m + 1):
        prev = [False] * (n + 1)
        prev[0] = False

        for j in range(1, n + 1):
            if p[j-1] == '*':
                prev[j] = prev[j-2]
                if p[j-2] == '.' or p[j-2] == s[i-1]:
                    prev[j] = prev[j] or dp[j]
            else:
                if p[j-1] == '.' or p[j-1] == s[i-1]:
                    prev[j] = dp[j-1]

        dp = prev

    return dp[n]
```

Time Complexity: O(m×n) | Space Complexity: O(n)
Idea: Use two arrays to store previous and current row.

TRIE (Prefix Tree)

208. Implement Trie (Prefix Tree)

A **trie** (pronounced as "try") or **prefix tree** is a tree data structure used to efficiently store and retrieve keys in a dataset of strings. There are various applications of this data structure, such as autocomplete and spellchecker.

Implement the Trie class:

- Trie() Initializes the trie object.
- void insert(String word) Inserts the string word into the trie.
- boolean search(String word) Returns true if the string word is in the trie (i.e., was inserted before), and false otherwise.
- boolean startsWith(String prefix) Returns true if there is a previously inserted string word that has the prefix prefix, and false otherwise.

Example 1:

Input
["Trie", "insert", "search", "search", "startsWith", "insert", "search"]
[[], ["apple"], ["apple"], ["app"], ["app"], ["app"], ["app"]]
Output
[null, null, true, false, true, null, true]

Explanation
Trie trie = new Trie();
trie.insert("apple");
trie.search("apple"); // return True
trie.search("app"); // return False
trie.startsWith("app"); // return True
trie.insert("app");
trie.search("app"); // return True

Solution 1: Array-based Trie (Lowercase Letters)

```
class TrieNode:
    def __init__(self):
        self.children = [None] * 26
        self.is_end = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def _char_to_index(self, ch):
        return ord(ch) - ord('a')

    def insert(self, word: str) -> None:
        node = self.root
        for ch in word:
            idx = self._char_to_index(ch)
            if not node.children[idx]:
                node.children[idx] = TrieNode()
            node = node.children[idx]
        node.is_end = True

    def search(self, word: str) -> bool:
        node = self.root
        for ch in word:
            idx = self._char_to_index(ch)
            if not node.children[idx]:
                return False
            node = node.children[idx]
        return node.is_end

    def startsWith(self, prefix: str) -> bool:
        node = self.root
        for ch in prefix:
            idx = self._char_to_index(ch)
            if not node.children[idx]:
                return False
            node = node.children[idx]
        return True
```

Time Complexity: O(L) for all operations | Space Complexity: O(N * L) where N is number of words
Idea: Use array of size 26 for children (efficient for lowercase letters).

Solution 2: HashMap-based Trie (Any Characters)

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word: str) -> None:
        node = self.root
        for ch in word:
            if ch not in node.children:
                node.children[ch] = TrieNode()
            node = node.children[ch]
        node.is_end = True

    def search(self, word: str) -> bool:
        node = self.root
        for ch in word:
            if ch not in node.children:
                return False
            node = node.children[ch]
        return node.is_end

    def startsWith(self, prefix: str) -> bool:
        node = self.root
        for ch in prefix:
            if ch not in node.children:
                return False
            node = node.children[ch]
        return True
```

Time Complexity: O(L) for all operations | Space Complexity: O(N * L)
Idea: Use hashmap for flexible character support.

Solution 3: Recursive Implementation

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word: str) -> None:
        self._insert_recursive(self.root, word, 0)

    def _insert_recursive(self, node, word, idx):
        if idx == len(word):
            node.is_end = True
            return

        ch = word[idx]
        if ch not in node.children:
            node.children[ch] = TrieNode()
        self._insert_recursive(node.children[ch], word, idx + 1)

    def search(self, word: str) -> bool:
        return self._search_recursive(self.root, word, 0)

    def _search_recursive(self, node, word, idx):
        if idx == len(word):
            return node.is_end

        ch = word[idx]
        if ch not in node.children:
            return False
        return self._search_recursive(node.children[ch], word, idx + 1)

    def startsWith(self, prefix: str) -> bool:
        return self._starts_recursive(self.root, prefix, 0)

    def _starts_recursive(self, node, prefix, idx):
        if idx == len(prefix):
            return True

        ch = prefix[idx]
        if ch not in node.children:
            return False
        return self._starts_recursive(node.children[ch], prefix, idx + 1)
```

Time Complexity: O(L) | Space Complexity: O(L) for recursion stack
Idea: Use recursion instead of iteration.

Solution 4: Defaultdict-based Trie

```
class TrieNode:
    def __init__(self):
        self.children = defaultdict(TrieNode)
        self.is_end = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word: str) -> None:
        node = self.root
        for ch in word:
            node = node.children[ch]
        node.is_end = True

    def search(self, word: str) -> bool:
        node = self.root
        for ch in word:
            if ch not in node.children:
                return False
            node = node.children[ch]
        return node.is_end

    def startsWith(self, prefix: str) -> bool:
        node = self.root
        for ch in prefix:
            if ch not in node.children:
                return False
            node = node.children[ch]
        return True
```

Time Complexity: O(L) | Space Complexity: O(N * L)
Idea: Use defaultdict for automatic node creation.

211. Design Add and Search Words Data Structure

Design a data structure that supports adding new words and finding if a string matches any previously added string.

Implement the WordDictionary class:

- WordDictionary() Initializes the object.
- void addWord(word) Adds word to the data structure, it can be matched later.
- bool search(word) Returns true if there is any string in the data structure that matches word or false otherwise. word may contain dots '.' where dots can be matched with any letter.

Example:

Input
["WordDictionary","addWord","addWord","addWord","search","search","search","search"]
[[],["bad"],["dad"],["mad"],["pad"],["bad"],[".ad"],["b.."]]
Output
[null,null,null,null,false,true,true,true]

Explanation
WordDictionary wordDictionary = new WordDictionary();
wordDictionary.addWord("bad");
wordDictionary.addWord("dad");
wordDictionary.addWord("mad");
wordDictionary.search("pad"); // return False
wordDictionary.search("bad"); // return True
wordDictionary.search(".ad"); // return True
wordDictionary.search("b.."); // return True

Solution 1: Standard Trie with DFS Search

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end = False

class WordDictionary:
    def __init__(self):
        self.root = TrieNode()

    def addWord(self, word: str) -> None:
        node = self.root
        for ch in word:
            if ch not in node.children:
                node.children[ch] = TrieNode()
            node = node.children[ch]
        node.is_end = True
```

```python
def search(self, word: str) -> bool:
    def dfs(node, idx):
        if idx == len(word):
            return node.is_end

        ch = word[idx]
        if ch == '.':
            for child in node.children.values():
                if dfs(child, idx + 1):
                    return True
            return False
        else:
            if ch not in node.children:
                return False
            return dfs(node.children[ch], idx + 1)

    return dfs(self.root, 0)
```

Time Complexity:
- addWord: O(L)
- search: O(26^M) where M is number of dots (worst case)
Space Complexity: O(N * L)
Idea: DFS to handle '.' wildcard.

Solution 2: BFS Approach for Search

```python
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end = False

class WordDictionary:
    def __init__(self):
        self.root = TrieNode()

    def addWord(self, word: str) -> None:
        node = self.root
        for ch in word:
            if ch not in node.children:
                node.children[ch] = TrieNode()
            node = node.children[ch]
        node.is_end = True

    def search(self, word: str) -> bool:
        queue = deque([(self.root, 0)])

        while queue:
            node, idx = queue.popleft()

            if idx == len(word):
                if node.is_end:
                    return True
                continue

            ch = word[idx]
            if ch == '.':
                for child in node.children.values():
                    queue.append((child, idx + 1))
            else:
                if ch in node.children:
                    queue.append((node.children[ch], idx + 1))

        return False
```

Time Complexity: O(26^M) | Space Complexity: O(26^M)
Idea: Level-order traversal for dot matching.

Solution 3: Optimized with Word Length Buckets

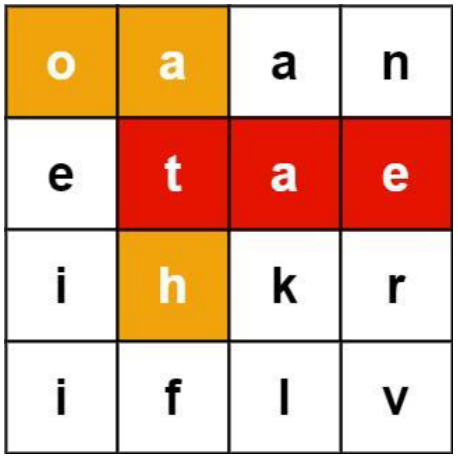
```python
class WordDictionary:
    def __init__(self):
        self.words = defaultdict(list)

    def addWord(self, word: str) -> None:
        self.words[len(word)].append(word)

    def search(self, word: str) -> bool:
        n = len(word)

        for w in self.words[n]:
            match = True
            for i in range(n):
                if word[i] != '.' and word[i] != w[i]:
                    match = False
                    break
            if match:
                return True
        return False
```

Time Complexity:
- addWord: O(1)
- search: O(N * L) where N is number of words with same length
Space Complexity: O(N * L)
Idea: Bucket words by length for faster candidate filtering.

212. Word Search II

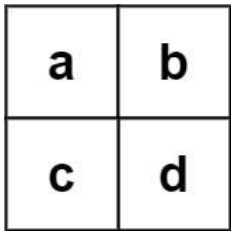
Given an m x n board of characters and a list of strings words, return *all words on the board*.
Each word must be constructed from letters of sequentially adjacent cells, where **adjacent cells** are horizontally or vertically neighboring. The same letter cell may not be used more than once in a word.

Example 1:



Input: board = [["o","a","a","n"],["e","t","a","e"],["i","h","k","r"],["i","f","l","v"]], words = ["oath","pea","eat","rain"]
Output: ["eat","oath"]

Example 2:



Input: board = [["a","b"],["c","d"]], words = ["abcb"]
Output: []

Solution 1: Trie + Backtracking

```python
class TrieNode:
    def __init__(self):
        self.children = {}
        self.word = None  # Store word at end node

class Solution:
    def findWords(self, board: List[List[str]], words: List[str]) -> List[str]:
        # Build Trie
        root = TrieNode()
        for word in words:
            node = root
            for ch in word:
                if ch not in node.children:
                    node.children[ch] = TrieNode()
                node = node.children[ch]
            node.word = word

        result = []
        m, n = len(board), len(board[0])

        def dfs(i, j, node):
            if i < 0 or i >= m or j < 0 or j >= n or board[i][j] == '#':
                return

            ch = board[i][j]
            if ch not in node.children:
                return

            next_node = node.children[ch]
            if next_node.word:
                result.append(next_node.word)
                next_node.word = None  # Avoid duplicates

            # Mark as visited
            board[i][j] = '#'

            # Explore neighbors
            dfs(i+1, j, next_node)
            dfs(i-1, j, next_node)
            dfs(i, j+1, next_node)
            dfs(i, j-1, next_node)

            # Unmark
            board[i][j] = ch

        for i in range(m):
            for j in range(n):
                dfs(i, j, root)

        return result
```

Time Complexity: O(M * N * 4^L) where L is max word length | Space Complexity: O(total characters)
Idea: Use Trie to prune search paths.

Solution 2: Optimized with Visited Set

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.word = None

class Solution:
    def findWords(self, board: List[List[str]], words: List[str]) -> List[str]:
        root = TrieNode()
        for word in words:
            node = root
            for ch in word:
                if ch not in node.children:
                    node.children[ch] = TrieNode()
                node = node.children[ch]
            node.word = word

        result = []
        m, n = len(board), len(board[0])
        visited = [[False] * n for _ in range(m)]

        def dfs(i, j, node):
            if visited[i][j]:
                return

            ch = board[i][j]
            if ch not in node.children:
                return

            node = node.children[ch]
            if node.word:
                result.append(node.word)
                node.word = None

            visited[i][j] = True

            for di, dj in [(0,1), (1,0), (0,-1), (-1,0)]:
                ni, nj = i + di, j + dj
                if 0 <= ni < m and 0 <= nj < n:
                    dfs(ni, nj, node)

            visited[i][j] = False

        for i in range(m):
            for j in range(n):
                dfs(i, j, root)

        return result
```

Time Complexity: O(M * N * 4^L) | Space Complexity: O(M*N + total chars)
Idea: Use visited matrix instead of modifying board.

336. Palindrome Pairs

You are given a **0-indexed** array of **unique** strings words.

A **palindrome pair** is a pair of integers (i, j) such that:

- 0 <= i, j < words.length,
- i != j, and
- words[i] + words[j] (the concatenation of the two strings) is a palindrome.

Return *an array of all the* ***palindrome pairs*** *of* words.

You must write an algorithm with O(sum of words[i].length) runtime complexity.

Example 1:

Input: words = ["abcd","dcba","lls","s","sssll"]
Output: [[0,1],[1,0],[3,2],[2,4]]
Explanation: The palindromes are ["abcddcba","dcbaabcd","slls","llssssll"]

Example 2:

Input: words = ["bat","tab","cat"]
Output: [[0,1],[1,0]]
Explanation: The palindromes are ["battab","tabbat"]

Example 3:

Input: words = ["a",""]
Output: [[0,1],[1,0]]
Explanation: The palindromes are ["a","a"]

Solution 1: Brute Force

```
def palindromePairs(self, words: List[str]) -> List[List[int]]:
    def is_palindrome(s):
        return s == s[::-1]

    result = []
    n = len(words)

    for i in range(n):
        for j in range(n):
            if i != j and is_palindrome(words[i] + words[j]):
                result.append([i, j])

    return result
```

Time Complexity: O(n² * L) | Space Complexity: O(1)
Idea: Check all pairs.

Solution 2: HashMap + Prefix/Suffix

```
def palindromePairs(self, words: List[str]) -> List[List[int]]:
    def is_palindrome(s):
        return s == s[::-1]

    word_map = {word: i for i, word in enumerate(words)}
    result = []

    for i, word in enumerate(words):
        # Case 1: Empty string - pairs with any palindrome
        if word == "":
            for j, w in enumerate(words):
                if i != j and is_palindrome(w):
                    result.append([i, j])
                    result.append([j, i])
            continue

        # Case 2: Reverse of word exists
        reverse = word[::-1]
        if reverse in word_map and word_map[reverse] != i:
            result.append([i, word_map[reverse]])

        # Case 3: Word can be split into prefix and suffix
        for k in range(1, len(word)):
            prefix = word[:k]
            suffix = word[k:]

            # If prefix is palindrome and reverse of suffix exists
            if is_palindrome(prefix):
                rev_suffix = suffix[::-1]
                if rev_suffix in word_map:
                    result.append([word_map[rev_suffix], i])

            # If suffix is palindrome and reverse of prefix exists
            if is_palindrome(suffix):
                rev_prefix = prefix[::-1]
                if rev_prefix in word_map:
                    result.append([i, word_map[rev_prefix]])

    return result
```

Time Complexity: O(n * L²) | Space Complexity: O(n)
Idea: Check palindrome conditions using hashmap.

Solution 3: Trie-based Approach

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.index = -1
        self.palindrome_indices = []  # indices that form palindrome from this node

class Solution:
    def palindromePairs(self, words: List[str]) -> List[List[int]]:
        def is_palindrome(s, start, end):
            while start < end:
                if s[start] != s[end]:
                    return False
                start += 1
                end -= 1
            return True

        # Build Trie with reversed words
        root = TrieNode()

        for i, word in enumerate(words):
            node = root
            rev_word = word[::-1]

            for j, ch in enumerate(rev_word):
                # Check if remaining part is palindrome
                if is_palindrome(rev_word, j, len(rev_word)-1):
                    node.palindrome_indices.append(i)

                if ch not in node.children:
                    node.children[ch] = TrieNode()
                node = node.children[ch]

            node.index = i

        result = []

        # Search for pairs
        for i, word in enumerate(words):
            node = root

            for j, ch in enumerate(word):
                # Case: Current word longer, check if node has word and remaining is palindrome
                if node.index != -1 and node.index != i and is_palindrome(word, j, len(word)-1):
                    result.append([i, node.index])

                if ch not in node.children:
                    break
                node = node.children[ch]
            else:
                # Case: Reached end of current word
                if node.index != -1 and node.index != i:
```

```
        result.append([i, node.index])

        # Case: Words in Trie longer, check palindrome indices
        for idx in node.palindrome_indices:
            if idx != i:
                result.append([i, idx])

    return result
```

Time Complexity: O(n * L²) | Space Complexity: O(n * L)
Idea: Use Trie to store reversed words and check palindrome conditions.

648. Replace Words

In English, we have a concept called **root**, which can be followed by some other word to form another longer word - let's call this word **derivative**. For example, when the **root** "help" is followed by the word "ful", we can form a derivative "helpful".

Given a dictionary consisting of many **roots** and a sentence consisting of words separated by spaces, replace all the derivatives in the sentence with the **root** forming it. If a derivative can be replaced by more than one **root**, replace it with the **root** that has **the shortest length**.

Return *the sentence* after the replacement.

Example 1:

Input: dictionary = ["cat","bat","rat"], sentence = "the cattle was rattled by the battery"
Output: "the cat was rat by the bat"

Example 2:

Input: dictionary = ["a","b","c"], sentence = "aadsfasf absbs bbab cadsfafs"
Output: "a a b c"

Solution 1: HashSet Approach

```
def replaceWords(self, dictionary: List[str], sentence: str) -> str:
    roots = set(dictionary)
    words = sentence.split()

    for i, word in enumerate(words):
        for j in range(1, len(word) + 1):
            if word[:j] in roots:
                words[i] = word[:j]
                break

    return " ".join(words)
```

Time Complexity: O(n * L²) | Space Complexity: O(n)
Idea: Check all prefixes for each word.

Solution 2: Trie Approach

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_root = False

class Solution:
    def replaceWords(self, dictionary: List[str], sentence:
str) -> str:
        # Build Trie
        root = TrieNode()
        for word in dictionary:
            node = root
            for ch in word:
                if ch not in node.children:
                    node.children[ch] = TrieNode()
                node = node.children[ch]
            node.is_root = True

        def replace(word):
            node = root
            for i, ch in enumerate(word):
                if ch not in node.children:
                    return word
                node = node.children[ch]
                if node.is_root:
                    return word[:i+1]
            return word

        words = sentence.split()
        return " ".join(replace(word) for word in words)
```

Time Complexity: O(n * L) | Space Complexity: O(total chars in dictionary)
Idea: Use Trie for efficient prefix matching.

677. Map Sum Pairs

Design a map that allows you to do the following:

- Maps a string key to a given value.
- Returns the sum of the values that have a key with a prefix equal to a given string.

Implement the MapSum class:

- MapSum() Initializes the MapSum object.
- void insert(String key, int val) Inserts the key-val pair into the map. If the key already existed, the original key-value pair will be overridden to the new one.
- int sum(string prefix) Returns the sum of all the pairs' value whose key starts with the prefix.

Example 1:

Input
["MapSum", "insert", "sum", "insert", "sum"]
[[], ["apple", 3], ["ap"], ["app", 2], ["ap"]]
Output
[null, null, 3, null, 5]

Explanation
MapSum mapSum = new MapSum();
mapSum.insert("apple", 3);
mapSum.sum("ap"); // return 3 (apple = 3)
mapSum.insert("app", 2);
mapSum.sum("ap"); // return 5 (apple + app = 3 + 2 = 5)

Solution 1: Brute Force

```
class MapSum:
    def __init__(self):
        self.map = {}

    def insert(self, key: str, val: int) -> None:
        self.map[key] = val

    def sum(self, prefix: str) -> int:
        total = 0
        for key, val in self.map.items():
            if key.startswith(prefix):
                total += val
        return total
```

Time Complexity:
- insert: O(1)
- sum: O(n * L)
Space Complexity: O(n)
Idea: Store in hashmap, compute sum by iterating.

Solution 2: Trie with Value at Node

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.value = 0

class MapSum:
    def __init__(self):
        self.root = TrieNode()
        self.map = {}

    def insert(self, key: str, val: int) -> None:
        delta = val - self.map.get(key, 0)
        self.map[key] = val

        node = self.root
        for ch in key:
            if ch not in node.children:
                node.children[ch] = TrieNode()
            node = node.children[ch]
            node.value += delta

    def sum(self, prefix: str) -> int:
        node = self.root
        for ch in prefix:
            if ch not in node.children:
                return 0
            node = node.children[ch]
        return node.value
```

Time Complexity: O(L) for both operations | Space Complexity: O(total chars)
Idea: Store cumulative sum at each trie node.

Solution 3: Trie with End Marker

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end = False
        self.value = 0

class MapSum:
    def __init__(self):
        self.root = TrieNode()
        self.map = {}

    def insert(self, key: str, val: int) -> None:
        delta = val - self.map.get(key, 0)
        self.map[key] = val

        node = self.root
        for ch in key:
            if ch not in node.children:
                node.children[ch] = TrieNode()
            node = node.children[ch]
        node.is_end = True
        node.value += delta

    def sum(self, prefix: str) -> int:
        node = self.root
        for ch in prefix:
            if ch not in node.children:
                return 0
            node = node.children[ch]

        # DFS to sum all values under this node
        def dfs(node):
            total = node.value
            for child in node.children.values():
                total += dfs(child)
            return total

        return dfs(node)
```

Time Complexity: O(L) for insert, O(L + K) for sum where K is size of subtree | Space Complexity: O(total chars)
Idea: Store value at end nodes, sum by traversing subtree.

720. Longest Word in Dictionary

Given an array of strings words representing an English Dictionary, return *the longest word in* words *that can be built one character at a time by other words in* words.

If there is more than one possible answer, return the longest word with the smallest lexicographical order. If there is no answer, return the empty string.

Note that the word should be built from left to right with each additional character being added to the end of a previous word.

Example 1:

Input: words = ["w","wo","wor","worl","world"]
Output: "world"
Explanation: The word "world" can be built one character at a time by "w", "wo", "wor", and "worl".

Example 2:

Input: words = ["a","banana","app","appl","ap","apply","apple"]
Output: "apple"
Explanation: Both "apply" and "apple" can be built from other words in the dictionary. However, "apple" is lexicographically smaller than "apply".

Solution 1: Sorting + HashSet

```
def longestWord(self, words: List[str]) -> str:
    words.sort()
    valid = set([""])
    result = ""

    for word in words:
        if word[:-1] in valid:
            valid.add(word)
            if len(word) > len(result) or (len(word) == len(result) and word < result):
                result = word

    return result
```

Time Complexity: O(n log n + n * L) | Space Complexity: O(n)
Idea: Sort lexicographically and check prefixes.

Solution 2: Trie + DFS

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end = False
        self.word = ""

class Solution:
    def longestWord(self, words: List[str]) -> str:
        # Build Trie
        root = TrieNode()
        for word in words:
            node = root
            for ch in word:
                if ch not in node.children:
                    node.children[ch] = TrieNode()
                node = node.children[ch]
            node.is_end = True
            node.word = word

        # DFS to find longest word
        def dfs(node):
            result = node.word
            for child in node.children.values():
                if child.is_end:
                    candidate = dfs(child)
                    if len(candidate) > len(result) or (len(candidate) == len(result) and candidate < result):
                        result = candidate
            return result

        return dfs(root)
```

Time Complexity: O(total chars) | Space Complexity: O(total chars)
Idea: Buildable words are those where every prefix is end node.

1268. Search Suggestions System

You are given an array of strings products and a string searchWord.

Design a system that suggests at most three product names from products after each character of searchWord is typed. Suggested products should have common prefix with searchWord. If there are more than three products with a common prefix return the three lexicographically minimums products.

Return *a list of lists of the suggested products after each character of* searchWord *is typed*.

Example 1:

Input: products = ["mobile","mouse","moneypot","monitor","mousepad"], searchWord = "mouse"
Output: [["mobile","moneypot","monitor"],["mobile","moneypot","monitor"],["mouse","mousepad"],["mouse","mousepad"],["mouse","mousepad"]]
Explanation: products sorted lexicographically = ["mobile","moneypot","monitor","mouse","mousepad"].
After typing m and mo all products match and we show user ["mobile","moneypot","monitor"].
After typing mou, mous and mouse the system suggests ["mouse","mousepad"].

Example 2:

Input: products = ["havana"], searchWord = "havana"
Output: [["havana"],["havana"],["havana"],["havana"],["havana"],["havana"]]
Explanation: The only word "havana" will be always suggested while typing the search word.

Solution 1: Sorting + Binary Search

```
def suggestedProducts(self, products: List[str], searchWord: str) -> List[List[str]]:
    products.sort()
    result = []
    prefix = ""

    for ch in searchWord:
        prefix += ch
        # Binary search for first occurrence
        idx = bisect_left(products, prefix)
        suggestions = []
        for i in range(idx, min(idx + 3, len(products))):
            if products[i].startswith(prefix):
                suggestions.append(products[i])
        result.append(suggestions)

    return result
```

Time Complexity: O(n log n + m log n) | Space Complexity: O(1) excluding output
Idea: Sort and binary search for each prefix.

Solution 2: Two Pointers

```
def suggestedProducts(self, products: List[str], searchWord: str) -> List[List[str]]:
    products.sort()
    result = []
    left, right = 0, len(products) - 1

    for i, ch in enumerate(searchWord):
        # Filter products that match prefix
        while left <= right and (len(products[left]) <= i or products[left][i] != ch):
            left += 1
        while left <= right and (len(products[right]) <= i or products[right][i] != ch):
            right -= 1

        # Get top 3 suggestions
        suggestions = []
        for j in range(left, min(left + 3, right + 1)):
            suggestions.append(products[j])
        result.append(suggestions)

    return result
```

Time Complexity: O(n log n + n * m) | Space Complexity: O(1)
Idea: Maintain window of valid products that shrink with each character.

Solution 3: Trie with Priority Queue

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.suggestions = []

class Solution:
    def suggestedProducts(self, products: List[str], searchWord: str) -> List[List[str]]:
        # Build Trie
        root = TrieNode()
        for product in sorted(products):
            node = root
            for ch in product:
                if ch not in node.children:
                    node.children[ch] = TrieNode()
                node = node.children[ch]
                # Keep only top 3 lexicographically
                if len(node.suggestions) < 3:
                    node.suggestions.append(product)

        # Search
        result = []
        node = root
        for ch in searchWord:
            if node and ch in node.children:
                node = node.children[ch]
                result.append(node.suggestions)
            else:
                node = None
                result.append([])

        return result
```

Time Complexity: O(n log n + total chars) | Space Complexity: O(total chars)
Idea: Precompute suggestions at each trie node during insertion.

745. Prefix and Suffix Search

Design a special dictionary that searches the words in it by a prefix and a suffix.

Implement the WordFilter class:

- WordFilter(string[] words) Initializes the object with the words in the dictionary.
- f(string pref, string suff) Returns *the index of the word in the dictionary,* which has the prefix pref and the suffix suff. If there is more than one valid index, return **the largest** of them. If there is no such word in the dictionary, return -1.

Example 1:

Input
["WordFilter", "f"]
[[["apple"]], ["a", "e"]]
Output
[null, 0]
Explanation
WordFilter wordFilter = new WordFilter(["apple"]);
wordFilter.f("a", "e"); // return 0, because the word at index 0 has prefix = "a" and suffix = "e".

Solution 1: HashMap of Prefix+Suffix Combinations

```
class WordFilter:
    def __init__(self, words: List[str]):
        self.combinations = {}

        for weight, word in enumerate(words):
            for i in range(len(word) + 1):
                for j in range(len(word) + 1):
                    key = word[:i] + "#" + word[j:]
                    self.combinations[key] = weight

    def f(self, pref: str, suff: str) -> int:
        key = pref + "#" + suff
        return self.combinations.get(key, -1)
```

Time Complexity:
- init: O(n * L^3)
- f: O(1)
Space Complexity: O(n * L^2)
Idea: Store all prefix-suffix combinations.

Solution 2: Two Tries (Prefix and Suffix)

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.weights = []  # Store weights of words passing through

class WordFilter:
    def __init__(self, words: List[str]):
        self.prefix_trie = TrieNode()
        self.suffix_trie = TrieNode()

        for weight, word in enumerate(words):
            # Insert into prefix trie
            node = self.prefix_trie
            node.weights.append(weight)
            for ch in word:
                if ch not in node.children:
                    node.children[ch] = TrieNode()
                node = node.children[ch]
                node.weights.append(weight)

            # Insert into suffix trie (reversed)
            node = self.suffix_trie
            node.weights.append(weight)
            for ch in word[::-1]:
                if ch not in node.children:
                    node.children[ch] = TrieNode()
                node = node.children[ch]
                node.weights.append(weight)

    def f(self, pref: str, suff: str) -> int:
        # Get weights from prefix trie
        node = self.prefix_trie
        for ch in pref:
            if ch not in node.children:
                return -1
            node = node.children[ch]
        prefix_weights = node.weights

        # Get weights from suffix trie
        node = self.suffix_trie
        for ch in suff[::-1]:
            if ch not in node.children:
                return -1
            node = node.children[ch]
        suffix_weights = set(node.weights)

        # Find largest common weight
        for weight in reversed(prefix_weights):
            if weight in suffix_weights:
                return weight
        return -1
```

Time Complexity:
- init: O(n * L)
- f: O(n) in worst case

Space Complexity: O(n * L)
Idea: Store weights at each node for intersection.

Solution 3: Optimized with Weight Filtering

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.weight = -1  # Store max weight at node

class WordFilter:
    def __init__(self, words: List[str]):
        self.trie = TrieNode()

        for weight, word in enumerate(words):
            # Insert all combinations of prefix + # + suffix
            word = word + "#" + word
            length = len(word)

            for i in range(len(word)):
                node = self.trie
                node.weight = max(node.weight, weight)

                for j in range(i, length):
                    ch = word[j]
                    if ch not in node.children:
                        node.children[ch] = TrieNode()
                    node = node.children[ch]
                    node.weight = max(node.weight, weight)

    def f(self, pref: str, suff: str) -> int:
        node = self.trie
        key = suff + "#" + pref

        for ch in key:
            if ch not in node.children:
                return -1
            node = node.children[ch]

        return node.weight
```

Time Complexity:
- init: $O(n * L^2)$
- f: $O(L)$

Space Complexity: $O(n * L^2)$
Idea: Store combined prefix-suffix strings in single trie.

Solution 4: HashMap with Word Indices

```
class WordFilter:
    def __init__(self, words: List[str]):
        self.prefix_map = {}
        self.suffix_map = {}

        for weight, word in enumerate(words):
            # Store all prefixes
            for i in range(len(word) + 1):
                prefix = word[:i]
                if prefix not in self.prefix_map:
                    self.prefix_map[prefix] = []
                self.prefix_map[prefix].append(weight)

            # Store all suffixes
            for i in range(len(word) + 1):
                suffix = word[i:]
                if suffix not in self.suffix_map:
                    self.suffix_map[suffix] = []
                self.suffix_map[suffix].append(weight)

    def f(self, pref: str, suff: str) -> int:
        if pref not in self.prefix_map or suff not in
self.suffix_map:
            return -1

        prefix_weights = self.prefix_map[pref]
        suffix_weights = set(self.suffix_map[suff])

        # Find largest common weight
        for weight in reversed(prefix_weights):
            if weight in suffix_weights:
                return weight
        return -1
```

Time Complexity:
- init: $O(n * L^2)$
- f: $O(\min(n, L))$

Space Complexity: $O(n * L^2)$
Idea: Store all prefixes and suffixes separately and find intersection.

UNION-FIND (Disjoint Set)

547. Number of Provinces

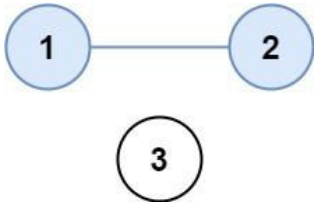
There are n cities. Some of them are connected, while some are not. If city a is connected directly with city b, and city b is connected directly with city c, then city a is connected indirectly with city c.

A **province** is a group of directly or indirectly connected cities and no other cities outside of the group.

You are given an n x n matrix isConnected where isConnected[i][j] = 1 if the i_{th} city and the j_{th} city are directly connected, and isConnected[i][j] = 0 otherwise.

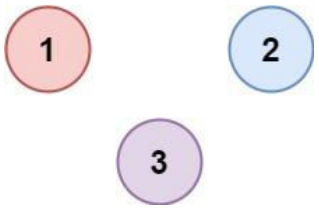
Return *the total number of* ***provinces***.

Example 1:



Input: isConnected = [[1,1,0],[1,1,0],[0,0,1]]
Output: 2

Example 2:



Input: isConnected = [[1,0,0],[0,1,0],[0,0,1]]
Output: 3

Solution 1: DFS

```
def findCircleNum(self, isConnected: List[List[int]]) -> int:
    def dfs(city):
        visited[city] = True
        for neighbor in range(n):
            if isConnected[city][neighbor] == 1 and not visited[neighbor]:
                dfs(neighbor)

    n = len(isConnected)
    visited = [False] * n
    provinces = 0

    for city in range(n):
        if not visited[city]:
            dfs(city)
            provinces += 1

    return provinces
```

Time Complexity: $O(n^2)$ | Space Complexity: $O(n)$
Idea: DFS from each unvisited city to mark entire province.

Solution 2: BFS

```
def findCircleNum(self, isConnected: List[List[int]]) -> int:
    n = len(isConnected)
    visited = [False] * n
    provinces = 0

    for city in range(n):
        if not visited[city]:
            queue = deque([city])
            visited[city] = True

            while queue:
                curr = queue.popleft()
                for neighbor in range(n):
                    if isConnected[curr][neighbor] == 1 and not
visited[neighbor]:
                        visited[neighbor] = True
                        queue.append(neighbor)
            provinces += 1

    return provinces
```

Time Complexity: $O(n^2)$ | Space Complexity: $O(n)$
Idea: BFS level-order traversal of each province.

Solution 3: Union Find (Disjoint Set)

```
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n
        self.count = n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, x, y):
        px, py = self.find(x), self.find(y)
        if px == py:
            return

        if self.rank[px] < self.rank[py]:
            self.parent[px] = py
        elif self.rank[px] > self.rank[py]:
            self.parent[py] = px
        else:
            self.parent[py] = px
            self.rank[px] += 1
        self.count -= 1

def findCircleNum(self, isConnected: List[List[int]]) -> int:
    n = len(isConnected)
    uf = UnionFind(n)

    for i in range(n):
        for j in range(i + 1, n):
            if isConnected[i][j] == 1:
                uf.union(i, j)

    return uf.count
```

Time Complexity: $O(n^2 * \alpha(n))$ | Space Complexity: $O(n)$
Idea: Union connected cities, count distinct roots.

323. Number of Connected Components

Problem Statement: Count connected components in undirected graph.

Solution 1: DFS

```
def countComponents(self, n: int, edges: List[List[int]]) -> int:
    def dfs(node):
        visited[node] = True
        for neighbor in graph[node]:
            if not visited[neighbor]:
                dfs(neighbor)

    # Build graph
    graph = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append(v)
        graph[v].append(u)

    visited = [False] * n
    components = 0

    for node in range(n):
        if not visited[node]:
            dfs(node)
            components += 1

    return components
```

Time Complexity: $O(V + E)$ | Space Complexity: $O(V + E)$
Idea: DFS to mark entire component.

Solution 2: BFS

```
def countComponents(self, n: int, edges: List[List[int]]) -> int:
    # Build graph
    graph = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append(v)
        graph[v].append(u)

    visited = [False] * n
    components = 0

    for node in range(n):
        if not visited[node]:
            queue = deque([node])
            visited[node] = True

            while queue:
                curr = queue.popleft()
                for neighbor in graph[curr]:
                    if not visited[neighbor]:
                        visited[neighbor] = True
                        queue.append(neighbor)
            components += 1

    return components
```

Time Complexity: $O(V + E)$ | Space Complexity: $O(V + E)$
Idea: BFS level-order traversal.

Solution 3: Union Find

```
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n
        self.count = n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, x, y):
        px, py = self.find(x), self.find(y)
        if px == py:
            return

        if self.rank[px] < self.rank[py]:
            self.parent[px] = py
        elif self.rank[px] > self.rank[py]:
            self.parent[py] = px
        else:
            self.parent[py] = px
            self.rank[px] += 1
        self.count -= 1

def countComponents(self, n: int, edges: List[List[int]]) -> int:
    uf = UnionFind(n)

    for u, v in edges:
        uf.union(u, v)

    return uf.count
```

Time Complexity: $O((V+E) * \alpha(V))$ | Space Complexity: $O(V)$
Idea: Union connected nodes.

261. Graph Valid Tree

Problem Statement: Check if given graph is a valid tree (connected and no cycles).

Solution 1: DFS Cycle Detection

```
def validTree(self, n: int, edges: List[List[int]]) -> bool:
    def dfs(node, parent):
        visited[node] = True
        for neighbor in graph[node]:
            if neighbor == parent:
                continue
            if visited[neighbor]:
                return False
            if not dfs(neighbor, node):
                return False
        return True

    # Build graph
    graph = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append(v)
        graph[v].append(u)

    visited = [False] * n

    # Check for cycle and connectivity
    if not dfs(0, -1):
        return False

    # Check if all nodes visited
    return all(visited)
```

Time Complexity: $O(V + E)$ | Space Complexity: $O(V + E)$
Idea: DFS with parent tracking to detect cycles.

Solution 2: BFS Cycle Detection

```
def validTree(self, n: int, edges: List[List[int]]) -> bool:
    if len(edges) != n - 1:
        return False

    # Build graph
    graph = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append(v)
        graph[v].append(u)

    visited = [False] * n
    queue = deque([0])
    visited[0] = True
    visited_count = 1

    while queue:
        node = queue.popleft()
        for neighbor in graph[node]:
```

```
        if not visited[neighbor]:
            visited[neighbor] = True
            visited_count += 1
            queue.append(neighbor)

    return visited_count == n
```

Time Complexity: O(V + E) | Space Complexity: O(V + E)
Idea: Tree must have exactly n-1 edges and be connected.

Solution 3: Union Find

```
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, x, y):
        px, py = self.find(x), self.find(y)
        if px == py:
            return False

        if self.rank[px] < self.rank[py]:
            self.parent[px] = py
        elif self.rank[px] > self.rank[py]:
            self.parent[py] = px
        else:
            self.parent[py] = px
            self.rank[px] += 1
        return True

def validTree(self, n: int, edges: List[List[int]]) -> bool:
    if len(edges) != n - 1:
        return False

    uf = UnionFind(n)

    for u, v in edges:
        if not uf.union(u, v):
            return False

    return True
```

Time Complexity: O((V+E) * α(V)) | Space Complexity: O(V)
Idea: Union will fail if cycle exists.

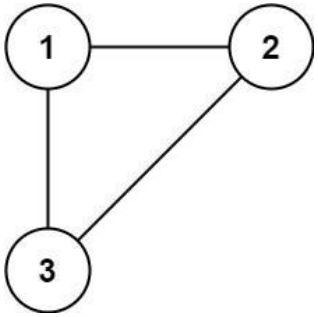
684. Redundant Connection

In this problem, a tree is an **undirected graph** that is connected and has no cycles.

You are given a graph that started as a tree with n nodes labeled from 1 to n, with one additional edge added. The added edge has two **different** vertices chosen from 1 to n, and was not an edge that already existed. The graph is represented as an array edges of length n where edges[i] = [a_i, b_i] indicates that there is an edge between nodes a_i and b_i in the graph.

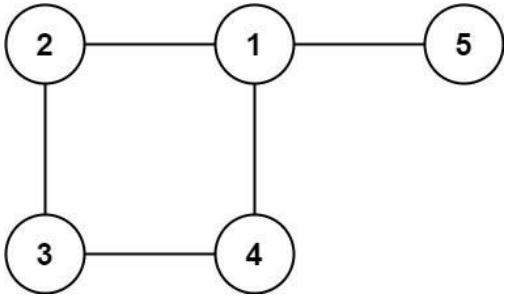
Return *an edge that can be removed so that the resulting graph is a tree of* n *nodes*. If there are multiple answers, return the answer that occurs last in the input.

Example 1:



Input: edges = [[1,2],[1,3],[2,3]]
Output: [2,3]

Example 2:



Input: edges = [[1,2],[2,3],[3,4],[1,4],[1,5]]
Output: [1,4]

Solution 1: Union Find

```
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n + 1))
        self.rank = [0] * (n + 1)

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, x, y):
        px, py = self.find(x), self.find(y)
        if px == py:
            return False

        if self.rank[px] < self.rank[py]:
            self.parent[px] = py
        elif self.rank[px] > self.rank[py]:
            self.parent[py] = px
        else:
            self.parent[py] = px
            self.rank[px] += 1
        return True

def findRedundantConnection(self, edges: List[List[int]]) -> List[int]:
    uf = UnionFind(len(edges))

    for u, v in edges:
        if not uf.union(u, v):
            return [u, v]

    return []
```

Time Complexity: O(N * α(N)) | Space Complexity: O(N)
Idea: Return edge whose union fails (creates cycle).

Solution 2: DFS Cycle Detection

```
def findRedundantConnection(self, edges: List[List[int]]) -> List[int]:
    def dfs(u, v, visited):
        if u == v:
            return True
        visited.add(u)
        for neighbor in graph[u]:
            if neighbor not in visited:
                if dfs(neighbor, v, visited):
                    return True
        return False

    graph = defaultdict(set)

    for u, v in edges:
        if u in graph and v in graph and dfs(u, v, set()):
            return [u, v]
        graph[u].add(v)
        graph[v].add(u)

    return []
```

Time Complexity: $O(N^2)$ | Space Complexity: O(N)
Idea: Build graph incrementally, check if edge connects already connected nodes.

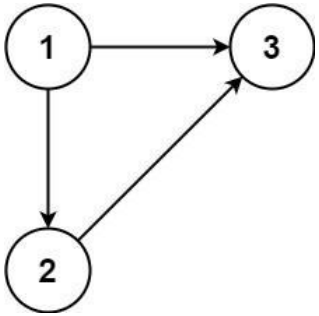
685. Redundant Connection II

The given input is a directed graph that started as a rooted tree with n nodes (with distinct values from 1 to n), with one additional directed edge added. The added edge has two different vertices chosen from 1 to n, and was not an edge that already existed.

The resulting graph is given as a 2D-array of edges. Each element of edges is a pair $[u_i, v_i]$ that represents a **directed** edge connecting nodes u_i and v_i, where u_i is a parent of child v_i.

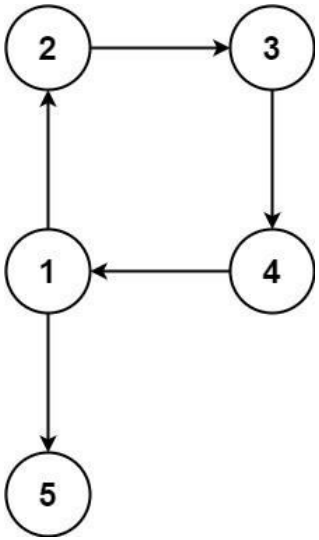
Return *an edge that can be removed so that the resulting graph is a rooted tree of* n *nodes*. If there are multiple answers, return the answer that occurs last in the given 2D-array.

Example 1:



Input: edges = [[1,2],[1,3],[2,3]]
Output: [2,3]

Example 2:



Input: edges = [[1,2],[2,3],[3,4],[4,1],[1,5]]
Output: [4,1]

Solution 1: Union Find with Cases

```
def findRedundantDirectedConnection(self, edges: List[List[int]]) -> List[int]:
    n = len(edges)
    parent = [0] * (n + 1)
    candidates = []

    # Find node with two parents
    for u, v in edges:
        if parent[v] != 0:
            candidates.append([parent[v], v])
            candidates.append([u, v])
        else:
            parent[v] = u

    # Union Find to detect cycle
    def find(x):
        if uf[x] != x:
            uf[x] = find(uf[x])
        return uf[x]

    uf = list(range(n + 1))

    for u, v in edges:
        # Skip candidate edge if exists
        if candidates and [u, v] == candidates[1]:
            continue

        pu, pv = find(u), find(v)
        if pu == pv:
            return candidates[0] if candidates else [u, v]
        uf[pv] = pu

    return candidates[1]
```

Time Complexity: O(N * α(N)) | Space Complexity: O(N)
Idea: Handle two cases: node with 2 parents or cycle.

721. Accounts Merge

Given a list of accounts where each element accounts[i] is a list of strings, where the first element accounts[i][0] is a name, and the rest of the elements are **emails** representing emails of the account.

Now, we would like to merge these accounts. Two accounts definitely belong to the same person if there is some common email to both accounts. Note that even if two accounts have the same name, they may belong to different people as people could have the same name. A person can have any number of accounts initially, but all of their accounts definitely have the same name.

After merging the accounts, return the accounts in the following format: the first element of each account is the name, and the rest of the elements are emails **in sorted order**. The accounts themselves can be returned in **any order**.

Example 1:

Input: accounts = [["John","johnsmith@mail.com","john_newyork@mail.com"],["John","johnsmith@mail.com","john00@mail.com"],["Mary","mary@mail.com"],["John","johnnybravo@mail.com"]]
Output: [["John","john00@mail.com","john_newyork@mail.com","johnsmith@mail.com"],["Mary","mary@mail.com"],["John","johnnybravo@mail.com"]]
Explanation:
The first and second John's are the same person as they have the common email "johnsmith@mail.com".
The third John and Mary are different people as none of their email addresses are used by other accounts.
We could return these lists in any order, for example the answer [['Mary', 'mary@mail.com'], ['John', 'johnnybravo@mail.com'], ['John', 'john00@mail.com', 'john_newyork@mail.com', 'johnsmith@mail.com']] would still be accepted.

Example 2:

Input: accounts = [["Gabe","Gabe0@m.co","Gabe3@m.co","Gabe1@m.co"],["Kevin","Kevin3@m.co","Kevin5@m.co","Kevin0@m.co"],["Ethan","Ethan5@m.co","Ethan4@m.co","Ethan0@m.co"],["Hanzo","Hanzo3@m.co","Hanzo1@m.co","Hanzo0@m.co"],["Fern","Fern5@m.co","Fern1@m.co","Fern0@m.co"]]
Output: [["Ethan","Ethan0@m.co","Ethan4@m.co","Ethan5@m.co"],["Gabe","Gabe0@m.co","Gabe1@m.co","Gabe3@m.co"],["Hanzo","Hanzo0@m.co","Hanzo1@m.co","Hanzo3@m.co"],["Kevin","Kevin0@m.co","Kevin3@m.co","Kevin5@m.co"],["Fern","Fern0@m.co","Fern1@m.co","Fern5@m.co"]]

Solution 1: Union Find

```
def accountsMerge(self, accounts: List[List[str]]) -> List[List[str]]:
    class UnionFind:
        def __init__(self, n):
            self.parent = list(range(n))

        def find(self, x):
            if self.parent[x] != x:
                self.parent[x] = self.find(self.parent[x])
            return self.parent[x]

        def union(self, x, y):
            px, py = self.find(x), self.find(y)
            if px != py:
                self.parent[py] = px

    n = len(accounts)
    uf = UnionFind(n)
    email_to_account = {}

    # Union accounts sharing emails
    for i, account in enumerate(accounts):
        for email in account[1:]:
            if email in email_to_account:
                uf.union(i, email_to_account[email])
            else:
                email_to_account[email] = i

    # Group emails by root account
    root_to_emails = defaultdict(list)
    for email, account in email_to_account.items():
        root = uf.find(account)
        root_to_emails[root].append(email)

    # Build result
    result = []
    for root, emails in root_to_emails.items():
        name = accounts[root][0]
        result.append([name] + sorted(emails))

    return result
```

Time Complexity: O(N * M * log M) | Space Complexity: O(N * M)
Idea: Union accounts sharing emails, then merge.

Solution 2: DFS

```
def accountsMerge(self, accounts: List[List[str]]) ->
List[List[str]]:
    def dfs(email, component):
        component.add(email)
        for neighbor in graph[email]:
            if neighbor not in component:
                dfs(neighbor, component)

    # Build email graph
    graph = defaultdict(set)
    email_to_name = {}

    for account in accounts:
        name = account[0]
        first_email = account[1]
        for email in account[1:]:
            email_to_name[email] = name
            graph[first_email].add(email)
            graph[email].add(first_email)

    # DFS to find connected components
    result = []
    visited = set()

    for email in graph:
        if email not in visited:
            component = set()
            dfs(email, component)
            visited.update(component)
            result.append([email_to_name[email]] +
sorted(component))

    return result
```

Time Complexity: O(N * M * log M) | Space Complexity: O(N * M)
Idea: Build email graph and find connected components.

1101. The Earliest Moment When Everyone Become Friends

Problem Statement: Find earliest time when all people become connected.

Solution 1: Union Find with Sorting

```
def earliestAcq(self, logs: List[List[int]], n: int) -> int:
    class UnionFind:
        def __init__(self, n):
            self.parent = list(range(n))
            self.rank = [0] * n
            self.components = n

        def find(self, x):
            if self.parent[x] != x:
                self.parent[x] = self.find(self.parent[x])
            return self.parent[x]

        def union(self, x, y):
            px, py = self.find(x), self.find(y)
            if px == py:
                return False

            if self.rank[px] < self.rank[py]:
                self.parent[px] = py
            elif self.rank[px] > self.rank[py]:
                self.parent[py] = px
            else:
                self.parent[py] = px
                self.rank[px] += 1

            self.components -= 1
            return True

    logs.sort(key=lambda x: x[0])
    uf = UnionFind(n)

    for timestamp, a, b in logs:
        uf.union(a, b)
        if uf.components == 1:
            return timestamp

    return -1
```

Time Complexity: O(M log M + M * α(N)) | Space Complexity: O(N)
Idea: Sort by time, union until one component.

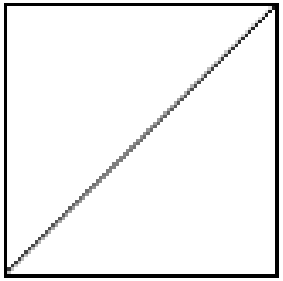
959. Regions Cut By Slashes

An n x n grid is composed of 1 x 1 squares where each 1 x 1 square consists of a '/', '\', or blank space ' '. These characters divide the square into contiguous regions.

Given the grid grid represented as a string array, return *the number of regions*.

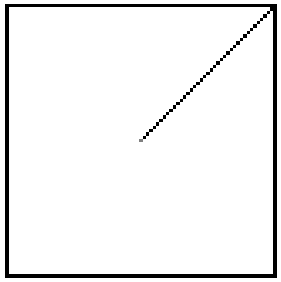
Note that backslash characters are escaped, so a '\' is represented as '\\'.

Example 1:



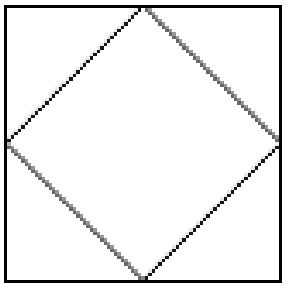
Input: grid = [" /","/ "]
Output: 2

Example 2:



Input: grid = [" /"," "]
Output: 1

Example 3:



Input: grid = ["/\","\/"]
Output: 5

Explanation: Recall that because \ characters are escaped, "\/" refers to \/, and "/\" refers to /\.

Solution 1: Union Find on Expanded Grid

```
def regionsBySlashes(self, grid: List[str]) -> int:
    n = len(grid)
    # Expand grid to 3x3 for each cell to handle slashes
    expanded = [[0] * (n * 3) for _ in range(n * 3)]

    for i in range(n):
        for j in range(n):
            if grid[i][j] == '/':
                expanded[i*3][j*3+2] = 1
                expanded[i*3+1][j*3+1] = 1
                expanded[i*3+2][j*3] = 1
            elif grid[i][j] == '\\':
                expanded[i*3][j*3] = 1
                expanded[i*3+1][j*3+1] = 1
                expanded[i*3+2][j*3+2] = 1

    # DFS to count regions
    def dfs(i, j):
        if i < 0 or i >= n*3 or j < 0 or j >= n*3 or expanded[i][j] == 1:
            return
        expanded[i][j] = 1
        for di, dj in [(0,1), (1,0), (0,-1), (-1,0)]:
            dfs(i + di, j + dj)

    count = 0
    for i in range(n*3):
        for j in range(n*3):
            if expanded[i][j] == 0:
                dfs(i, j)
                count += 1

    return count
```

Time Complexity: O(n²) | Space Complexity: O(n²)
Idea: Expand grid and count connected components.

Solution 2: Union Find on Triangles

```
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n
        self.count = n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, x, y):
        px, py = self.find(x), self.find(y)
        if px == py:
            return

        if self.rank[px] < self.rank[py]:
            self.parent[px] = py
        elif self.rank[px] > self.rank[py]:
            self.parent[py] = px
        else:
            self.parent[py] = px
            self.rank[px] += 1
        self.count -= 1

def regionsBySlashes(self, grid: List[str]) -> int:
    n = len(grid)
    # Each cell divided into 4 triangles: 0(top), 1(right), 2(bottom), 3(left)
    uf = UnionFind(4 * n * n)

    for i in range(n):
        for j in range(n):
            index = 4 * (i * n + j)

            # Connect triangles within cell
            if grid[i][j] == '/':
                uf.union(index, index + 3)
                uf.union(index + 1, index + 2)
            elif grid[i][j] == '\\':
                uf.union(index, index + 1)
                uf.union(index + 2, index + 3)
            else:  # space
                uf.union(index, index + 1)
                uf.union(index + 1, index + 2)
                uf.union(index + 2, index + 3)

            # Connect with adjacent cells
            if j + 1 < n:  # right
                uf.union(index + 1, 4 * (i * n + j + 1) + 3)
            if i + 1 < n:  # bottom
                uf.union(index + 2, 4 * ((i + 1) * n + j))

    return uf.count
```

Time Complexity: $O(n^2 * \alpha(n^2))$ | Space Complexity: $O(n^2)$

Idea: Each cell into 4 triangles, union based on slashes.

1202. Smallest String With Swaps

You are given a string s, and an array of pairs of indices in the string pairs where pairs[i] = [a, b] indicates 2 indices(0-indexed) of the string.

You can swap the characters at any pair of indices in the given pairs **any number of times**.

Return the lexicographically smallest string that s can be changed to after using the swaps.

Example 1:

Input: s = "dcab", pairs = [[0,3],[1,2]]
Output: "bacd"
Explaination:
Swap s[0] and s[3], s = "bcad"
Swap s[1] and s[2], s = "bacd"

Example 2:

Input: s = "dcab", pairs = [[0,3],[1,2],[0,2]]
Output: "abcd"
Explaination:
Swap s[0] and s[3], s = "bcad"
Swap s[0] and s[2], s = "acbd"
Swap s[1] and s[2], s = "abcd"

Example 3:

Input: s = "cba", pairs = [[0,1],[1,2]]
Output: "abc"
Explaination:
Swap s[0] and s[1], s = "bca"
Swap s[1] and s[2], s = "bac"
Swap s[0] and s[1], s = "abc"

Solution 1: Union Find

```
def smallestStringWithSwaps(self, s: str, pairs: List[List[int]]) -> str:
    class UnionFind:
        def __init__(self, n):
            self.parent = list(range(n))
            self.rank = [0] * n

        def find(self, x):
            if self.parent[x] != x:
                self.parent[x] = self.find(self.parent[x])
            return self.parent[x]

        def union(self, x, y):
            px, py = self.find(x), self.find(y)
            if px == py:
                return
            if self.rank[px] < self.rank[py]:
                self.parent[px] = py
            elif self.rank[px] > self.rank[py]:
                self.parent[py] = px
            else:
                self.parent[py] = px
                self.rank[px] += 1

    n = len(s)
    uf = UnionFind(n)

    # Union indices that can be swapped
    for a, b in pairs:
        uf.union(a, b)

    # Group indices by root
    groups = defaultdict(list)
    for i in range(n):
        root = uf.find(i)
        groups[root].append(i)

    # Sort characters in each group
    result = list(s)
    for indices in groups.values():
        chars = sorted([s[i] for i in indices])
        for i, ch in zip(sorted(indices), chars):
            result[i] = ch

    return "".join(result)
```

Time Complexity: O(N log N + M * α(N)) | Space Complexity: O(N)
Idea: Group swappable indices, sort within groups.

Solution 2: DFS

```
def smallestStringWithSwaps(self, s: str, pairs: List[List[int]]) -> str:
    def dfs(node, component):
        component.append(node)
        visited[node] = True
        for neighbor in graph[node]:
            if not visited[neighbor]:
                dfs(neighbor, component)

    # Build graph
    graph = defaultdict(list)
    for a, b in pairs:
        graph[a].append(b)
        graph[b].append(a)

    visited = [False] * len(s)
    result = list(s)
    for i in range(len(s)):
        if not visited[i]:
            component = []
            dfs(i, component)

            # Sort indices and characters
            indices = sorted(component)
            chars = sorted([s[idx] for idx in indices])

            for idx, ch in zip(indices, chars):
                result[idx] = ch

    return "".join(result)
```

Time Complexity: O(N log N + V + E) | Space Complexity: O(N + E)
Idea: Find connected components via DFS.

305. Number of Islands II

Problem Statement: Track number of islands as positions are added.

Solution 1: Union Find

```
def numIslands2(self, m: int, n: int, positions: List[List[int]]) -> List[int]:
    class UnionFind:
        def __init__(self, n):
            self.parent = [-1] * n
            self.rank = [0] * n
            self.count = 0

        def find(self, x):
            if self.parent[x] != x:
                self.parent[x] = self.find(self.parent[x])
            return self.parent[x]

        def add(self, x):
            if self.parent[x] == -1:
                self.parent[x] = x
                self.count += 1

        def union(self, x, y):
            px, py = self.find(x), self.find(y)
            if px == py:
                return

            if self.rank[px] < self.rank[py]:
                self.parent[px] = py
            elif self.rank[px] > self.rank[py]:
                self.parent[py] = px
            else:
                self.parent[py] = px
                self.rank[px] += 1
            self.count -= 1
```

```
    def get_index(i, j):
        return i * n + j

    uf = UnionFind(m * n)
    result = []
    directions = [(0,1), (1,0), (0,-1), (-1,0)]

    for i, j in positions:
        idx = get_index(i, j)
        uf.add(idx)

        for di, dj in directions:
            ni, nj = i + di, j + dj
            if 0 <= ni < m and 0 <= nj < n:
                nidx = get_index(ni, nj)
                if uf.parent[nidx] != -1:
                    uf.union(idx, nidx)

        result.append(uf.count)

    return result
```

Time Complexity: O(k * α(mn)) | Space Complexity: O(mn)
Idea: Dynamic Union Find with land addition.

Solution 2: BFS with Visited

```
def numIslands2(self, m: int, n: int, positions: List[List[int]])
-> List[int]:
    def bfs(start_i, start_j, island_id):
        queue = deque([(start_i, start_j)])
        visited.add((start_i, start_j))
        island_map[start_i][start_j] = island_id

        while queue:
            i, j = queue.popleft()
            for di, dj in [(0,1), (1,0), (0,-1), (-1,0)]:
                ni, nj = i + di, j + dj
                if 0 <= ni < m and 0 <= nj < n and grid[ni][nj] ==
1 and (ni, nj) not in visited:
                    visited.add((ni, nj))
                    island_map[ni][nj] = island_id
                    queue.append((ni, nj))

    grid = [[0] * n for _ in range(m)]
    island_map = [[-1] * n for _ in range(m)]
    result = []
    island_count = 0
    visited = set()

    for i, j in positions:
        if grid[i][j] == 1:
            result.append(island_count)
            continue

        grid[i][j] = 1

        # Check adjacent islands
        adjacent_islands = set()
        for di, dj in [(0,1), (1,0), (0,-1), (-1,0)]:
            ni, nj = i + di, j + dj
            if 0 <= ni < m and 0 <= nj < n and grid[ni][nj] == 1:
                adjacent_islands.add(island_map[ni][nj])

        if not adjacent_islands:
            # New island
            island_count += 1
            visited = set()
            bfs(i, j, island_count)
        elif len(adjacent_islands) == 1:
            # Connect to one island
            island_id = next(iter(adjacent_islands))
            visited = set()
            bfs(i, j, island_id)
        else:
            # Merge multiple islands
            island_count -= len(adjacent_islands) - 1
            # BFS to update all connected cells to smallest
island_id
            min_id = min(adjacent_islands)
            visited = set()
            bfs(i, j, min_id)
            # Update other islands to min_id
            for ai, aj in positions:
                if grid[ai][aj] == 1 and island_map[ai][aj] in
adjacent_islands:
                    island_map[ai][aj] = min_id

        result.append(island_count)

    return result
```

Time Complexity: O(k * mn) | Space Complexity: O(mn)
Idea: Track islands with BFS on each addition.

These solutions cover various Union Find and Graph problems with multiple approaches, demonstrating different trade-offs in time and space complexity.

BIT MANIPULATION

191. Number of 1 Bits

Given a positive integer n, write a function that returns the number of set bits in its binary representation (also known as the Hamming weight).

Example 1:

Input: n = 11

Output: 3

Explanation:

The input binary string **1011** has a total of three set bits.

Example 2:

Input: n = 128

Output: 1

Explanation:

The input binary string **10000000** has a total of one set bit.

Example 3:

Input: n = 2147483645

Output: 30

Explanation:

The input binary string **11111111111111111111111111111101** has a total of thirty set bits.

Solution 1: Bit Manipulation (Right Shift)

```
def hammingWeight(self, n: int) -> int:
    count = 0
    while n:
        count += n & 1
        n >>= 1
    return count
```

Time Complexity: O(32) | Space Complexity: O(1)
Idea: Check LSB and shift right until n becomes 0.

Solution 2: Brian Kernighan's Algorithm

```
def hammingWeight(self, n: int) -> int:
    count = 0
    while n:
        n &= (n - 1)  # Removes the lowest set bit
        count += 1
    return count
```

Time Complexity: O(number of 1 bits) | Space Complexity: O(1)
Idea: n & (n-1) flips the lowest set bit to 0.

Solution 3: Built-in Function

```
def hammingWeight(self, n: int) -> int:
    return bin(n).count('1')
```

Time Complexity: O(32) | Space Complexity: O(1)
Idea: Use Python's built-in binary conversion.

Solution 4: Lookup Table (Precomputation)

```
def hammingWeight(self, n: int) -> int:
    # Precompute for 8-bit chunks
    lookup = [0] * 256
    for i in range(256):
        lookup[i] = lookup[i >> 1] + (i & 1)

    count = 0
    for i in range(4):  # 32 bits = 4 bytes
        count += lookup[n & 0xFF]
        n >>= 8
    return count
```

Time Complexity: O(1) | Space Complexity: O(256)
Idea: Precompute popcount for 8-bit chunks and combine.

338. Counting Bits

Given an integer n, return *an array* ans *of length* n + 1 *such that for each* i (0 <= i <= n), ans[i] *is the* ***number of*** 1***'s*** *in the binary representation of* i.

Example 1:

Input: n = 2
Output: [0,1,1]
Explanation:
0 --> 0
1 --> 1
2 --> 10

Example 2:

Input: n = 5
Output: [0,1,1,2,1,2]
Explanation:
0 --> 0
1 --> 1
2 --> 10
3 --> 11
4 --> 100
5 --> 101

Solution 1: Brian Kernighan's for Each

```
def countBits(self, n: int) -> List[int]:
    def popcount(x):
        count = 0
        while x:
            x &= (x - 1)
            count += 1
        return count

    return [popcount(i) for i in range(n + 1)]
```

Time Complexity: O(n log n) | Space Complexity: O(n)
Idea: Apply Kernighan's algorithm for each number.

Solution 2: DP with LSB

```
def countBits(self, n: int) -> List[int]:
    dp = [0] * (n + 1)

    for i in range(1, n + 1):
        dp[i] = dp[i >> 1] + (i & 1)

    return dp
```

Time Complexity: O(n) | Space Complexity: O(n)
Idea: dp[i] = dp[i//2] + i%2 (right shift removes LSB).

Solution 3: DP with Lowest Set Bit

```
def countBits(self, n: int) -> List[int]:
    dp = [0] * (n + 1)

    for i in range(1, n + 1):
        dp[i] = dp[i & (i - 1)] + 1

    return dp
```

Time Complexity: O(n) | Space Complexity: O(n)
Idea: i & (i-1) removes lowest set bit, so add 1.

Solution 4: DP with Offset (Power of Two)

```
def countBits(self, n: int) -> List[int]:
    dp = [0] * (n + 1)
    offset = 1

    for i in range(1, n + 1):
        if offset * 2 == i:
            offset = i
        dp[i] = 1 + dp[i - offset]

    return dp
```

Time Complexity: O(n) | Space Complexity: O(n)
Idea: Numbers between powers of two follow pattern: dp[i] = 1 + dp[i - offset].

190. Reverse Bits

Reverse bits of a given 32 bits signed integer.

Example 1:

Input: n = 43261596

Output: 964176192

Explanation:

Integer	Binary
43261596	00000010100101000001111010011100
964176192	00111001011110000010100101000000

Example 2:

Input: n = 2147483644

Output: 1073741822

Explanation:

Integer	Binary
2147483644	01111111111111111111111111111100
1073741822	00111111111111111111111111111110

Solution 1: Bit by Bit

```
def reverseBits(self, n: int) -> int:
    result = 0
    for i in range(32):
        result = (result << 1) | (n & 1)
        n >>= 1
    return result
```

Time Complexity: O(32) | Space Complexity: O(1)
Idea: Take LSB of n and put as MSB of result.

Solution 2: Divide and Conquer (Masking)

```
def reverseBits(self, n: int) -> int:
    # 32-bit masks
    m1 = 0x55555555  #
01010101010101010101010101010101
    m2 = 0x33333333  #
00110011001100110011001100110011
    m4 = 0x0f0f0f0f  #
00001111000011110000111100001111
    m8 = 0x00ff00ff  #
00000000111111110000000011111111
    m16 = 0x0000ffff  #
00000000000000001111111111111111

    # Swap adjacent bits
    n = ((n >> 1) & m1) | ((n & m1) << 1)
    # Swap 2-bit chunks
    n = ((n >> 2) & m2) | ((n & m2) << 2)
    # Swap 4-bit chunks
    n = ((n >> 4) & m4) | ((n & m4) << 4)
    # Swap 8-bit chunks
    n = ((n >> 8) & m8) | ((n & m8) << 8)
    # Swap 16-bit chunks
    n = ((n >> 16) & m16) | ((n & m16) << 16)

    return n
```

Time Complexity: O(1) | Space Complexity: O(1)
Idea: Swap adjacent bits, then larger chunks recursively.

Solution 3: Byte Swap with Lookup Table

```
def reverseBits(self, n: int) -> int:
    # Precompute reverse for 8-bit numbers
    rev = [0] * 256
    for i in range(256):
        rev[i] = int('{:08b}'.format(i)[::-1], 2)

    # Reverse each byte and combine
    return (rev[n & 0xFF] << 24) | \
           (rev[(n >> 8) & 0xFF] << 16) | \
           (rev[(n >> 16) & 0xFF] << 8) | \
           (rev[(n >> 24) & 0xFF])
```

Time Complexity: O(1) | Space Complexity: O(256)
Idea: Precompute reversed bytes and combine.

268. Missing Number

Given an array nums containing n distinct numbers in the range [0, n], return *the only number in the range that is missing from the array.*

Example 1:

Input: nums = [3,0,1]

Output: 2

Explanation:

n = 3 since there are 3 numbers, so all numbers are in the range [0,3]. 2 is the missing number in the range since it does not appear in nums.

Example 2:

Input: nums = [0,1]

Output: 2

Explanation:

n = 2 since there are 2 numbers, so all numbers are in the range [0,2]. 2 is the missing number in the range since it does not appear in nums.

Example 3:

Input: nums = [9,6,4,2,3,5,7,0,1]

Output: 8

Explanation:

n = 9 since there are 9 numbers, so all numbers are in the range [0,9]. 8 is the missing number in the range since it does not appear in nums.

Solution 1: XOR

```
def missingNumber(self, nums: List[int]) -> int:
    missing = len(nums)
    for i, num in enumerate(nums):
        missing ^= i ^ num
    return missing
```

Time Complexity: O(n) | Space Complexity: O(1)
Idea: XOR of index and value cancels out existing numbers.

Solution 2: Math (Sum)

```
def missingNumber(self, nums: List[int]) -> int:
    n = len(nums)
    expected_sum = n * (n + 1) // 2
    actual_sum = sum(nums)
    return expected_sum - actual_sum
```

Time Complexity: O(n) | Space Complexity: O(1)
Idea: Sum of 0..n minus sum of array.

Solution 3: Sorting

```
def missingNumber(self, nums: List[int]) -> int:
    nums.sort()

    for i, num in enumerate(nums):
        if i != num:
            return i

    return len(nums)
```

Time Complexity: O(n log n) | Space Complexity: O(1)
Idea: Sorted array should have numbers matching indices.

Solution 4: HashSet

```
def missingNumber(self, nums: List[int]) -> int:
    num_set = set(nums)

    for i in range(len(nums) + 1):
        if i not in num_set:
            return i

    return -1
```

Time Complexity: O(n) | Space Complexity: O(n)
Idea: Check presence of each number from 0 to n.

371. Sum of Two Integers

Given two integers a and b, return *the sum of the two integers without using the operators + and -.*

Example 1:

Input: a = 1, b = 2
Output: 3

Example 2:

Input: a = 2, b = 3
Output: 5

Solution 1: Bit Manipulation (Iterative)

```
def getSum(self, a: int, b: int) -> int:
    mask = 0xFFFFFFFF  # 32-bit mask
    max_int = 0x7FFFFFFF

    while b != 0:
        # Calculate carry
        carry = (a & b) & mask
        # Sum without carry
        a = (a ^ b) & mask
        # Shift carry for next iteration
        b = (carry << 1) & mask

    return a if a <= max_int else ~(a ^ mask)
```

Time Complexity: O(1) | Space Complexity: O(1)
Idea: XOR for sum, AND for carry, repeat until no carry.

Solution 2: Bit Manipulation (Recursive)

```
def getSum(self, a: int, b: int) -> int:
    if b == 0:
        return a

    mask = 0xFFFFFFFF
    max_int = 0x7FFFFFFF

    # Calculate sum without carry and carry
    sum_without_carry = (a ^ b) & mask
    carry = ((a & b) << 1) & mask

    result = self.getSum(sum_without_carry, carry)
    return result if result <= max_int else ~(result ^ mask)
```

Time Complexity: O(1) | Space Complexity: O(1)
Idea: Recursive version of bit manipulation.

Solution 3: Using Log and Exp (Cheating)

```
def getSum(self, a: int, b: int) -> int:
    return int(math.log(math.exp(a) * math.exp(b)))
```

Time Complexity: O(1) | Space Complexity: O(1)
Idea: e^a * e^b = e^(a+b), then take log. (Not reliable for negatives)

---

78. Subsets

Given an integer array nums of **unique** elements, return *all possible subsets (the power set).*

The solution set **must not** contain duplicate subsets. Return the solution in **any order**.

Example 1:

Input: nums = [1,2,3]
Output: [[],[1],[2],[1,2],[3],[1,3],[2,3],[1,2,3]]

Example 2:

Input: nums = [0]
Output: [[],[0]]

Solution 1: Bitmasking (Iterative)

```
def subsets(self, nums: List[int]) -> List[List[int]]:
    n = len(nums)
    result = []

    for mask in range(1 << n):
        subset = []
        for i in range(n):
            if mask & (1 << i):
                subset.append(nums[i])
        result.append(subset)

    return result
```

Time Complexity: $O(n * 2^n)$ | Space Complexity: $O(n * 2^n)$
Idea: Use bits to represent inclusion/exclusion of each element.

Solution 2: Backtracking (DFS)

```
def subsets(self, nums: List[int]) -> List[List[int]]:
    def backtrack(start, path):
        result.append(path[:])

        for i in range(start, len(nums)):
            path.append(nums[i])
            backtrack(i + 1, path)
            path.pop()

    result = []
    backtrack(0, [])
    return result
```

Time Complexity: O(n * 2^n) | Space Complexity: O(n * 2^n)
Idea: Recursively build subsets by including/excluding elements.

Solution 3: Cascading (Iterative)

```
def subsets(self, nums: List[int]) -> List[List[int]]:
    result = [[]]

    for num in nums:
        result += [curr + [num] for curr in result]

    return result
```

Time Complexity: O(n * 2^n) | Space Complexity: O(n * 2^n)
Idea: Start with empty set, add each number to all existing subsets.

201. Bitwise AND of Numbers Range

Given two integers left and right that represent the range [left, right], return *the bitwise AND of all numbers in this range, inclusive*.

Example 1:

Input: left = 5, right = 7
Output: 4

Example 2:

Input: left = 0, right = 0
Output: 0

Example 3:

Input: left = 1, right = 2147483647
Output: 0

Solution 1: Shift Common Prefix

```
def rangeBitwiseAnd(self, left: int, right: int) -> int:
    shift = 0

    while left < right:
        left >>= 1
        right >>= 1
        shift += 1

    return left << shift
```

Time Complexity: O(log n) | Space Complexity: O(1)
Idea: Find common prefix of left and right, rest bits become 0.

Solution 2: Clear Lowest Bits

```
def rangeBitwiseAnd(self, left: int, right: int) -> int:
    while left < right:
        right &= (right - 1)

    return right
```

Time Complexity: O(log n) | Space Complexity: O(1)
Idea: Clear lowest set bit of right until it's <= left.

Solution 3: Brian Kernighan's Approach

```
def rangeBitwiseAnd(self, left: int, right: int) -> int:
    while left < right:
        right -= right & -right  # Remove lowest set bit
    return right
```

Time Complexity: O(log n) | Space Complexity: O(1)
Idea: Remove lowest set bit from right until it's <= left.

136. Single Number

Given a **non-empty** array of integers nums, every element appears *twice* except for one. Find that single one.

You must implement a solution with a linear runtime complexity and use only constant extra space.

Example 1:

Input: nums = [2,2,1]

Output: 1

Example 2:

Input: nums = [4,1,2,1,2]

Output: 4

Example 3:

Input: nums = [1]

Output: 1

Solution 1: XOR (Optimal)

```
def singleNumber(self, nums: List[int]) -> int:
    result = 0
    for num in nums:
        result ^= num
    return result
```

Time Complexity: O(n) | Space Complexity: O(1)
Idea: XOR of a number with itself is 0, XOR with 0 is the number itself.

Solution 2: HashMap

```
def singleNumber(self, nums: List[int]) -> int:
    count = {}
    for num in nums:
        count[num] = count.get(num, 0) + 1

    for num, freq in count.items():
        if freq == 1:
            return num

    return -1
```

Time Complexity: O(n) | Space Complexity: O(n)
Idea: Count frequencies and find element with count 1.

Solution 3: Math (2*sum(set) - sum)

```
def singleNumber(self, nums: List[int]) -> int:
    return 2 * sum(set(nums)) - sum(nums)
```

Time Complexity: O(n) | Space Complexity: O(n)
Idea: If all numbers appeared twice, sum would be 2*sum(set).

Solution 4: Sorting

```
def singleNumber(self, nums: List[int]) -> int:
    nums.sort()

    for i in range(0, len(nums) - 1, 2):
        if nums[i] != nums[i + 1]:
            return nums[i]

    return nums[-1]
```

Time Complexity: O(n log n) | Space Complexity: O(1)
Idea: Sorted array has pairs adjacent except the single element.

137. Single Number II

Given an integer array nums where every element appears **three times** except for one, which appears **exactly once**. *Find the single element and return it*.

You must implement a solution with a linear runtime complexity and use only constant extra space.

Example 1:

Input: nums = [2,2,3,2]
Output: 3

Example 2:

Input: nums = [0,1,0,1,0,1,99]
Output: 99

Solution 1: Bit Counting

```
def singleNumber(self, nums: List[int]) -> int:
    result = 0

    for i in range(32):
        count = 0
        for num in nums:
            # Count 1's at ith position
            if (num >> i) & 1:
                count += 1

        # If count not multiple of 3, this bit belongs to single number
        if count % 3:
            result |= (1 << i)

    # Handle negative numbers
    if result >= 231:
        result -= 232

    return result
```

Time Complexity: O(32n) | Space Complexity: O(1)
Idea: Count bits at each position, single number's bits appear once mod 3.

Solution 2: Finite State Machine

```
def singleNumber(self, nums: List[int]) -> int:
    ones = twos = 0

    for num in nums:
        ones = (ones ^ num) & ~twos
        twos = (twos ^ num) & ~ones

    return ones
```

Time Complexity: O(n) | Space Complexity: O(1)
Idea: Use two variables to track bits appearing 1,2,3 times (mod 3).

Solution 3: HashMap

```
def singleNumber(self, nums: List[int]) -> int:
    count = {}
    for num in nums:
        count[num] = count.get(num, 0) + 1

    for num, freq in count.items():
        if freq == 1:
            return num

    return -1
```

Time Complexity: O(n) | Space Complexity: O(n)
Idea: Count frequencies and find element with count 1.

---

260. Single Number III

Given an integer array nums, in which exactly two elements appear only once and all the other elements appear exactly twice. Find the two elements that appear only once. You can return the answer in **any order**.

You must write an algorithm that runs in linear runtime complexity and uses only constant extra space.

Example 1:

Input: nums = [1,2,1,3,2,5]
Output: [3,5]
Explanation: [5, 3] is also a valid answer.

Example 2:

Input: nums = [-1,0]
Output: [-1,0]

Example 3:

Input: nums = [0,1]
Output: [1,0]

Solution 1: XOR + Bitmask

```
def singleNumber(self, nums: List[int]) -> List[int]:
    # XOR all numbers to get xor of the two unique numbers
    xor_all = 0
    for num in nums:
        xor_all ^= num

    # Find rightmost set bit (differentiating bit)
    diff_bit = xor_all & -xor_all

    # Partition numbers based on that bit
    x = y = 0
    for num in nums:
        if num & diff_bit:
            x ^= num
        else:
            y ^= num

    return [x, y]
```

Time Complexity: O(n) | Space Complexity: O(1)
Idea: XOR gives a^b, find a differentiating bit to separate numbers.

Solution 2: HashMap

```
def singleNumber(self, nums: List[int]) -> List[int]:
    count = {}
    for num in nums:
        count[num] = count.get(num, 0) + 1

    result = []
    for num, freq in count.items():
        if freq == 1:
            result.append(num)

    return result
```

Time Complexity: O(n) | Space Complexity: O(n)
Idea: Count frequencies and find elements with count 1.

Solution 3: Sorting + Pair Check

```
def singleNumber(self, nums: List[int]) -> List[int]:
    nums.sort()
    result = []
    i = 0

    while i < len(nums) - 1:
        if nums[i] == nums[i + 1]:
            i += 2
        else:
            result.append(nums[i])
            i += 1

    if i < len(nums):
        result.append(nums[i])

    return result
```

Time Complexity: O(n log n) | Space Complexity: O(1)
Idea: Sorted array has pairs except the two single numbers.

Solution 4: Set Subtraction

```
def singleNumber(self, nums: List[int]) -> List[int]:
    from collections import Counter

    counter = Counter(nums)
    return [num for num, count in counter.items() if count == 1]
```

Time Complexity: O(n) | Space Complexity: O(n)
Idea: Use Counter to get frequencies.

INTERVALS

56. Merge Intervals

Given an array of intervals where intervals[i] = [$start_i$, end_i], merge all overlapping intervals, and return *an array of the non-overlapping intervals that cover all the intervals in the input*.

Example 1:

Input: intervals = [[1,3],[2,6],[8,10],[15,18]]
Output: [[1,6],[8,10],[15,18]]
Explanation: Since intervals [1,3] and [2,6] overlap, merge them into [1,6].

Example 2:

Input: intervals = [[1,4],[4,5]]
Output: [[1,5]]
Explanation: Intervals [1,4] and [4,5] are considered overlapping.

Example 3:

Input: intervals = [[4,7],[1,4]]
Output: [[1,7]]
Explanation: Intervals [1,4] and [4,7] are considered overlapping.

Solution 1: Sorting + Iteration

```
def merge(self, intervals: List[List[int]]) -> List[List[int]]:
    if not intervals:
        return []

    # Sort by start time
    intervals.sort(key=lambda x: x[0])
    merged = [intervals[0]]

    for start, end in intervals[1:]:
        last_end = merged[-1][1]

        if start <= last_end:
            # Overlapping - merge
            merged[-1][1] = max(last_end, end)
        else:
            # Non-overlapping - add new interval
            merged.append([start, end])

    return merged
```

Time Complexity: O(n log n) | Space Complexity: O(n)
Idea: Sort by start, then merge overlapping intervals.

Solution 2: Sorting + In-place

```
def merge(self, intervals: List[List[int]]) -> List[List[int]]:
    intervals.sort(key=lambda x: x[0])
    i = 0

    while i < len(intervals) - 1:
        if intervals[i][1] >= intervals[i+1][0]:
            # Merge
            intervals[i][1] = max(intervals[i][1], intervals[i+1][1])
            intervals.pop(i+1)
        else:
            i += 1

    return intervals
```

Time Complexity: O(n log n) | Space Complexity: O(1)
Idea: Merge in-place to save space.

Solution 3: Graph Approach

```
def merge(self, intervals: List[List[int]]) -> List[List[int]]:
    # Build graph of overlapping intervals
    n = len(intervals)
    graph = {i: [] for i in range(n)}

    for i in range(n):
        for j in range(i+1, n):
            if intervals[i][1] >= intervals[j][0] and intervals[j][1] >= intervals[i][0]:
                graph[i].append(j)
                graph[j].append(i)

    # Find connected components
    visited = [False] * n
    result = []

    for i in range(n):
        if not visited[i]:
            # DFS to find component
            stack = [i]
            visited[i] = True
            component = []

            while stack:
                node = stack.pop()
                component.append(node)
                for neighbor in graph[node]:
                    if not visited[neighbor]:
                        visited[neighbor] = True
                        stack.append(neighbor)

            # Merge intervals in component
            start = min(intervals[node][0] for node in component)
            end = max(intervals[node][1] for node in component)
            result.append([start, end])

    return result
```

Time Complexity: O(n²) | Space Complexity: O(n²)
Idea: Build overlap graph and find connected components.

57. Insert Interval

You are given an array of non-overlapping intervals intervals where intervals[i] = [$start_i$, end_i] represent the start and the end of the ith interval and intervals is sorted in ascending order by $start_i$. You are also given an interval newInterval = [start, end] that represents the start and end of another interval.

Insert newInterval into intervals such that intervals is still sorted in ascending order by $start_i$ and intervals still does not have any overlapping intervals (merge overlapping intervals if necessary).

Return intervals *after the insertion*.

Note that you don't need to modify intervals in-place. You can make a new array and return it.

Example 1:

Input: intervals = [[1,3],[6,9]], newInterval = [2,5]
Output: [[1,5],[6,9]]

Example 2:

Input: intervals = [[1,2],[3,5],[6,7],[8,10],[12,16]], newInterval = [4,8]
Output: [[1,2],[3,10],[12,16]]
Explanation: Because the new interval [4,8] overlaps with [3,5],[6,7],[8,10].

Solution 1: Linear Scan

```
def insert(self, intervals: List[List[int]], newInterval: List[int]) -> List[List[int]]:
    result = []
    i = 0
    n = len(intervals)

    # Add all intervals ending before new interval starts
    while i < n and intervals[i][1] < newInterval[0]:
        result.append(intervals[i])
        i += 1

    # Merge overlapping intervals
    while i < n and intervals[i][0] <= newInterval[1]:
        newInterval[0] = min(newInterval[0], intervals[i][0])
        newInterval[1] = max(newInterval[1], intervals[i][1])
        i += 1

    result.append(newInterval)

    # Add remaining intervals
    while i < n:
        result.append(intervals[i])
        i += 1

    return result
```

Time Complexity: O(n) | Space Complexity: O(n)
Idea: Process intervals in three phases: before, overlapping, after.

Solution 2: Binary Search + Merge

```
def insert(self, intervals: List[List[int]], newInterval: List[int])
-> List[List[int]]:
    # Find position to insert using binary search
    def binary_search(intervals, val):
        left, right = 0, len(intervals)
        while left < right:
            mid = (left + right) // 2
            if intervals[mid][0] < val:
                left = mid + 1
            else:
                right = mid
        return left

    pos = binary_search(intervals, newInterval[0])
    intervals.insert(pos, newInterval)

    # Merge
    result = [intervals[0]]
    for start, end in intervals[1:]:
        if start <= result[-1][1]:
            result[-1][1] = max(result[-1][1], end)
        else:
            result.append([start, end])

    return result
```

Time Complexity: O(n) | Space Complexity: O(n)
Idea: Insert at correct position then merge.

Solution 3: Two Pointers

```
def insert(self, intervals: List[List[int]], newInterval: List[int])
-> List[List[int]]:
    result = []
    i = 0
    n = len(intervals)
    new_start, new_end = newInterval

    while i < n and intervals[i][1] < new_start:
        result.append(intervals[i])
        i += 1

    while i < n and intervals[i][0] <= new_end:
        new_start = min(new_start, intervals[i][0])
        new_end = max(new_end, intervals[i][1])
        i += 1

    result.append([new_start, new_end])

    while i < n:
        result.append(intervals[i])
        i += 1

    return result
```

Time Complexity: O(n) | Space Complexity: O(n)
Idea: Simplified version of linear scan.

252. Meeting Rooms

Given an array of meeting time intervals consisting of start and end times [[s1,e1],[s2,e2],...] (si < ei), determine if a person could attend all meetings.

Example 1:

Input: [[0,30],[5,10],[15,20]]
Output: false
Example 2:

Input: [[7,10],[2,4]]
Output: true

Solution 1: Sorting

```
def canAttendMeetings(self, intervals: List[List[int]]) ->
bool:
    intervals.sort(key=lambda x: x[0])

    for i in range(1, len(intervals)):
        if intervals[i][0] < intervals[i-1][1]:
            return False

    return True
```

Time Complexity: O(n log n) | Space Complexity: O(1)
Idea: Sort and check if next starts before previous ends.

Solution 2: Line Sweep

```
def canAttendMeetings(self, intervals: List[List[int]]) ->
bool:
    events = []
    for start, end in intervals:
        events.append((start, 1))  # +1 for start
        events.append((end, -1))   # -1 for end

    events.sort()
    count = 0

    for time, delta in events:
        count += delta
        if count > 1:
            return False

    return True
```

Time Complexity: O(n log n) | Space Complexity: O(n)
Idea: Count active meetings at any time.

Solution 3: Brute Force

```
def canAttendMeetings(self, intervals: List[List[int]]) ->
bool:
    for i in range(len(intervals)):
        for j in range(i+1, len(intervals)):
            a, b = intervals[i], intervals[j]
            if not (a[1] <= b[0] or b[1] <= a[0]):
                return False
    return True
```

Time Complexity: O(n²) | Space Complexity: O(1)
Idea: Check all pairs for overlap.

---

253. Meeting Rooms II

Given an array of meeting time intervals consisting of start and end times [[s1,e1],[s2,e2],...] (si < ei), find the minimum number of conference rooms required.

Example 1:

Input: [[0, 30],[5, 10],[15, 20]]
Output: 2
Example 2:

Input: [[7,10],[2,4]]
Output: 1

Solution 1: Priority Queue (Min Heap)

```
def minMeetingRooms(self, intervals: List[List[int]]) -> int:
    if not intervals:
        return 0

    intervals.sort(key=lambda x: x[0])
    rooms = [intervals[0][1]]  # Store end times

    for start, end in intervals[1:]:
        if start >= rooms[0]:
            heapq.heapreplace(rooms, end)
        else:
            heapq.heappush(rooms, end)

    return len(rooms)
```

Time Complexity: O(n log n) | Space Complexity: O(n)
Idea: Track earliest ending meeting room.

Solution 2: Two Pointers (Chronological Ordering)

```
def minMeetingRooms(self, intervals: List[List[int]]) -> int:
    starts = sorted([i[0] for i in intervals])
    ends = sorted([i[1] for i in intervals])

    rooms = 0
    end_ptr = 0

    for start in starts:
        if start < ends[end_ptr]:
            rooms += 1
        else:
            end_ptr += 1

    return rooms
```

Time Complexity: O(n log n) | Space Complexity: O(n)
Idea: Separate starts and ends, move end pointer when meeting ends.

Solution 3: Line Sweep

```
def minMeetingRooms(self, intervals: List[List[int]]) -> int:
    events = []
    for start, end in intervals:
        events.append((start, 1))
        events.append((end, -1))

    events.sort()
    count = 0
    max_count = 0

    for time, delta in events:
        count += delta
        max_count = max(max_count, count)

    return max_count
```

Time Complexity: O(n log n) | Space Complexity: O(n)
Idea: Track active meetings count over time.

Solution 4: TreeMap

```
def minMeetingRooms(self, intervals: List[List[int]]) -> int:
    from sortedcontainers import SortedDict

    timeline = SortedDict()

    for start, end in intervals:
        timeline[start] = timeline.get(start, 0) + 1
        timeline[end] = timeline.get(end, 0) - 1

    rooms = 0
    max_rooms = 0

    for count in timeline.values():
        rooms += count
        max_rooms = max(max_rooms, rooms)

    return max_rooms
```

Time Complexity: O(n log n) | Space Complexity: O(n)
Idea: Use sorted map for timeline.

435. Non-overlapping Intervals

Given an array of intervals intervals where intervals[i] = [$start_i$, end_i], return *the minimum number of intervals you need to remove to make the rest of the intervals non-overlapping*.

Note that intervals which only touch at a point are **non-overlapping**. For example, [1, 2] and [2, 3] are non-overlapping.

Example 1:

Input: intervals = [[1,2],[2,3],[3,4],[1,3]]
Output: 1
Explanation: [1,3] can be removed and the rest of the intervals are non-overlapping.

Example 2:

Input: intervals = [[1,2],[1,2],[1,2]]
Output: 2
Explanation: You need to remove two [1,2] to make the rest of the intervals non-overlapping.

Example 3:

Input: intervals = [[1,2],[2,3]]
Output: 0
Explanation: You don't need to remove any of the intervals since they're already non-overlapping.

Solution 1: Greedy by End Time

```
def eraseOverlapIntervals(self, intervals: List[List[int]]) -> int:
    if not intervals:
        return 0

    intervals.sort(key=lambda x: x[1])
    count = 0
    end = intervals[0][1]

    for i in range(1, len(intervals)):
        if intervals[i][0] < end:
            count += 1
        else:
            end = intervals[i][1]

    return count
```

Time Complexity: O(n log n) | Space Complexity: O(1)
Idea: Keep interval with earliest end to maximize remaining.

Solution 2: Greedy by Start Time

```
def eraseOverlapIntervals(self, intervals: List[List[int]]) -> int:
    intervals.sort(key=lambda x: x[0])
    count = 0
    prev_end = intervals[0][1]

    for start, end in intervals[1:]:
        if start >= prev_end:
            prev_end = end
        else:
            count += 1
            prev_end = min(prev_end, end)

    return count
```

Time Complexity: O(n log n) | Space Complexity: O(1)
Idea: When overlapping, keep the one ending earlier.

Solution 3: DP (Longest Non-overlapping Chain)

```
def eraseOverlapIntervals(self, intervals: List[List[int]]) -> int:
    intervals.sort()
    n = len(intervals)
    dp = [1] * n  # Longest non-overlapping chain ending at i

    for i in range(n):
        for j in range(i):
            if intervals[j][1] <= intervals[i][0]:
                dp[i] = max(dp[i], dp[j] + 1)

    return n - max(dp) if intervals else 0
```

Time Complexity: O(n^2) | Space Complexity: O(n)
Idea: Find longest chain of non-overlapping intervals.

452. Minimum Number of Arrows to Burst Balloons

There are some spherical balloons taped onto a flat wall that represents the XY-plane. The balloons are represented as a 2D integer array points where points[i] = [x_{start}, x_{end}] denotes a balloon whose **horizontal diameter** stretches between x_{start} and x_{end}. You do not know the exact y-coordinates of the balloons.

Arrows can be shot up **directly vertically** (in the positive y-direction) from different points along the x-axis. A balloon with x_{start} and x_{end} is **burst** by an arrow shot at x if x_{start} <= x <= x_{end}. There is **no limit** to the number of arrows that can be shot. A shot arrow keeps traveling up infinitely, bursting any balloons in its path.

Given the array points, return *the **minimum** number of arrows that must be shot to burst all balloons*.

Example 1:

Input: points = [[10,16],[2,8],[1,6],[7,12]]
Output: 2
Explanation: The balloons can be burst by 2 arrows:
- Shoot an arrow at x = 6, bursting the balloons [2,8] and [1,6].
- Shoot an arrow at x = 11, bursting the balloons [10,16] and [7,12].

Example 2:

Input: points = [[1,2],[3,4],[5,6],[7,8]]
Output: 4
Explanation: One arrow needs to be shot for each balloon for a total of 4 arrows.

Example 3:

Input: points = [[1,2],[2,3],[3,4],[4,5]]
Output: 2
Explanation: The balloons can be burst by 2 arrows:
- Shoot an arrow at x = 2, bursting the balloons [1,2] and [2,3].
- Shoot an arrow at x = 4, bursting the balloons [3,4] and [4,5].

Solution 1: Greedy by End

```
def findMinArrowShots(self, points: List[List[int]]) -> int:
    if not points:
        return 0

    points.sort(key=lambda x: x[1])
    arrows = 1
    end = points[0][1]

    for x_start, x_end in points[1:]:
        if x_start > end:
            arrows += 1
            end = x_end

    return arrows
```

Time Complexity: O(n log n) | Space Complexity: O(1)
Idea: Shoot arrow at end of current balloon to burst maximum.

Solution 2: Greedy by Start

```
def findMinArrowShots(self, points: List[List[int]]) -> int:
    points.sort()
    arrows = 0
    i = 0
    n = len(points)

    while i < n:
        # Current arrow covers up to current end
        arrow_end = points[i][1]
        arrows += 1

        # Burst all balloons that can be burst
        while i < n and points[i][0] <= arrow_end:
            arrow_end = min(arrow_end, points[i][1])
            i += 1

    return arrows
```

Time Complexity: O(n log n) | Space Complexity: O(1)
Idea: Shoot arrow and burst all overlapping balloons.

986. Interval List Intersections

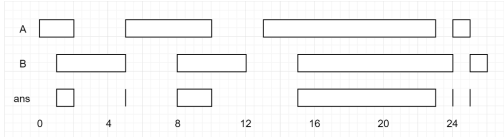
You are given two lists of closed intervals, firstList and secondList, where firstList[i] = [$start_i$, end_i] and secondList[j] = [$start_j$, end_j]. Each list of intervals is pairwise **disjoint** and in **sorted order**.

Return *the intersection of these two interval lists*.

A **closed interval** [a, b] (with a <= b) denotes the set of real numbers x with a <= x <= b.

The **intersection** of two closed intervals is a set of real numbers that are either empty or represented as a closed interval. For example, the intersection of [1, 3] and [2, 4] is [2, 3].

Example 1:



Input: firstList = [[0,2],[5,10],[13,23],[24,25]], secondList = [[1,5],[8,12],[15,24],[25,26]]
Output: [[1,2],[5,5],[8,10],[15,23],[24,24],[25,25]]

Example 2:

Input: firstList = [[1,3],[5,9]], secondList = []
Output: []

Solution 1: Two Pointers

```
def intervalIntersection(self, firstList: List[List[int]], secondList: List[List[int]]) -> List[List[int]]:
    i = j = 0
    result = []

    while i < len(firstList) and j < len(secondList):
        start = max(firstList[i][0], secondList[j][0])
        end = min(firstList[i][1], secondList[j][1])

        if start <= end:
            result.append([start, end])

        # Move pointer of interval that ends earlier
        if firstList[i][1] < secondList[j][1]:
            i += 1
        else:
            j += 1

    return result
```

Time Complexity: O(m + n) | Space Complexity: O(1)
Idea: Compare current intervals, move pointer of ending earlier.

Solution 2: Merged Timeline

```
def intervalIntersection(self, firstList: List[List[int]], secondList: List[List[int]]) -> List[List[int]]:
    events = []
    for start, end in firstList:
        events.append((start, 0, 1))  # (time, type, list_id)
        events.append((end, 1, 1))
    for start, end in secondList:
        events.append((start, 0, 2))
        events.append((end, 1, 2))

    events.sort()
    active = set()
    result = []
    start_intersection = None

    for time, typ, list_id in events:
        if typ == 0:  # Start
            if len(active) == 1 and list_id not in active:
                start_intersection = time
            active.add(list_id)
        else:  # End
            if len(active) == 2:
                result.append([start_intersection, time])
            active.remove(list_id)

    return result
```

Time Complexity: O((m+n) log(m+n)) | Space Complexity: O(m+n)
Idea: Track active intervals to find intersections.

759. Employee Free Time

We are given a list schedule of employees, which represents the working time for each employee.

Each employee has a list of non-overlapping Intervals, and these intervals are in sorted order.

Return the list of finite intervals representing **common, positive-length free time** for *all* employees, also in sorted order.

Example 1:

Input: schedule = [[[1,2],[5,6]],[[1,3]],[[4,10]]]
Output: [[3,4]]
Explanation:
There are a total of three employees, and all common free time intervals would be [-inf, 1], [3, 4], [10, inf].
We discard any intervals that contain inf as they aren't finite.

Example 2:

Input: schedule = [[[1,3],[6,7]],[[2,4]],[[2,5],[9,12]]]
Output: [[5,6],[7,9]]

(Even though we are representing Intervals in the form [x, y], the objects inside are Intervals, not lists or arrays. For example, schedule[0][0].start = 1, schedule[0][0].end = 2, and schedule[0][0][0] is not defined.)

Also, we wouldn't include intervals like [5, 5] in our answer, as they have zero length.

Solution 1: Merge + Scan

```
def employeeFreeTime(self, schedule: '[[Interval]]') -> '[Interval]':
    # Flatten and sort all intervals
    intervals = []
    for employee in schedule:
        for interval in employee:
            intervals.append([interval.start, interval.end])

    intervals.sort(key=lambda x: x[0])

    # Merge intervals
    merged = [intervals[0]]
    for start, end in intervals[1:]:
        if start <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], end)
        else:
            merged.append([start, end])
```

```
# Find gaps
    result = []
    for i in range(1, len(merged)):
        if merged[i][0] > merged[i-1][1]:
            result.append(Interval(merged[i-1][1], merged[i][0]))

    return result
```

Time Complexity: O(n log n) | Space Complexity: O(n)
Idea: Merge all intervals, find gaps between merged intervals.

Solution 2: Priority Queue

```
def employeeFreeTime(self, schedule: '[[Interval]]') -> '[Interval]':
    import heapq

    # Min heap of (start, employee_index, interval_index)
    heap = []
    for i, employee in enumerate(schedule):
        if employee:
            heapq.heappush(heap, (employee[0].start, i, 0))

    result = []
    prev_end = schedule[heap[0][1]][heap[0][2]].start

    while heap:
        start, emp_idx, int_idx = heapq.heappop(heap)
        interval = schedule[emp_idx][int_idx]

        if start > prev_end:
            result.append(Interval(prev_end, start))

        prev_end = max(prev_end, interval.end)

        if int_idx + 1 < len(schedule[emp_idx]):
            next_interval = schedule[emp_idx][int_idx + 1]
            heapq.heappush(heap, (next_interval.start, emp_idx, int_idx + 1))

    return result
```

Time Complexity: O(n log k) | Space Complexity: O(k)
Idea: Use min heap to process intervals in order.

228. Summary Ranges

You are given a **sorted unique** integer array nums.

A **range** [a,b] is the set of all integers from a to b (inclusive).

Return *the **smallest sorted** list of ranges that **cover all the numbers in the array exactly***. That is, each element of nums is covered by exactly one of the ranges, and there is no integer x such that x is in one of the ranges but not in nums.

Each range [a,b] in the list should be output as:

- "a->b" if a != b
- "a" if a == b

Example 1:

Input: nums = [0,1,2,4,5,7]
Output: ["0->2","4->5","7"]
Explanation: The ranges are:
[0,2] --> "0->2"
[4,5] --> "4->5"
[7,7] --> "7"

Example 2:

Input: nums = [0,2,3,4,6,8,9]
Output: ["0","2->4","6","8->9"]
Explanation: The ranges are:
[0,0] --> "0"
[2,4] --> "2->4"
[6,6] --> "6"
[8,9] --> "8->9"

Solution 1: Two Pointers

```
def summaryRanges(self, nums: List[int]) -> List[str]:
    if not nums:
        return []

    result = []
    start = nums[0]

    for i in range(1, len(nums)):
        if nums[i] != nums[i-1] + 1:
            if start == nums[i-1]:
                result.append(str(start))
            else:
                result.append(f"{start}->{nums[i-1]}")
            start = nums[i]

    # Handle last range
    if start == nums[-1]:
        result.append(str(start))
    else:
        result.append(f"{start}->{nums[-1]}")

    return result
```

Time Complexity: O(n) | Space Complexity: O(1)
Idea: Track start of current range, end when gap found.

Solution 2: While Loop

```
def summaryRanges(self, nums: List[int]) -> List[str]:
    result = []
    i = 0

    while i < len(nums):
        start = nums[i]

        # Find end of current range
        while i < len(nums) - 1 and nums[i+1] == nums[i] + 1:
            i += 1

        if start == nums[i]:
            result.append(str(start))
        else:
            result.append(f"{start}->{nums[i]}")

        i += 1

    return result
```

Time Complexity: O(n) | Space Complexity: O(1)
Idea: Use while to find consecutive sequences.

715. Range Module

A Range Module is a module that tracks ranges of numbers. Design a data structure to track the ranges represented as **half-open intervals** and query about them.

A **half-open interval** [left, right) denotes all the real numbers x where left <= x < right.

Implement the RangeModule class:

- RangeModule() Initializes the object of the data structure.
- void addRange(int left, int right) Adds the **half-open interval** [left, right), tracking every real number in that interval. Adding an interval that partially overlaps with currently tracked numbers should add any numbers in the interval [left, right) that are not already tracked.
- boolean queryRange(int left, int right) Returns true if every real number in the interval [left, right) is currently being tracked, and false otherwise.
- void removeRange(int left, int right) Stops tracking every real number currently being tracked in the **half-open interval** [left, right).

Example 1:

Input
["RangeModule", "addRange", "removeRange", "queryRange", "queryRange", "queryRange"]
[[], [10, 20], [14, 16], [10, 14], [13, 15], [16, 17]]
Output
[null, null, null, true, false, true]

Explanation
RangeModule rangeModule = new RangeModule();
rangeModule.addRange(10, 20);
rangeModule.removeRange(14, 16);
rangeModule.queryRange(10, 14); // return True,(Every number in [10, 14) is being tracked)
rangeModule.queryRange(13, 15); // return False,(Numbers like 14, 14.03, 14.17 in [13, 15) are not being tracked)
rangeModule.queryRange(16, 17); // return True, (The number 16 in [16, 17) is still being tracked, despite the remove operation)

Solution 1: Sorted List of Intervals

```
class RangeModule:
    def __init__(self):
        self.ranges = []  # Sorted list of non-overlapping intervals

    def addRange(self, left: int, right: int) -> None:
        new_range = [left, right]
        merged = []
        inserted = False

        for start, end in self.ranges:
            if end < new_range[0]:
                # Before new range
                merged.append([start, end])
            elif start > new_range[1]:
                # After new range
                if not inserted:
                    merged.append(new_range)
                    inserted = True
                merged.append([start, end])
            else:
                # Overlapping - merge
                new_range[0] = min(new_range[0], start)
                new_range[1] = max(new_range[1], end)

        if not inserted:
            merged.append(new_range)

        self.ranges = merged

    def queryRange(self, left: int, right: int) -> bool:
        # Binary search to find potential interval
        l, r = 0, len(self.ranges) - 1

        while l <= r:
            mid = (l + r) // 2
            start, end = self.ranges[mid]

            if left >= start and right <= end:
                return True
            elif right <= start:
                r = mid - 1
            else:
                l = mid + 1

        return False

    def removeRange(self, left: int, right: int) -> None:
        new_ranges = []

        for start, end in self.ranges:
            if end <= left or start >= right:
                # No overlap
                new_ranges.append([start, end])
            else:
                # Overlap - split
                if start < left:
                    new_ranges.append([start, left])
                if end > right:
                    new_ranges.append([right, end])

        self.ranges = new_ranges
```

Time Complexity:
- addRange: O(n)
- queryRange: O(log n)
- removeRange: O(n)

Space Complexity: O(n)
Idea: Maintain sorted non-overlapping intervals with binary search for queries.

Solution 2: TreeMap of Intervals

```
class RangeModule:
    def __init__(self):
        self.intervals = SortedDict()  # start -> end

    def addRange(self, left: int, right: int) -> None:
        # Find intervals that overlap
        to_remove = []

        # Find first interval that could overlap
        start_idx = self.intervals.bisect_right(left) - 1

        for start in list(self.intervals.keys())[start_idx:]:
            end = self.intervals[start]

            if start > right:
                break

            if end >= left:
                left = min(left, start)
                right = max(right, end)
                to_remove.append(start)

        # Remove overlapped intervals
        for start in to_remove:
            del self.intervals[start]

        self.intervals[left] = right

    def queryRange(self, left: int, right: int) -> bool:
        # Find interval that could contain left
        idx = self.intervals.bisect_right(left) - 1

        if idx >= 0:
            start = self.intervals.keys()[idx]
            end = self.intervals[start]
            if left >= start and right <= end:
                return True

        return False

    def removeRange(self, left: int, right: int) -> None:
        # Find intervals that overlap
        to_remove = []
        new_intervals = []

        start_idx = self.intervals.bisect_right(left) - 1

        for start in list(self.intervals.keys())[start_idx:]:
            end = self.intervals[start]

            if start >= right:
                break

            if end > left:
                to_remove.append(start)

                if start < left:
                    new_intervals.append((start, left))
                if end > right:
                    new_intervals.append((right, end))

        # Remove and add new intervals
        for start in to_remove:
            del self.intervals[start]

        for start, end in new_intervals:
            self.intervals[start] = end
```

Time Complexity: O(k log n) where k is number of overlapping intervals | Space Complexity: O(n)
Idea: Use TreeMap for ordered interval management.

Solution 3: Segment Tree

```
class SegmentTreeNode:
    def __init__(self, start, end):
        self.start = start
        self.end = end
        self.covered = False
        self.left = None
        self.right = None

class RangeModule:
    def __init__(self):
        self.root = SegmentTreeNode(0, 109)

    def addRange(self, left: int, right: int) -> None:
        self._update(self.root, left, right - 1, True)

    def queryRange(self, left: int, right: int) -> bool:
        return self._query(self.root, left, right - 1)

    def removeRange(self, left: int, right: int) -> None:
        self._update(self.root, left, right - 1, False)

    def _update(self, node, l, r, covered):
        if node.start > r or node.end < l:
            return

        if l <= node.start and node.end <= r:
            node.covered = covered
            node.left = node.right = None
            return

        mid = (node.start + node.end) // 2

        if not node.left:
            node.left = SegmentTreeNode(node.start, mid)
            node.right = SegmentTreeNode(mid + 1, node.end)

        self._update(node.left, l, r, covered)
        self._update(node.right, l, r, covered)

        node.covered = node.left.covered and node.right.covered

    def _query(self, node, l, r):
        if node.start > r or node.end < l:
            return True

        if l <= node.start and node.end <= r:
            return node.covered

        if not node.left:
            return node.covered

        mid = (node.start + node.end) // 2
        if r <= mid:
            return self._query(node.left, l, r)
        elif l > mid:
            return self._query(node.right, l, r)
        else:
            return self._query(node.left, l, mid) and self._query(node.right, mid + 1, r)
```

Time Complexity: O(log M) where M is range size | Space Complexity: O(log M)
Idea: Use segment tree for efficient range operations.

These solutions cover all major interval-based problems with multiple approaches, demonstrating various techniques from greedy algorithms to advanced data structures.

GRAPHS

200. Number of Islands

Given an m x n 2D binary grid grid which represents a map of '1's (land) and '0's (water), return *the number of islands*.

An **island** is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the grid are all surrounded by water.

Example 1:

Input: grid = [
["1","1","1","1","0"],
["1","1","0","1","0"],
["1","1","0","0","0"],
["0","0","0","0","0"]
]
Output: 1

Example 2:

Input: grid = [
["1","1","0","0","0"],
["1","1","0","0","0"],
["0","0","1","0","0"],
["0","0","0","1","1"]
]
Output: 3

Solution 1: DFS

```
def numIslands(self, grid: List[List[str]]) -> int:
    def dfs(i, j):
        if i < 0 or i >= m or j < 0 or j >= n or grid[i][j] != '1':
            return

        grid[i][j] = '0'  # Mark as visited
        dfs(i+1, j)
        dfs(i-1, j)
        dfs(i, j+1)
        dfs(i, j-1)

    m, n = len(grid), len(grid[0])
    count = 0

    for i in range(m):
        for j in range(n):
            if grid[i][j] == '1':
                dfs(i, j)
                count += 1

    return count
```

Time Complexity: O(m×n) | Space Complexity: O(m×n) in worst case (recursion stack)
Idea: DFS to mark entire island as visited.

Solution 2: BFS

```
def numIslands(self, grid: List[List[str]]) -> int:
    def bfs(start_i, start_j):
        queue = deque([(start_i, start_j)])
        grid[start_i][start_j] = '0'

        while queue:
            i, j = queue.popleft()
            for di, dj in [(0,1), (1,0), (0,-1), (-1,0)]:
                ni, nj = i + di, j + dj
                if 0 <= ni < m and 0 <= nj < n and grid[ni][nj] == '1':
                    grid[ni][nj] = '0'
                    queue.append((ni, nj))

    m, n = len(grid), len(grid[0])
    count = 0

    for i in range(m):
        for j in range(n):
            if grid[i][j] == '1':
                bfs(i, j)
                count += 1

    return count
```

Time Complexity: O(m×n) | Space Complexity: O(min(m,n)) for queue
Idea: BFS to explore island level by level.

Solution 3: Union Find

```
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n
        self.count = 0

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, x, y):
        px, py = self.find(x), self.find(y)
        if px == py:
            return

        if self.rank[px] < self.rank[py]:
            self.parent[px] = py
        elif self.rank[px] > self.rank[py]:
            self.parent[py] = px
        else:
            self.parent[py] = px
            self.rank[px] += 1
        self.count -= 1

def numIslands(self, grid: List[List[str]]) -> int:
    m, n = len(grid), len(grid[0])
    uf = UnionFind(m * n)

    # Initialize count
    for i in range(m):
        for j in range(n):
            if grid[i][j] == '1':
                uf.count += 1

    # Union adjacent 1's
    for i in range(m):
        for j in range(n):
            if grid[i][j] == '1':
                idx = i * n + j
                for di, dj in [(0,1), (1,0)]:  # Only check right and down
                    ni, nj = i + di, j + dj
                    if ni < m and nj < n and grid[ni][nj] == '1':
                        uf.union(idx, ni * n + nj)

    return uf.count
```

Time Complexity: O(m×n × α(mn)) | Space Complexity: O(m×n)
Idea: Union adjacent 1's, count distinct sets.

133. Clone Graph

Given a reference of a node in a **connected** undirected graph.

Return a **deep copy** (clone) of the graph.

Each node in the graph contains a value (int) and a list (List[Node]) of its neighbors.

```
class Node {
    public int val;
    public List<Node> neighbors;
}
```

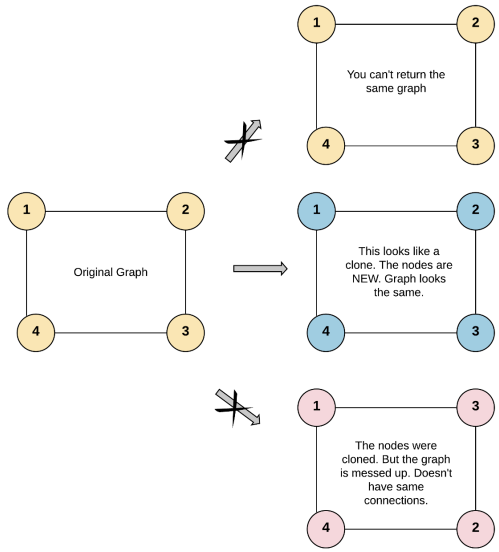
Test case format:

For simplicity, each node's value is the same as the node's index (1-indexed). For example, the first node with val == 1, the second node with val == 2, and so on. The graph is represented in the test case using an adjacency list.

An adjacency list is a collection of unordered **lists** used to represent a finite graph. Each list describes the set of neighbors of a node in the graph.

The given node will always be the first node with val = 1. You must return the **copy of the given node** as a reference to the cloned graph.

Example 1:



Input: adjList = [[2,4],[1,3],[2,4],[1,3]]
Output: [[2,4],[1,3],[2,4],[1,3]]
Explanation: There are 4 nodes in the graph.
1st node (val = 1)'s neighbors are 2nd node (val = 2) and 4th node (val = 4).

2nd node (val = 2)'s neighbors are 1st node (val = 1) and 3rd node (val = 3).
3rd node (val = 3)'s neighbors are 2nd node (val = 2) and 4th node (val = 4).
4th node (val = 4)'s neighbors are 1st node (val = 1) and 3rd node (val = 3).

Example 2:

Input: adjList = [[]]
Output: [[]]
Explanation: Note that the input contains one empty list. The graph consists of only one node with val = 1 and it does not have any neighbors.

Example 3:

Input: adjList = []
Output: []
Explanation: This an empty graph, it does not have any nodes.

Solution 1: DFS with HashMap

```
def cloneGraph(self, node: 'Node') -> 'Node':
    if not node:
        return None

    cloned = {}

    def dfs(node):
        if node in cloned:
            return cloned[node]

        copy = Node(node.val)
        cloned[node] = copy

        for neighbor in node.neighbors:
            copy.neighbors.append(dfs(neighbor))

        return copy

    return dfs(node)
```

Time Complexity: O(V + E) | Space Complexity: O(V)
Idea: DFS with memoization to avoid cycles.

Solution 2: BFS with Queue

```
def cloneGraph(self, node: 'Node') -> 'Node':
    if not node:
        return None

    cloned = {node: Node(node.val)}
    queue = deque([node])

    while queue:
        curr = queue.popleft()

        for neighbor in curr.neighbors:
            if neighbor not in cloned:
                cloned[neighbor] = Node(neighbor.val)
                queue.append(neighbor)
            cloned[curr].neighbors.append(cloned[neighbor])

    return cloned[node]
```

Time Complexity: O(V + E) | Space Complexity: O(V)
Idea: BFS to clone level by level.

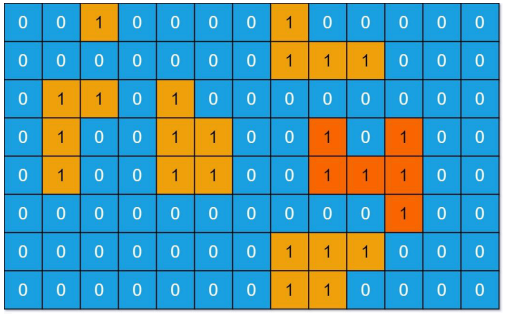
695. Max Area of Island

You are given an m x n binary matrix grid. An island is a group of 1's (representing land) connected **4-directionally** (horizontal or vertical.) You may assume all four edges of the grid are surrounded by water.

The **area** of an island is the number of cells with a value 1 in the island.

Return *the maximum **area** of an island in* grid. If there is no island, return 0.

Example 1:



0	0	1	0	0	0	0	1	0	0	0	0	0
0	0	0	0	0	0	0	1	1	1	0	0	0
0	1	1	0	1	0	0	0	0	0	0	0	0
0	1	0	0	1	1	0	0	1	0	1	0	0
0	1	0	0	1	1	0	0	1	1	1	0	0
0	0	0	0	0	0	0	0	0	0	1	0	0
0	0	0	0	0	0	0	1	1	1	0	0	0
0	0	0	0	0	0	0	1	1	0	0	0	0

Input: grid = [[0,0,1,0,0,0,0,1,0,0,0,0,0],[0,0,0,0,0,0,0,1,1,1,0,0,0],[0,1,1,0,1,0,0,0,0,0,0,0,0],[0,1,0,0,1,1,0,0,1,0,1,0,0],[0,1,0,0,1,1,0,0,1,1,1,0,0],[0,0,0,0,0,0,0,0,0,0,1,0,0],[0,0,0,0,0,0,0,1,1,1,0,0,0],[0,0,0,0,0,0,0,1,1,0,0,0,0]]
Output: 6
Explanation: The answer is not 11, because the island must be connected 4-directionally.

Example 2:

Input: grid = [[0,0,0,0,0,0,0,0]]
Output: 0

Solution 1: DFS

```
def maxAreaOfIsland(self, grid: List[List[int]]) -> int:
    def dfs(i, j):
        if i < 0 or i >= m or j < 0 or j >= n or grid[i][j] == 0:
            return 0

        grid[i][j] = 0
        area = 1
        for di, dj in [(0,1), (1,0), (0,-1), (-1,0)]:
            area += dfs(i + di, j + dj)
        return area

    m, n = len(grid), len(grid[0])
    max_area = 0

    for i in range(m):
        for j in range(n):
            if grid[i][j] == 1:
                max_area = max(max_area, dfs(i, j))

    return max_area
```

Time Complexity: O(m×n) | Space Complexity: O(m×n)
Idea: DFS and return area of each island.

Solution 2: BFS

```
def maxAreaOfIsland(self, grid: List[List[int]]) -> int:
    def bfs(start_i, start_j):
        queue = deque([(start_i, start_j)])
        grid[start_i][start_j] = 0
        area = 1

        while queue:
            i, j = queue.popleft()
            for di, dj in [(0,1), (1,0), (0,-1), (-1,0)]:
                ni, nj = i + di, j + dj
                if 0 <= ni < m and 0 <= nj < n and grid[ni][nj] == 1:
                    grid[ni][nj] = 0
                    area += 1
                    queue.append((ni, nj))
        return area

    m, n = len(grid), len(grid[0])
    max_area = 0

    for i in range(m):
        for j in range(n):
            if grid[i][j] == 1:
                max_area = max(max_area, bfs(i, j))

    return max_area
```

Time Complexity: O(m×n) | Space Complexity: O(min(m,n))
Idea: BFS to compute area.

417. Pacific Atlantic Water Flow

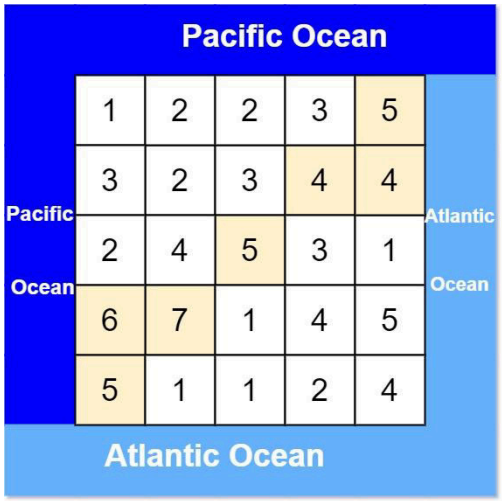
There is an m x n rectangular island that borders both the **Pacific Ocean** and **Atlantic Ocean**. The **Pacific Ocean** touches the island's left and top edges, and the **Atlantic Ocean** touches the island's right and bottom edges.

The island is partitioned into a grid of square cells. You are given an m x n integer matrix heights where heights[r][c] represents the **height above sea level** of the cell at coordinate (r, c).

The island receives a lot of rain, and the rain water can flow to neighboring cells directly north, south, east, and west if the neighboring cell's height is **less than or equal to** the current cell's height. Water can flow from any cell adjacent to an ocean into the ocean.

Return *a **2D list** of grid coordinates* result *where* result[i] = [r_i, c_i] *denotes that rain water can flow from cell* (r_i, c_i) *to **both** the Pacific and Atlantic oceans.*

Example 1:



Input: heights = [[1,2,2,3,5],[3,2,3,4,4],[2,4,5,3,1],[6,7,1,4,5],[5,1,1,2,4]]
Output: [[0,4],[1,3],[1,4],[2,2],[3,0],[3,1],[4,0]]
Explanation: The following cells can flow to the Pacific and Atlantic oceans, as shown below:
[0,4]: [0,4] -> Pacific Ocean
[0,4] -> Atlantic Ocean
[1,3]: [1,3] -> [0,3] -> Pacific Ocean
[1,3] -> [1,4] -> Atlantic Ocean
[1,4]: [1,4] -> [1,3] -> [0,3] -> Pacific Ocean
[1,4] -> Atlantic Ocean
[2,2]: [2,2] -> [1,2] -> [0,2] -> Pacific Ocean
[2,2] -> [2,3] -> [2,4] -> Atlantic Ocean
[3,0]: [3,0] -> Pacific Ocean
[3,0] -> [4,0] -> Atlantic Ocean
[3,1]: [3,1] -> [3,0] -> Pacific Ocean
[3,1] -> [4,1] -> Atlantic Ocean
[4,0]: [4,0] -> Pacific Ocean
[4,0] -> Atlantic Ocean
Note that there are other possible paths for these cells to flow to the Pacific and Atlantic oceans.

Example 2:

Input: heights = [[1]]
Output: [[0,0]]
Explanation: The water can flow from the only cell to the Pacific and Atlantic oceans.

Solution 1: DFS from Borders

```
def pacificAtlantic(self, heights: List[List[int]]) -> List[List[int]]:
    def dfs(i, j, visited, prev_height):
        if (i, j) in visited or i < 0 or i >= m or j < 0 or j >= n or heights[i][j] < prev_height:
            return

        visited.add((i, j))
        for di, dj in [(0,1), (1,0), (0,-1), (-1,0)]:
            dfs(i + di, j + dj, visited, heights[i][j])

    m, n = len(heights), len(heights[0])
    pacific = set()
    atlantic = set()

    # Top and bottom borders
    for j in range(n):
        dfs(0, j, pacific, heights[0][j])
        dfs(m-1, j, atlantic, heights[m-1][j])

    # Left and right borders
    for i in range(m):
        dfs(i, 0, pacific, heights[i][0])
        dfs(i, n-1, atlantic, heights[i][n-1])

    return list(pacific & atlantic)
```

Time Complexity: O(m×n) | Space Complexity: O(m×n)
Idea: DFS from borders, find intersection.

Solution 2: BFS from Borders

```
def pacificAtlantic(self, heights: List[List[int]]) -> List[List[int]]:
    def bfs(starts):
        queue = deque(starts)
        visited = set(starts)

        while queue:
            i, j = queue.popleft()
            for di, dj in [(0,1), (1,0), (0,-1), (-1,0)]:
                ni, nj = i + di, j + dj
                if 0 <= ni < m and 0 <= nj < n and (ni, nj) not in visited and heights[ni][nj] >= heights[i][j]:
                    visited.add((ni, nj))
                    queue.append((ni, nj))
        return visited

    m, n = len(heights), len(heights[0])

    # Collect border cells
    pacific_borders = [(0, j) for j in range(n)] + [(i, 0) for i in range(m)]
    atlantic_borders = [(m-1, j) for j in range(n)] + [(i, n-1) for i in range(m)]

    pacific = bfs(pacific_borders)
    atlantic = bfs(atlantic_borders)

    return list(pacific & atlantic)
```

Time Complexity: O(m×n) | Space Complexity: O(m×n)
Idea: BFS from both oceans.

994. Rotting Oranges

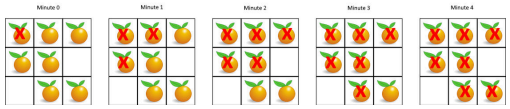
You are given an m x n grid where each cell can have one of three values:

- 0 representing an empty cell,
- 1 representing a fresh orange, or
- 2 representing a rotten orange.

Every minute, any fresh orange that is **4-directionally adjacent** to a rotten orange becomes rotten.

Return *the minimum number of minutes that must elapse until no cell has a fresh orange.* If *this is impossible, return* -1.

Example 1:



Input: grid = [[2,1,1],[1,1,0],[0,1,1]]
Output: 4

Example 2:

Input: grid = [[2,1,1],[0,1,1],[1,0,1]]
Output: -1
Explanation: The orange in the bottom left corner (row 2, column 0) is never rotten, because rotting only happens 4-directionally.

Example 3:

Input: grid = [[0,2]]
Output: 0
Explanation: Since there are already no fresh oranges at minute 0, the answer is just 0.

Solution 1: BFS Multisource

```
def orangesRotting(self, grid: List[List[int]]) -> int:
    m, n = len(grid), len(grid[0])
    queue = deque()
    fresh = 0

    # Find all rotten oranges and count fresh
    for i in range(m):
        for j in range(n):
            if grid[i][j] == 2:
                queue.append((i, j, 0))
            elif grid[i][j] == 1:
                fresh += 1
```

```
    if fresh == 0:
        return 0

    time = 0
    while queue:
        i, j, t = queue.popleft()
        time = max(time, t)

        for di, dj in [(0,1), (1,0), (0,-1), (-1,0)]:
            ni, nj = i + di, j + dj
            if 0 <= ni < m and 0 <= nj < n and grid[ni][nj] == 1:
                grid[ni][nj] = 2
                fresh -= 1
                queue.append((ni, nj, t + 1))

    return time if fresh == 0 else -1
```

Time Complexity: O(m×n) | Space Complexity: O(m×n)
Idea: BFS from all rotten oranges simultaneously.

Solution 2: In-place BFS

```
def orangesRotting(self, grid: List[List[int]]) -> int:
    m, n = len(grid), len(grid[0])
    queue = deque()

    for i in range(m):
        for j in range(n):
            if grid[i][j] == 2:
                queue.append((i, j))

    time = 0
    while queue:
        size = len(queue)
        rotted = False

        for _ in range(size):
            i, j = queue.popleft()
            for di, dj in [(0,1), (1,0), (0,-1), (-1,0)]:
                ni, nj = i + di, j + dj
                if 0 <= ni < m and 0 <= nj < n and grid[ni][nj] == 1:
                    grid[ni][nj] = 2
                    queue.append((ni, nj))
                    rotted = True

        if rotted:
            time += 1

    # Check for fresh oranges
    for i in range(m):
        for j in range(n):
            if grid[i][j] == 1:
                return -1

    return time
```

Time Complexity: O(m×n) | Space Complexity: O(m×n)
Idea: Level-by-level BFS counting time.

207. Course Schedule

There are a total of numCourses courses you have to take, labeled from 0 to numCourses - 1. You are given an array prerequisites where prerequisites[i] = [a_i, b_i] indicates that you **must** take course b_i first if you want to take course a_i.

- For example, the pair [0, 1], indicates that to take course 0 you have to first take course 1.

Return true if you can finish all courses. Otherwise, return false.

Example 1:

Input: numCourses = 2, prerequisites = [[1,0]]
Output: true
Explanation: There are a total of 2 courses to take.
To take course 1 you should have finished course 0. So it is possible.

Example 2:

Input: numCourses = 2, prerequisites = [[1,0],[0,1]]
Output: false
Explanation: There are a total of 2 courses to take.
To take course 1 you should have finished course 0, and to take course 0 you should also have finished course 1. So it is impossible.

Solution 1: DFS Cycle Detection

```
def canFinish(self, numCourses: int, prerequisites: List[List[int]]) -> bool:
    def has_cycle(course):
        if visited[course] == 1:  # Currently visiting
            return True
        if visited[course] == 2:  # Already processed
            return False

        visited[course] = 1
        for neighbor in graph[course]:
            if has_cycle(neighbor):
                return True
        visited[course] = 2
        return False

    # Build graph
    graph = [[] for _ in range(numCourses)]
    for course, prereq in prerequisites:
        graph[prereq].append(course)

    visited = [0] * numCourses  # 0: unvisited, 1: visiting, 2: visited

    for i in range(numCourses):
        if has_cycle(i):
            return False
    return True
```

Time Complexity: O(V + E) | Space Complexity: O(V + E)
Idea: Detect cycle using DFS with 3-state visited array.

Solution 2: BFS (Topological Sort - Kahn's Algorithm)

```
def canFinish(self, numCourses: int, prerequisites: List[List[int]]) -> bool:
    # Build graph and indegree array
    graph = [[] for _ in range(numCourses)]
    indegree = [0] * numCourses

    for course, prereq in prerequisites:
        graph[prereq].append(course)
        indegree[course] += 1

    # Queue for courses with no prerequisites
    queue = deque([i for i in range(numCourses) if indegree[i] == 0])
    count = 0

    while queue:
        course = queue.popleft()
        count += 1

        for neighbor in graph[course]:
            indegree[neighbor] -= 1
            if indegree[neighbor] == 0:
                queue.append(neighbor)

    return count == numCourses
```

Time Complexity: O(V + E) | Space Complexity: O(V + E)
Idea: Remove nodes with indegree 0 until none left.

210. Course Schedule II

There are a total of numCourses courses you have to take, labeled from 0 to numCourses - 1. You are given an array prerequisites where prerequisites[i] = [a_i, b_i] indicates that you **must** take course b_i first if you want to take course a_i.

- For example, the pair [0, 1], indicates that to take course 0 you have to first take course 1.

Return *the ordering of courses you should take to finish all courses*. If there are many valid answers, return **any** of them. If it is impossible to finish all courses, return **an empty array**.

Example 1:

Input: numCourses = 2, prerequisites = [[1,0]]
Output: [0,1]
Explanation: There are a total of 2 courses to take. To take course 1 you should have finished course 0. So the correct course order is [0,1].

Example 2:

Input: numCourses = 4, prerequisites = [[1,0],[2,0],[3,1],[3,2]]
Output: [0,2,1,3]
Explanation: There are a total of 4 courses to take. To take course 3 you should have finished both courses 1 and 2. Both courses 1 and 2 should be taken after you finished course 0.
So one correct course order is [0,1,2,3]. Another correct ordering is [0,2,1,3].

Example 3:

Input: numCourses = 1, prerequisites = []
Output: [0]

Solution 1: DFS with Stack

```
def findOrder(self, numCourses: int, prerequisites: List[List[int]]) -> List[int]:
    def dfs(course):
        if visited[course] == 1:  # Cycle detected
            return False
        if visited[course] == 2:  # Already processed
            return True

        visited[course] = 1
        for neighbor in graph[course]:
            if not dfs(neighbor):
                return False
        visited[course] = 2
        result.append(course)
        return True
```

```
    # Build graph
    graph = [[] for _ in range(numCourses)]
    for course, prereq in prerequisites:
        graph[prereq].append(course)

    visited = [0] * numCourses
    result = []

    for i in range(numCourses):
        if not dfs(i):
            return []

    return result[::-1]  # Reverse for correct order
```

Time Complexity: O(V + E) | Space Complexity: O(V + E)
Idea: DFS with post-order traversal.

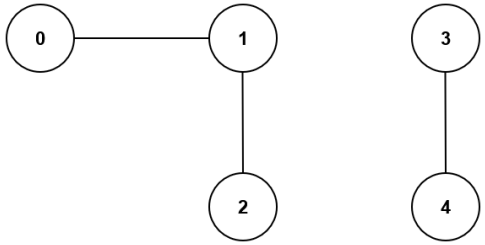
Solution 2: BFS (Kahn's Algorithm)

```
def findOrder(self, numCourses: int, prerequisites:
List[List[int]]) -> List[int]:
    # Build graph and indegree array
    graph = [[] for _ in range(numCourses)]
    indegree = [0] * numCourses

    for course, prereq in prerequisites:
        graph[prereq].append(course)
        indegree[course] += 1

    queue = deque([i for i in range(numCourses) if
indegree[i] == 0])
    result = []

    while queue:
        course = queue.popleft()
        result.append(course)

        for neighbor in graph[course]:
            indegree[neighbor] -= 1
            if indegree[neighbor] == 0:
                queue.append(neighbor)

    return result if len(result) == numCourses else []
```

Time Complexity: O(V + E) | Space Complexity: O(V + E)
Idea: Topological sort using indegree array.

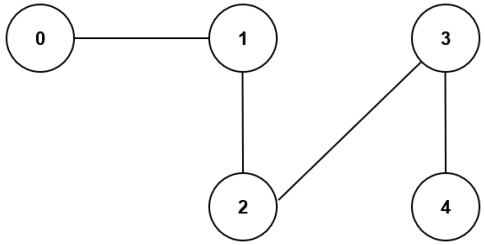
323. Number of Connected Components

You have a graph of n nodes. You are given an integer n and an array edges where edges[i] = [a_i, b_i] indicates that there is an edge between a_i and b_i in the graph. Return the number of connected components in the graph.
Example 1:



Input:
n = 5, edges = [[0,1],[1,2],[3,4]]

Output: 2
Example 2:



Input:
n = 5, edges = [[0,1],[1,2],[2,3],[3,4]]

Output: 1

Solution 1: DFS

```
def countComponents(self, n: int, edges: List[List[int]]) ->
int:
    def dfs(node):
        visited[node] = True
        for neighbor in graph[node]:
            if not visited[neighbor]:
                dfs(neighbor)

    # Build graph
    graph = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append(v)
        graph[v].append(u)

    visited = [False] * n
    count = 0

    for i in range(n):
        if not visited[i]:
            dfs(i)
            count += 1

    return count
```

Time Complexity: O(V + E) | Space Complexity: O(V + E)
Idea: DFS to mark each component.

Solution 2: Union Find

```
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n
        self.count = n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, x, y):
        px, py = self.find(x), self.find(y)
        if px == py:
            return

        if self.rank[px] < self.rank[py]:
            self.parent[px] = py
        elif self.rank[px] > self.rank[py]:
            self.parent[py] = px
        else:
            self.parent[py] = px
            self.rank[px] += 1
        self.count -= 1

def countComponents(self, n: int, edges: List[List[int]]) ->
int:
    uf = UnionFind(n)

    for u, v in edges:
        uf.union(u, v)

    return uf.count
```

Time Complexity: O((V+E) × α(V)) | Space Complexity: O(V)
Idea: Union connected nodes.

---

261. Graph Valid Tree

Given n nodes labeled from 0 to n - 1 and a list of undirected edges (each edge is a pair of nodes), write a function to check whether these edges make up a valid tree.
Example 1:
Input:
n = 5
edges = [[0, 1], [0, 2], [0, 3], [1, 4]]

Output: true
Example 2:
Input:
n = 5
edges = [[0, 1], [1, 2], [2, 3], [1, 3], [1, 4]]

Output: false

- You can assume that no duplicate edges will appear in edges. Since all edges are undirected, [0, 1] is the same as [1, 0] and thus will not appear together in edges.

Solution 1: DFS

```
def validTree(self, n: int, edges: List[List[int]]) -> bool:
    if len(edges) != n - 1:
        return False

    def dfs(node, parent):
        visited[node] = True
        for neighbor in graph[node]:
            if neighbor == parent:
                continue
            if visited[neighbor]:
                return False
            if not dfs(neighbor, node):
                return False
        return True

    # Build graph
    graph = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append(v)
        graph[v].append(u)

    visited = [False] * n

    if not dfs(0, -1):
        return False

    return all(visited)
```

Time Complexity: O(V + E) | Space Complexity: O(V + E)
Idea: DFS with parent tracking to detect cycles.

Solution 2: BFS

```
from collections import deque

def validTree(self, n: int, edges: List[List[int]]) -> bool:
    if len(edges) != n - 1:
        return False

    # Build graph
    graph = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append(v)
        graph[v].append(u)

    visited = [False] * n
    parent = [-1] * n
    queue = deque([0])
    visited[0] = True

    while queue:
        node = queue.popleft()
        for neighbor in graph[node]:
            if not visited[neighbor]:
                visited[neighbor] = True
                parent[neighbor] = node
                queue.append(neighbor)
            elif neighbor != parent[node]:
                return False

    return all(visited)
```

Time Complexity: O(V + E) | Space Complexity: O(V + E)
Idea: BFS with parent tracking.

841. Keys and Rooms

There are n rooms labeled from 0 to n - 1 and all the rooms are locked except for room 0. Your goal is to visit all the rooms. However, you cannot enter a locked room without having its key.

When you visit a room, you may find a set of **distinct keys** in it. Each key has a number on it, denoting which room it unlocks, and you can take all of them with you to unlock the other rooms.

Given an array rooms where rooms[i] is the set of keys that you can obtain if you visited room i, return true *if you can visit **all** the rooms, or* false *otherwise*.

Example 1:

Input: rooms = [[1],[2],[3],[]]
Output: true
Explanation:
We visit room 0 and pick up key 1.
We then visit room 1 and pick up key 2.
We then visit room 2 and pick up key 3.
We then visit room 3.
Since we were able to visit every room, we return true.

Example 2:

Input: rooms = [[1,3],[3,0,1],[2],[0]]
Output: false
Explanation: We can not enter room number 2 since the only key that unlocks it is in that room.

Solution 1: DFS

```
def canVisitAllRooms(self, rooms: List[List[int]]) -> bool:
    def dfs(room):
        visited[room] = True
        for key in rooms[room]:
            if not visited[key]:
                dfs(key)

    n = len(rooms)
    visited = [False] * n
    dfs(0)

    return all(visited)
```

Time Complexity: O(V + E) | Space Complexity: O(V)
Idea: DFS to explore all reachable rooms.

Solution 2: BFS

```
def canVisitAllRooms(self, rooms: List[List[int]]) -> bool:
    n = len(rooms)
    visited = [False] * n
    queue = deque([0])
    visited[0] = True

    while queue:
        room = queue.popleft()
        for key in rooms[room]:
            if not visited[key]:
                visited[key] = True
                queue.append(key)

    return all(visited)
```

Time Complexity: O(V + E) | Space Complexity: O(V)
Idea: BFS to collect keys level by level.

---

547. Number of Provinces

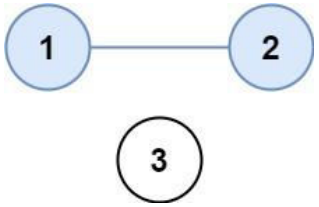
There are n cities. Some of them are connected, while some are not. If city a is connected directly with city b, and city b is connected directly with city c, then city a is connected indirectly with city c.

A **province** is a group of directly or indirectly connected cities and no other cities outside of the group.

You are given an n x n matrix isConnected where isConnected[i][j] = 1 if the i_{th} city and the j_{th} city are directly connected, and isConnected[i][j] = 0 otherwise.

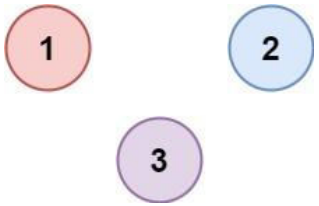
Return *the total number of **provinces***.

Example 1:



Input: isConnected = [[1,1,0],[1,1,0],[0,0,1]]
Output: 2

Example 2:



Input: isConnected = [[1,0,0],[0,1,0],[0,0,1]]
Output: 3

Solution 1: DFS

```
def findCircleNum(self, isConnected: List[List[int]]) -> int:
    def dfs(city):
        visited[city] = True
        for neighbor in range(n):
            if isConnected[city][neighbor] == 1 and not
visited[neighbor]:
                dfs(neighbor)

    n = len(isConnected)
    visited = [False] * n
    provinces = 0

    for city in range(n):
        if not visited[city]:
            dfs(city)
            provinces += 1

    return provinces
```

Time Complexity: O(n²) | Space Complexity: O(n)
Idea: DFS to mark entire province.

Solution 2: Union Find

```
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n
        self.count = n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, x, y):
        px, py = self.find(x), self.find(y)
        if px == py:
            return

        if self.rank[px] < self.rank[py]:
            self.parent[px] = py
        elif self.rank[px] > self.rank[py]:
            self.parent[py] = px
        else:
            self.parent[py] = px
            self.rank[px] += 1
        self.count -= 1
```

```
def findCircleNum(self, isConnected: List[List[int]]) -> int:
    n = len(isConnected)
    uf = UnionFind(n)

    for i in range(n):
        for j in range(i + 1, n):
            if isConnected[i][j] == 1:
                uf.union(i, j)

    return uf.count
```

Time Complexity: O(n² × α(n)) | Space Complexity: O(n)
Idea: Union connected cities.

1091. Shortest Path in Binary Matrix

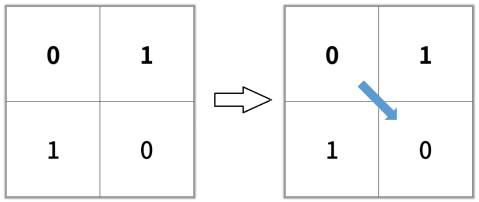
Given an n x n binary matrix grid, return *the length of the shortest **clear path** in the matrix*. If there is no clear path, return -1.

A **clear path** in a binary matrix is a path from the **top-left** cell (i.e., (0, 0)) to the **bottom-right** cell (i.e., (n - 1, n - 1)) such that:

- All the visited cells of the path are 0.
- All the adjacent cells of the path are **8-directionally** connected (i.e., they are different and they share an edge or a corner).

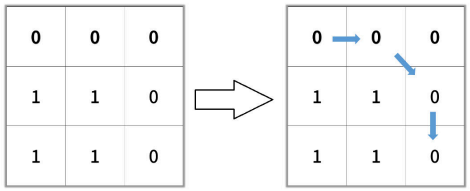
The **length of a clear path** is the number of visited cells of this path.

Example 1:



Input: grid = [[0,1],[1,0]]
Output: 2

Example 2:



Input: grid = [[0,0,0],[1,1,0],[1,1,0]]
Output: 4

Example 3:

Input: grid = [[1,0,0],[1,1,0],[1,1,0]]
Output: -1

Solution 1: BFS

```
def shortestPathBinaryMatrix(self, grid: List[List[int]]) -> int:
    if grid[0][0] == 1 or grid[-1][-1] == 1:
        return -1

    n = len(grid)
    directions = [(0,1), (1,0), (0,-1), (-1,0), (1,1), (-1,-1), (1,-1), (-1,1)]
    queue = deque([(0, 0, 1)])  # (i, j, distance)
    visited = set([(0, 0)])

    while queue:
        i, j, dist = queue.popleft()

        if i == n-1 and j == n-1:
            return dist

        for di, dj in directions:
            ni, nj = i + di, j + dj
            if 0 <= ni < n and 0 <= nj < n and grid[ni][nj] == 0 and (ni, nj) not in visited:
                visited.add((ni, nj))
                queue.append((ni, nj, dist + 1))

    return -1
```

Time Complexity: O(n²) | Space Complexity: O(n²)
Idea: BFS finds shortest path in unweighted graph.

Solution 2: A* Search (Heuristic)

```
def shortestPathBinaryMatrix(self, grid: List[List[int]]) -> int:
    if grid[0][0] == 1 or grid[-1][-1] == 1:
        return -1

    n = len(grid)
    directions = [(0,1), (1,0), (0,-1), (-1,0), (1,1), (-1,-1), (1,-1), (-1,1)]

    def heuristic(i, j):
        return max(n-1 - i, n-1 - j)  # Chebyshev distance

    # Priority queue: (estimated_total, distance, i, j)
    pq = [(heuristic(0, 0) + 1, 1, 0, 0)]
    dist = {(0, 0): 1}

    while pq:
        est, d, i, j = heapq.heappop(pq)

        if i == n-1 and j == n-1:
            return d

        for di, dj in directions:
            ni, nj = i + di, j + dj
            if 0 <= ni < n and 0 <= nj < n and grid[ni][nj] == 0:
                nd = d + 1
                if (ni, nj) not in dist or nd < dist[(ni, nj)]:
                    dist[(ni, nj)] = nd
                    priority = nd + heuristic(ni, nj)
                    heapq.heappush(pq, (priority, nd, ni, nj))

    return -1
```

Time Complexity: O(n² log n) | Space Complexity: O(n²)
Idea: A* with Chebyshev heuristic for faster search.

130. Surrounded Regions

You are given an m x n matrix board containing **letters** 'X' and 'O', **capture regions** that are **surrounded**:

- **Connect**: A cell is connected to adjacent cells horizontally or vertically.
- **Region**: To form a region **connect every** 'O' cell.
- **Surround**: A region is surrounded if none of the 'O' cells in that region are on the edge of the board. Such regions are **completely enclosed** by 'X' cells.

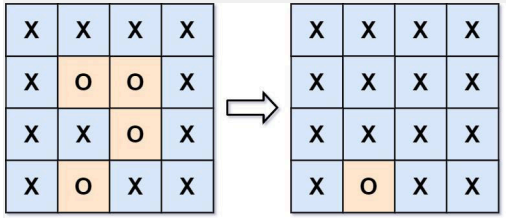
To capture a **surrounded region**, replace all 'O's with 'X's **in-place** within the original board. You do not need to return anything.

Example 1:

Input: board = [["X","X","X","X"],["X","O","O","X"],["X","X","O","X"],["X","O","X","X"]]

Output: [["X","X","X","X"],["X","X","X","X"],["X","X","X","X"],["X","O","X","X"]]

Explanation:



In the above diagram, the bottom region is not captured because it is on the edge of the board and cannot be surrounded.

Example 2:

Input: board = [["X"]]

Output: [["X"]]

Solution 1: DFS from Borders

```
def solve(self, board: List[List[str]]) -> None:
    def dfs(i, j):
        if i < 0 or i >= m or j < 0 or j >= n or board[i][j] != 'O':
            return

        board[i][j] = '#'  # Mark as safe
        for di, dj in [(0,1), (1,0), (0,-1), (-1,0)]:
            dfs(i + di, j + dj)

    m, n = len(board), len(board[0])

    # Mark border-connected 'O's
    for i in range(m):
        dfs(i, 0)
        dfs(i, n-1)

    for j in range(n):
        dfs(0, j)
        dfs(m-1, j)

    # Flip remaining 'O' to 'X' and '#' back to 'O'
    for i in range(m):
        for j in range(n):
            if board[i][j] == 'O':
                board[i][j] = 'X'
            elif board[i][j] == '#':
                board[i][j] = 'O'
```

Time Complexity: O(m×n) | Space Complexity: O(m×n)
Idea: Mark border-connected O's as safe, flip others.

Solution 2: BFS from Borders

```
def solve(self, board: List[List[str]]) -> None:
    m, n = len(board), len(board[0])
    queue = deque()

    # Add border O's to queue
    for i in range(m):
        if board[i][0] == 'O':
            queue.append((i, 0))
        if board[i][n-1] == 'O':
            queue.append((i, n-1))

    for j in range(n):
        if board[0][j] == 'O':
            queue.append((0, j))
        if board[m-1][j] == 'O':
            queue.append((m-1, j))

    # BFS to mark all border-connected O's
    while queue:
        i, j = queue.popleft()
        if 0 <= i < m and 0 <= j < n and board[i][j] == 'O':
            board[i][j] = '#'  # Mark as safe

            for di, dj in [(0,1), (1,0), (0,-1), (-1,0)]:
                queue.append((i + di, j + dj))

    # Flip remaining O's to X, and #'s back to O
    for i in range(m):
        for j in range(n):
            if board[i][j] == 'O':
                board[i][j] = 'X'
            elif board[i][j] == '#':
                board[i][j] = 'O'
```

Time Complexity: O(m×n) | Space Complexity: O(m×n)
Idea: BFS from border O's to mark safe regions.

DESIGN & SYSTEM DESIGN

146. LRU Cache

Design a data structure that follows the constraints of a **Least Recently Used (LRU) cache**.

Implement the LRUCache class:

- LRUCache(int capacity) Initialize the LRU cache with **positive** size capacity.
- int get(int key) Return the value of the key if the key exists, otherwise return -1.
- void put(int key, int value) Update the value of the key if the key exists. Otherwise, add the key-value pair to the cache. If the number of keys exceeds the capacity from this operation, **evict** the least recently used key.

The functions get and put must each run in O(1) average time complexity.

Example 1:

Input
["LRUCache", "put", "put", "get", "put", "get", "put", "get", "get", "get"]
[[2], [1, 1], [2, 2], [1], [3, 3], [2], [4, 4], [1], [3], [4]]
Output
[null, null, null, 1, null, -1, null, -1, 3, 4]

Explanation
LRUCache lRUCache = new LRUCache(2);
lRUCache.put(1, 1); // cache is {1=1}
lRUCache.put(2, 2); // cache is {1=1, 2=2}
lRUCache.get(1); // return 1
lRUCache.put(3, 3); // LRU key was 2, evicts key 2, cache is {1=1, 3=3}
lRUCache.get(2); // returns -1 (not found)
lRUCache.put(4, 4); // LRU key was 1, evicts key 1, cache is {4=4, 3=3}
lRUCache.get(1); // return -1 (not found)
lRUCache.get(3); // return 3
lRUCache.get(4); // return 4

Solution 1: OrderedDict (Python Built-in)

```
class LRUCache:
    def __init__(self, capacity: int):
        self.cache = OrderedDict()
        self.capacity = capacity

    def get(self, key: int) -> int:
        if key not in self.cache:
            return -1

        # Move to end (most recent)
        self.cache.move_to_end(key)
        return self.cache[key]

    def put(self, key: int, value: int) -> None:
        if key in self.cache:
            self.cache.move_to_end(key)
        self.cache[key] = value

        if len(self.cache) > self.capacity:
            # Remove least recently used (first item)
            self.cache.popitem(last=False)
```

Time Complexity: O(1) | Space Complexity: O(capacity)
Idea: Use OrderedDict to maintain order of access.

Solution 2: HashMap + Doubly Linked List

```
class Node:
    def __init__(self, key=0, value=0):
        self.key = key
        self.value = value
        self.prev = None
        self.next = None

class LRUCache:
    def __init__(self, capacity: int):
        self.capacity = capacity
        self.cache = {}  # key -> node
        self.head = Node()  # dummy head (most recent)
        self.tail = Node()  # dummy tail (least recent)
        self.head.next = self.tail
        self.tail.prev = self.head

    def _remove(self, node):
        prev = node.prev
        nxt = node.next
        prev.next = nxt
        nxt.prev = prev

    def _add(self, node):
        # Add after head (most recent)
        node.prev = self.head
        node.next = self.head.next
        self.head.next.prev = node
        self.head.next = node
```

```
    def get(self, key: int) -> int:
        if key not in self.cache:
            return -1

        node = self.cache[key]
        self._remove(node)
        self._add(node)
        return node.value

    def put(self, key: int, value: int) -> None:
        if key in self.cache:
            self._remove(self.cache[key])

        node = Node(key, value)
        self._add(node)
        self.cache[key] = node

        if len(self.cache) > self.capacity:
            # Remove least recently used (before tail)
            lru = self.tail.prev
            self._remove(lru)
            del self.cache[lru.key]
```

Time Complexity: O(1) | Space Complexity: O(capacity)
Idea: HashMap for O(1) access, DLL for O(1) removal/addition.

460. LFU Cache

Design and implement a data structure for a [Least Frequently Used (LFU)]() cache.

Implement the LFUCache class:

- LFUCache(int capacity) Initializes the object with the capacity of the data structure.
- int get(int key) Gets the value of the key if the key exists in the cache. Otherwise, returns -1.
- void put(int key, int value) Update the value of the key if present, or inserts the key if not already present. When the cache reaches its capacity, it should invalidate and remove the **least frequently used** key before inserting a new item. For this problem, when there is a **tie** (i.e., two or more keys with the same frequency), the **least recently used** key would be invalidated.

To determine the least frequently used key, a **use counter** is maintained for each key in the cache. The key with the smallest **use counter** is the least frequently used key.

When a key is first inserted into the cache, its **use counter** is set to 1 (due to the put operation). The **use counter** for a key in the cache is incremented either a get or put operation is called on it.

The functions get and put must each run in O(1) average time complexity.

Example 1:

Input
["LFUCache", "put", "put", "get", "put", "get", "get", "put", "get", "get", "get"]
[[2], [1, 1], [2, 2], [1], [3, 3], [2], [3], [4, 4], [1], [3], [4]]
Output
[null, null, null, 1, null, -1, 3, null, -1, 3, 4]

Explanation
// cnt(x) = the use counter for key x
// cache=[] will show the last used order for tiebreakers (leftmost element is most recent)
LFUCache lfu = new LFUCache(2);
lfu.put(1, 1); // cache=[1,_], cnt(1)=1
lfu.put(2, 2); // cache=[2,1], cnt(2)=1, cnt(1)=1
lfu.get(1); // return 1
// cache=[1,2], cnt(2)=1, cnt(1)=2
lfu.put(3, 3); // 2 is the LFU key because cnt(2)=1 is the smallest, invalidate 2.
// cache=[3,1], cnt(3)=1, cnt(1)=2
lfu.get(2); // return -1 (not found)
lfu.get(3); // return 3
// cache=[3,1], cnt(3)=2, cnt(1)=2
lfu.put(4, 4); // Both 1 and 3 have the same cnt, but 1 is LRU, invalidate 1.
// cache=[4,3], cnt(4)=1, cnt(3)=2
lfu.get(1); // return -1 (not found)
lfu.get(3); // return 3
// cache=[3,4], cnt(4)=1, cnt(3)=3
lfu.get(4); // return 4
// cache=[4,3], cnt(4)=2, cnt(3)=3

Solution: HashMap + Frequency Lists

```
class Node:
    def __init__(self, key=0, value=0, freq=1):
        self.key = key
        self.value = value
        self.freq = freq
        self.prev = None
        self.next = None

class DLL:
    def __init__(self):
        self.head = Node()  # dummy head
        self.tail = Node()  # dummy tail
        self.head.next = self.tail
        self.tail.prev = self.head
        self.size = 0

    def remove(self, node):
        node.prev.next = node.next
        node.next.prev = node.prev
        self.size -= 1

    def add(self, node):
        node.next = self.head.next
        node.prev = self.head
        self.head.next.prev = node
        self.head.next = node
        self.size += 1

    def remove_last(self):
        if self.size > 0:
            node = self.tail.prev
            self.remove(node)
            return node
        return None

class LFUCache:
    def __init__(self, capacity: int):
        self.capacity = capacity
        self.cache = {}  # key -> node
        self.freq_map = {}  # freq -> DLL
        self.min_freq = 0

    def _update(self, node):
        # Remove from current frequency list
        dll = self.freq_map[node.freq]
        dll.remove(node)

        if dll.size == 0 and node.freq == self.min_freq:
            self.min_freq += 1

        # Update frequency
        node.freq += 1

        # Add to new frequency list
        if node.freq not in self.freq_map:
            self.freq_map[node.freq] = DLL()
        self.freq_map[node.freq].add(node)

    def get(self, key: int) -> int:
        if key not in self.cache:
            return -1

        node = self.cache[key]
        self._update(node)
        return node.value

    def put(self, key: int, value: int) -> None:
        if self.capacity == 0:
            return

        if key in self.cache:
            node = self.cache[key]
            node.value = value
            self._update(node)
        else:
            if len(self.cache) >= self.capacity:
                # Remove LFU item
                dll = self.freq_map[self.min_freq]
                node = dll.remove_last()
                del self.cache[node.key]

            # Add new node
            node = Node(key, value, 1)
            self.cache[key] = node

            if 1 not in self.freq_map:
                self.freq_map[1] = DLL()
            self.freq_map[1].add(node)
            self.min_freq = 1
```

Time Complexity: O(1) | Space Complexity: O(capacity)
Idea: Maintain frequency lists, update on access.

155. Min Stack

Design a stack that supports push, pop, top, and retrieving the minimum element in constant time.

Implement the MinStack class:

- MinStack() initializes the stack object.
- void push(int val) pushes the element val onto the stack.
- void pop() removes the element on the top of the stack.
- int top() gets the top element of the stack.
- int getMin() retrieves the minimum element in the stack.

You must implement a solution with O(1) time complexity for each function.

Example 1:

Input
["MinStack","push","push","push","getMin","pop","top","getMin"]
[[],[-2],[0],[-3],[],[],[],[]]

Output
[null,null,null,null,-3,null,0,-2]

Explanation
MinStack minStack = new MinStack();
minStack.push(-2);
minStack.push(0);
minStack.push(-3);
minStack.getMin(); // return -3
minStack.pop();
minStack.top(); // return 0
minStack.getMin(); // return -2

Solution 1: Two Stacks

```
class MinStack:
    def __init__(self):
        self.stack = []
        self.min_stack = []

    def push(self, val: int) -> None:
        self.stack.append(val)
        if not self.min_stack or val <= self.min_stack[-1]:
            self.min_stack.append(val)

    def pop(self) -> None:
        if self.stack:
            val = self.stack.pop()
            if val == self.min_stack[-1]:
                self.min_stack.pop()

    def top(self) -> int:
        return self.stack[-1] if self.stack else None

    def getMin(self) -> int:
        return self.min_stack[-1] if self.min_stack else None
```

Time Complexity: O(1) | Space Complexity: O(n)
Idea: Keep track of minimum at each state.

Solution 2: Single Stack with Pairs

```
class MinStack:
    def __init__(self):
        self.stack = []  # (value, current_min)

    def push(self, val: int) -> None:
        if not self.stack:
            self.stack.append((val, val))
        else:
            current_min = min(val, self.stack[-1][1])
            self.stack.append((val, current_min))

    def pop(self) -> None:
        self.stack.pop()

    def top(self) -> int:
        return self.stack[-1][0]

    def getMin(self) -> int:
        return self.stack[-1][1]
```

Time Complexity: O(1) | Space Complexity: O(n)
Idea: Store (value, min) pairs.

Solution 3: Difference Method (Optimized)

```
class MinStack:
    def __init__(self):
        self.stack = []
        self.min_val = float('inf')

    def push(self, val: int) -> None:
        if not self.stack:
            self.stack.append(0)
            self.min_val = val
        else:
            # Store difference from current min
            self.stack.append(val - self.min_val)
            if val < self.min_val:
                self.min_val = val

    def pop(self) -> None:
        diff = self.stack.pop()
        if diff < 0:
            # This was a new min, restore previous min
            self.min_val = self.min_val - diff

    def top(self) -> int:
        diff = self.stack[-1]
        if diff < 0:
            return self.min_val
        return self.min_val + diff

    def getMin(self) -> int:
        return self.min_val
```

Time Complexity: O(1) | Space Complexity: O(n)
Idea: Store differences to encode min value.

232. Implement Queue using Stacks

Implement a first in first out (FIFO) queue using only two stacks. The implemented queue should support all the functions of a normal queue (push, peek, pop, and empty).

Implement the MyQueue class:

- void push(int x) Pushes element x to the back of the queue.
- int pop() Removes the element from the front of the queue and returns it.
- int peek() Returns the element at the front of the queue.
- boolean empty() Returns true if the queue is empty, false otherwise.

Notes:

- You must use **only** standard operations of a stack, which means only push to top, peek/pop from top, size, and is empty operations are valid.
- Depending on your language, the stack may not be supported natively. You may simulate a stack using a list or deque (double-ended queue) as long as you use only a stack's standard operations.

Example 1:

Input
["MyQueue", "push", "push", "peek", "pop", "empty"]
[[], [1], [2], [], [], []]
Output
[null, null, null, 1, 1, false]

Explanation
MyQueue myQueue = new MyQueue();
myQueue.push(1); // queue is: [1]
myQueue.push(2); // queue is: [1, 2] (leftmost is front of the queue)
myQueue.peek(); // return 1
myQueue.pop(); // return 1, queue is [2]
myQueue.empty(); // return false

Solution: Two Stacks (Amortized O(1))

```
class MyQueue:
    def __init__(self):
        self.input = []
        self.output = []

    def push(self, x: int) -> None:
        self.input.append(x)

    def pop(self) -> int:
        self._transfer()
        return self.output.pop()

    def peek(self) -> int:
        self._transfer()
        return self.output[-1]

    def empty(self) -> bool:
        return not self.input and not self.output

    def _transfer(self):
        if not self.output:
            while self.input:
                self.output.append(self.input.pop())
```

Time Complexity: Amortized O(1) | Space Complexity: O(n)
Idea: Use input stack for push, output stack for pop/peek.

295. Find Median from Data Stream

The **median** is the middle value in an ordered integer list. If the size of the list is even, there is no middle value, and the median is the mean of the two middle values.

- For example, for arr = [2,3,4], the median is 3.
- For example, for arr = [2,3], the median is (2 + 3) / 2 = 2.5.

Implement the MedianFinder class:

- MedianFinder() initializes the MedianFinder object.
- void addNum(int num) adds the integer num from the data stream to the data structure.
- double findMedian() returns the median of all elements so far. Answers within 10^{-5} of the actual answer will be accepted.

Example 1:

Input
["MedianFinder", "addNum", "addNum", "findMedian", "addNum", "findMedian"]
[[], [1], [2], [], [3], []]
Output
[null, null, null, 1.5, null, 2.0]

Explanation
MedianFinder medianFinder = new MedianFinder();
medianFinder.addNum(1); // arr = [1]
medianFinder.addNum(2); // arr = [1, 2]
medianFinder.findMedian(); // return 1.5 (i.e., (1 + 2) / 2)
medianFinder.addNum(3); // arr[1, 2, 3]
medianFinder.findMedian(); // return 2.0

Solution 1: Two Heaps

```
class MedianFinder:
    def __init__(self):
        self.small = []  # max heap (store negatives)
        self.large = []  # min heap

    def addNum(self, num: int) -> None:
        # Add to appropriate heap
        if len(self.small) == len(self.large):
            # Add to large via small
            heapq.heappush(self.small, -num)
            heapq.heappush(self.large,
-heapq.heappop(self.small))
        else:
            # Add to small via large
            heapq.heappush(self.large, num)
            heapq.heappush(self.small,
-heapq.heappop(self.large))

    def findMedian(self) -> float:
        if len(self.small) == len(self.large):
            return (-self.small[0] + self.large[0]) / 2
        else:
            return self.large[0]
```

Time Complexity: O(log n) add, O(1) find | Space Complexity: O(n)
Idea: Maintain two heaps - max for lower half, min for upper half.

Solution 2: Balanced BST (SortedList)

```
class MedianFinder:
    def __init__(self):
        self.nums = []

    def addNum(self, num: int) -> None:
        insort(self.nums, num)

    def findMedian(self) -> float:
        n = len(self.nums)
        if n % 2 == 1:
            return self.nums[n // 2]
        else:
            return (self.nums[n//2 - 1] + self.nums[n//2]) / 2
```

Time Complexity: O(n) insert, O(1) find | Space Complexity: O(n)
Idea: Keep sorted list using binary insertion.

380. Insert Delete GetRandom O(1)

Implement the RandomizedSet class:

- RandomizedSet() Initializes the RandomizedSet object.
- bool insert(int val) Inserts an item val into the set if not present. Returns true if the item was not present, false otherwise.
- bool remove(int val) Removes an item val from the set if present. Returns true if the item was present, false otherwise.
- int getRandom() Returns a random element from the current set of elements (it's guaranteed that at least one element exists when this method is called). Each element must have the **same probability** of being returned.

You must implement the functions of the class such that each function works in **average** O(1) time complexity.

Example 1:

Input
["RandomizedSet", "insert", "remove", "insert", "getRandom", "remove", "insert", "getRandom"]
[[], [1], [2], [2], [], [1], [2], []]
Output
[null, true, false, true, 2, true, false, 2]

Explanation
RandomizedSet randomizedSet = new RandomizedSet();
randomizedSet.insert(1); // Inserts 1 to the set. Returns true as 1 was inserted successfully.
randomizedSet.remove(2); // Returns false as 2 does not exist in the set.
randomizedSet.insert(2); // Inserts 2 to the set, returns true. Set now contains [1,2].
randomizedSet.getRandom(); // getRandom() should return either 1 or 2 randomly.
randomizedSet.remove(1); // Removes 1 from the set, returns true. Set now contains [2].
randomizedSet.insert(2); // 2 was already in the set, so return false.
randomizedSet.getRandom(); // Since 2 is the only number in the set, getRandom() will always return 2.

Solution: HashMap + ArrayList

```
class RandomizedSet:
    def __init__(self):
        self.values = []  # List of values
        self.indices = {}  # value -> index

    def insert(self, val: int) -> bool:
        if val in self.indices:
            return False

        self.indices[val] = len(self.values)
        self.values.append(val)
        return True

    def remove(self, val: int) -> bool:
        if val not in self.indices:
            return False

        # Move last element to val's position
        idx = self.indices[val]
        last_val = self.values[-1]

        self.values[idx] = last_val
        self.indices[last_val] = idx

        # Remove val
        self.values.pop()
        del self.indices[val]
        return True

    def getRandom(self) -> int:
        return random.choice(self.values)
```

Time Complexity: O(1) | Space Complexity: O(n)
Idea: HashMap for O(1) lookup, ArrayList for O(1) random access.

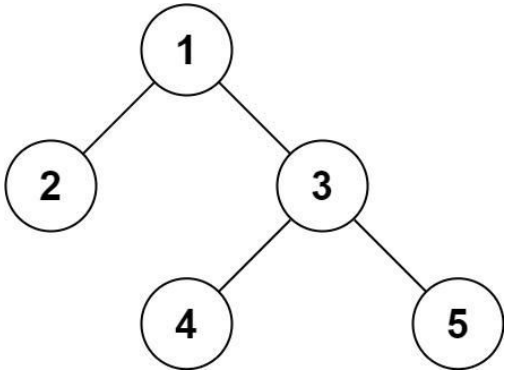
297. Serialize and Deserialize Binary Tree

Serialization is the process of converting a data structure or object into a sequence of bits so that it can be stored in a file or memory buffer, or transmitted across a network connection link to be reconstructed later in the same or another computer environment.

Design an algorithm to serialize and deserialize a binary tree. There is no restriction on how your serialization/deserialization algorithm should work. You just need to ensure that a binary tree can be serialized to a string and this string can be deserialized to the original tree structure.

Clarification: The input/output format is the same as [how LeetCode serializes a binary tree](). You do not necessarily need to follow this format, so please be creative and come up with different approaches yourself.

Example 1:



Input: root = [1,2,3,null,null,4,5]
Output: [1,2,3,null,null,4,5]

Example 2:

Input: root = []
Output: []

Solution 1: Preorder Traversal (DFS)

```
class Codec:
    def serialize(self, root):
        def dfs(node):
            if not node:
                return ['null']
            return [str(node.val)] + dfs(node.left) + dfs(node.right)

        return ','.join(dfs(root))

    def deserialize(self, data):
        def dfs():
            val = next(values)
            if val == 'null':
                return None

            node = TreeNode(int(val))
            node.left = dfs()
            node.right = dfs()
            return node

        values = iter(data.split(','))
        return dfs()
```

Time Complexity: O(n) | Space Complexity: O(n)
Idea: Preorder traversal with null markers.

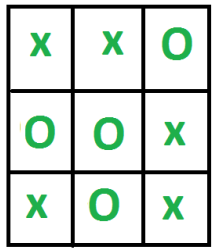
Solution 2: Level Order Traversal (BFS)

```
class Codec:
    def serialize(self, root):
        if not root:
            return ''

        result = []
        queue = deque([root])

        while queue:
            node = queue.popleft()
            if node:
                result.append(str(node.val))
                queue.append(node.left)
                queue.append(node.right)
            else:
                result.append('null')

        return ','.join(result)

    def deserialize(self, data):
        if not data:
            return None

        values = data.split(',')
        root = TreeNode(int(values[0]))
        queue = deque([root])
        i = 1
        while queue and i < len(values):
            node = queue.popleft()

            # Left child
            if values[i] != 'null':
                node.left = TreeNode(int(values[i]))
                queue.append(node.left)
            i += 1

            # Right child
            if i < len(values) and values[i] != 'null':
                node.right = TreeNode(int(values[i]))
                queue.append(node.right)
            i += 1

        return root
```

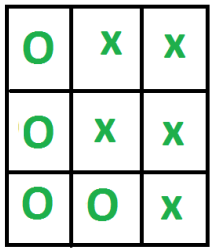
Time Complexity: O(n) | Space Complexity: O(n)
Idea: Level order traversal with null markers.

348. Design Tic-Tac-Toe

A Tic-Tac-Toe board is given after some moves are played. Find out if the given board is valid, i.e., is it possible to reach this board position after some moves or not.
Note that every arbitrary filled grid of 9 spaces isn't valid e.g. a grid filled with 3 X and 6 O isn't valid situation because each player needs to take alternate turns.



Valid Board



Invalid Board
(Both X and O cannot win)

Input is given as a 1D array of size 9.
Examples:
Input: board[] = {'X', 'X', 'O',
'O', 'O', 'X',
'X', 'O', 'X'};
Output: Valid
Input: board[] = {'O', 'X', 'X',
'O', 'X', 'X',
'O', 'O', 'X'};
Output: Invalid
(Both X and O cannot win)

Input: board[] = {'O', 'X', ' ',
' ', ' ', ' ',
' ', ' ', ' '};
Output: Valid
(Valid board with only two moves played)

Solution: Row/Column/Diagonal Tracking

```
class TicTacToe:
    def __init__(self, n: int):
        self.n = n
        self.rows = [0] * n
        self.cols = [0] * n
        self.diag = 0
        self.anti_diag = 0

    def move(self, row: int, col: int, player: int) -> int:
        # Player 1: +1, Player 2: -1
        val = 1 if player == 1 else -1

        self.rows[row] += val
        self.cols[col] += val

        if row == col:
            self.diag += val

        if row + col == self.n - 1:
            self.anti_diag += val

        # Check win
        if abs(self.rows[row]) == self.n or \
           abs(self.cols[col]) == self.n or \
           abs(self.diag) == self.n or \
           abs(self.anti_diag) == self.n:
            return player

        return 0
```

Time Complexity: O(1) | Space Complexity: O(n)
Idea: Track cumulative sums for rows, columns, and diagonals.

355. Design Twitter

Design a simplified version of Twitter where users can post tweets, follow/unfollow another user, and is able to see the 10 most recent tweets in the user's news feed.

Implement the Twitter class:

- Twitter() Initializes your twitter object.
- void postTweet(int userId, int tweetId) Composes a new tweet with ID tweetId by the user userId. Each call to this function will be made with a unique tweetId.
- List<Integer> getNewsFeed(int userId) Retrieves the 10 most recent tweet IDs in the user's news feed. Each item in the news feed must be posted by users who the user followed or by the user themself. Tweets must be **ordered from most recent to least recent**.
- void follow(int followerId, int followeeId) The user with ID followerId started following the user with ID followeeId.
- void unfollow(int followerId, int followeeId) The user with ID followerId started unfollowing the user with ID followeeId.

Example 1:

Input
["Twitter", "postTweet", "getNewsFeed", "follow", "postTweet", "getNewsFeed", "unfollow", "getNewsFeed"]
[[], [1, 5], [1], [1, 2], [2, 6], [1], [1, 2], [1]]
Output
[null, null, [5], null, null, [6, 5], null, [5]]

Explanation
Twitter twitter = new Twitter();
twitter.postTweet(1, 5); // User 1 posts a new tweet (id = 5).
twitter.getNewsFeed(1); // User 1's news feed should return a list with 1 tweet id -> [5]. return [5]
twitter.follow(1, 2); // User 1 follows user 2.
twitter.postTweet(2, 6); // User 2 posts a new tweet (id = 6).
twitter.getNewsFeed(1); // User 1's news feed should return a list with 2 tweet ids -> [6, 5]. Tweet id 6 should precede tweet id 5 because it is posted after tweet id 5.
twitter.unfollow(1, 2); // User 1 unfollows user 2.
twitter.getNewsFeed(1); // User 1's news feed should return a list with 1 tweet id -> [5], since user 1 is no longer following user 2.

Solution: HashMap + Heaps

```
class Twitter:
    def __init__(self):
        self.timestamp = 0
        self.tweets = defaultdict(list)  # userId -> [(timestamp, tweetId)]
        self.following = defaultdict(set)  # userId -> set of followees

    def postTweet(self, userId: int, tweetId: int) -> None:
        self.tweets[userId].append((self.timestamp, tweetId))
        self.timestamp += 1

    def getNewsFeed(self, userId: int) -> List[int]:
        # Get all users to consider
        users = {userId} | self.following[userId]

        # Collect recent tweets from each user
        heap = []
        for user in users:
            if user in self.tweets:
                # Get last 10 tweets from each user
                tweets = self.tweets[user][-10:]
                for timestamp, tweetId in tweets:
                    heapq.heappush(heap, (-timestamp, tweetId))

        # Get top 10 most recent
        result = []
        while heap and len(result) < 10:
            _, tweetId = heapq.heappop(heap)
            result.append(tweetId)

        return result

    def follow(self, followerId: int, followeeId: int) -> None:
        if followerId != followeeId:
            self.following[followerId].add(followeeId)

    def unfollow(self, followerId: int, followeeId: int) -> None:
        self.following[followerId].discard(followeeId)
```

Time Complexity:
- postTweet: O(1)
- getNewsFeed: O(n log n)
- follow/unfollow: O(1)
Space Complexity: O(n)
Idea: Store tweets per user, merge using heap.

588. Design In-Memory File System

Problem Statement: Design in-memory file system with directories and files.

Solution: Tree Structure

```
class FileNode:
    def __init__(self, name, is_file=False):
        self.name = name
        self.is_file = is_file
        self.content = "" if is_file else None
        self.children = {} if not is_file else None

class FileSystem:
    def __init__(self):
        self.root = FileNode("/")

    def _split_path(self, path):
        if path == "/":
            return []
        return path.split("/")[1:]  # Remove empty first element

    def _traverse(self, path_parts):
        node = self.root
        for part in path_parts:
            if part not in node.children:
                node.children[part] = FileNode(part)
            node = node.children[part]
        return node

    def ls(self, path: str) -> List[str]:
        parts = self._split_path(path)
        node = self._traverse(parts) if parts else self.root

        if node.is_file:
            return [node.name]
        return sorted(node.children.keys())

    def mkdir(self, path: str) -> None:
        parts = self._split_path(path)
        self._traverse(parts)  # Creates directories if they don't exist

    def addContentToFile(self, filePath: str, content: str) -> None:
        parts = self._split_path(filePath)
        dir_parts = parts[:-1]
        file_name = parts[-1]

        # Create directories if needed
        dir_node = self._traverse(dir_parts)

        # Create or update file
        if file_name not in dir_node.children:
```

```
            dir_node.children[file_name] =
FileNode(file_name, is_file=True)

        file_node = dir_node.children[file_name]
        file_node.content += content

    def readContentFromFile(self, filePath: str) -> str:
        parts = self._split_path(filePath)
        dir_parts = parts[:-1]
        file_name = parts[-1]

        dir_node = self._traverse(dir_parts)
        if file_name not in dir_node.children or not
dir_node.children[file_name].is_file:
            return ""

        return dir_node.children[file_name].content
```

Time Complexity: O(path_length) for all operations |
Space Complexity: O(n)
Idea: Use tree structure with nodes for files/directories.

These solutions cover various system design problems with multiple approaches, demonstrating efficient data structure combinations and algorithms for real-world scenarios.